考虑蒸发潜热与失水收缩的圆柱药材干燥模型

摘要

针对圆柱药材预热、恒温干燥及失水收缩过程，建立一维轴对称热质耦合模型。四问均在表面热平衡中计入实际水分通量乘汽化潜热，并区分药材干基含水率与空气湿度比，以内部供水能力和气膜传质能力共同限制蒸发。物性严格分别采用题给附录 2、3、4；通过材料坐标变换处理收缩并保持干物质量守恒，以有限体积法和 BDF 隐式积分求解。

问题一使用附件实测环境，1800 s 时中心、表面温度分别为 31.5960、34.0720 °C，含水率分别为 2.5500、2.4483 kg/kg。问题二建立全程变物性模型，前 4 h 保留附件输入，后续主方案依据文献选择 60 °C、20% 相对湿度；3 h 时中心温度为 41.5644 °C、含水率为 2.4411 kg/kg。

问题三以全域最大含水率低于 0.15 kg/kg 为判据，得到连续达标时长 46.9964 h。问题四结合实测半径及附录 4 物性，得到 44.0360 h，终点半径 1.2000 cm。另对附件升温曲线拟合渐近温度，构造 50.0082 °C、20% 相对湿度的第二方案，其问题三、四时长分别为 62.7635、58.2928 h；该温度依赖外推模型，不能视为已观测到的稳态。

对新模型开展空间加密、时间积分交叉、守恒收支及工况对照，并生成四个完整结果工作簿。药材种类与解吸数据缺失导致表面供水闭合仍有结构不确定性，数值收敛不等于物种实验验证。

关键词：药材干燥；蒸发潜热；热质耦合；移动边界；有限体积法

1　问题重述与分析

热风干燥中，外部热量经药材表面传入，内部水分扩散至表面后蒸发；蒸发消耗的潜热又影响温度，而温度与含水率共同决定物性及扩散速率。因此，预测烘干时间必须同时处理径向传热、非线性传质及表面热质耦合。问题四的尺寸收缩还会改变扩散距离、表面积和单位体积的干物质量。

题目给定初始长度 25 cm、半径 2 cm 的近似圆柱药材，初温 28 °C、初始干基含水率 2.55 kg/kg。问题一按附录 2 计算前 1800 s，并在七个时刻、五个径向位置列示温度和含水率；问题二统一使用附录 3 描述两阶段全过程，列示前 3 h 每半小时的状态；问题三根据各处含水率均低于 0.15 kg/kg 确定时长；问题四在附录 4 物性和附件 2 实测半径条件下重新求解 [1]。本文四问均计入水分蒸发潜热。

附件 1 只描述初始阶段环境，不能直接决定随后两三天的工况。本文保留其观测区间内的驱动，另依据公开文献为后续阶段选择温度与相对湿度，明确空气湿度和药材干基含水率的区别，并通过供水与气膜能力共同闭合传质边界。问题三与问题四使用相同的外部工况，使两者结果具有可比性；二者的差异仍同时包含物性公式变化和收缩影响，不能全部归因于收缩。

所有计算采用 m、s、K 与 kg/kg。论文表格分别按题目使用 s 或 h、cm 和 °C，四个工作簿的时间列均为 s。工作簿与论文表格均由同一次计算结果生成，保留四位小数仅表示规定的展示精度。

2　模型假设及代价

1. 药材均质、轴对称，忽略轴向与周向梯度。长度与直径之比为 6.25，两个端面与侧面面积之比为 0.08，因而采用一维径向模型。端面附近的局部快速干燥不能由此准确刻画，本文时长应理解为该径向模型的预测。

2. 固体骨架与液态水局部热平衡，以单一温度场及题给有效物性描述内部导热；忽略辐射、化学反应和内部气相显式迁移。蒸发集中在表面，潜热作为边界能量通量计入，避免在内部再添加同一笔蒸发耗热。

3. 问题一至三尺寸不变，采用恒定参考干物质密度；问题四长度不变、径向均匀收缩，以附件 2 规定半径并保持总干物质量守恒。均匀收缩不能刻画内部裂隙、各向异性或局部塌陷，给定半径也不是模型自主预测的结果。

4. 经验密度 ρ 用于有效热容量，ρ_d 用于含水率与实际水分通量的换算。二者作用不同，不额外强迫题给经验密度同时满足每一时刻的总质量与体积关系；否则会与给定几何约束发生矛盾。

5. 附件数据在相邻时间点间分段线性插值，保存原始测量值且不产生额外极值。附

件覆盖外的环境使用有文献依据的自主工况；所有外推、阶段切换和气膜闭合均在相应章节单独说明。

6. 未给出药材品种、品质劣变阈值、吸附实测曲线和风速场。所选工况与供水上限关系属于明确的模型闭合，预测时长具有条件性，不能据此声称对任何中药材均为品质或能耗最优。

3 符号说明

符号	含义
T	药材温度，内部 K
C	药材干基含水率，kg 水/kg 干物质
t	时间 s
r	物理径向距离 m
R	当前半径 m
R_0	初始半径 m
L	长度 m
ρ	题定经验密度，用于有效热容量
c_p	题定比热 J/(kg K)
k	导热系数 W/(m K)
D	有效水分扩散系数 m^2/s
h	对流换热系数 $W/(m^2\ K)$
h_m	固相基准的有效供水系数 m/s
T_∞	烘房温度 K
Y_∞	空气湿含量 kg 水/kg 干空气；Boundary.C_amb 为兼容旧接口的存储名
Y_{sat}	表面温度下饱和空气湿含量
Y_{int}	反算的界面空气湿含量
φ	相对湿度 0—1
p	空气总压 Pa
p_v	水蒸气分压 Pa
e_s	饱和蒸汽压 Pa
ε	水蒸气/干空气分子量比 0.621945

c_{pm}	每 kg 干空气的湿空气比热 1006+1860Y∞，J/(kg 干空气 K)
Le	Lewis 数，热扩散率/水蒸气扩散率
K_M	气膜湿含量驱动系数 kg 干空气/(m² s)
T_s	药材表面温度
C_s	药材表面干基含水率
ρ_d	参考干物质密度 kg/m³
ρ_{d0}	初始参考干密度 rho(C0,T0)/(1+C0)
j_v	向外相变质量通量 kg/(m² s)，负号为冷凝
j_s	内部供水能力 kg/(m² s)
j_g	气膜排湿能力 kg/(m² s)
ΔH_{vap}	纯水汽化潜热 J/kg
w	湿基水质量分数 C/(1+C)
ξ	材料坐标 r/R
v	材料径向速度 r Rdot/R，m/s
M_d	干物质总质量 kg
M_w	水总质量 kg
M	全内部场最大含水率
t_*	全场达到 0.15 的连续阈值时刻 s
Δt	时间步长 s，必须区分内部与输出
Δr	径向间距 m
N_r	内部径向节点数
V_i	无量纲环形控制体积权重
F^T, F^C	材料域的面通量，见 numerics.md

符号表由项目符号记录自动生成。T 在计算中使用 K，C 为水质量与干物质质量之比（kg/kg），湿基质量分数为 $w = C/(1 + C)$。长度和时间分别使用 m 和 s；h 单位为 $\mathrm{W/(m^2\,K)}$，h_m 单位为 m/s，j_v 单位为 $\mathrm{kg/(m^2\,s)}$，ΔH_{vap} 单位为 J/kg。下标 s 表示当前表面，0 表示初始状态，∞ 表示外部环境。

4　问题一：含蒸发潜热的预热模型

4.1　径向控制方程与量纲一致的表面通量

对单位轴向长度的圆柱环作收支，固定几何下有

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(r k \frac{\partial T}{\partial r} \right), \tag{1}$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(r D \frac{\partial C}{\partial r} \right), \tag{2}$$

其中 $0 < r < R_0$。C 是干基比值，不是水质量体积浓度。引入恒定参考干物质密度

$$\rho_d = \frac{\rho(C_0, T_0)}{1 + C_0}, \qquad j_v = \min\left\{ \rho_d h_m C_s,\ \frac{h}{c_{pm}} \mathrm{Le}^{-2/3} [Y_{\mathrm{sat}}(T_s) - Y_\infty] \right\}. \tag{3}$$

则 j_v 确为 $\mathrm{kg/(m^2\,s)}$ 的向外水分通量。固定域内部水分守恒式的两侧均有 ρ_d，消去后才得到式(2)；不能以湿药材密度乘 C 代替水质量密度。

取表面外法向沿径向增加方向，四问统一采用

$$-\rho_d D \left.\frac{\partial C}{\partial r}\right|_s = j_v, \tag{4}$$

$$k \left.\frac{\partial T}{\partial r}\right|_s = h(T_\infty - T_s) - \Delta H_{vap}(T_s) j_v. \tag{5}$$

环境向表面输入的显热，一部分补充水分汽化潜热，余量向内部传导。式(3)综合内部供水与外部气膜的限制，具体空气湿度换算与文献依据在问题二统一说明。当空气湿度超过表面饱和值时，带符号通量表示冷凝，并在热边界中对应放热；不通过事后裁剪状态掩盖异常。轴心通量为零，即 $\partial T/\partial r = \partial C/\partial r = 0$。初始条件为

$$T(r, 0) = 301.15\ \mathrm{K}, \qquad C(r, 0) = 2.55\ \mathrm{kg/kg}. \tag{6}$$

纯水汽化潜热采用 FAO56 附录 3 式 (3-1)[2]，温度取当前表面温度：

$$\Delta H_{vap}(T_s) = \left[2.501 - 2.361 \times 10^{-3} (T_s - 273.15) \right] \times 10^6 \quad \mathrm{J/kg}. \tag{7}$$

公式中的 $T_s - 273.15$ 为摄氏温度。此式只覆盖水的汽化潜热；低含水率下可能存在的结合水解吸附加焓没有可用材料参数，作为模型局限保留。

4.2　问题一物性与实测边界

第一问严格使用附录 2：

$$\rho = 820\ \mathrm{kg/m^3}, \quad c_p = 2600\ \mathrm{J/(kg\,K)}, \quad k = 0.36\ \mathrm{W/(m\,K)}, \tag{8}$$

$$D = 7 \times 10^{-9} \exp(-0.89/C)\ \mathrm{m^2/s}, \quad h = 25\ \mathrm{W/(m^2\,K)}, \quad h_m = 8 \times 10^{-7}\ \mathrm{m/s}. \tag{9}$$

附件 1 的温度与水分浓度分别构成 T_∞ 及空气湿度比 Y_∞。相邻观测时刻之间分段线性插值，第一问所需时段在原附件范围内，因此不需外推。将环境 kg/kg 按每千克干空气解释，使其只与表面饱和空气湿度比比较；药材干基含水率仅参与内部供水能力，不与空气湿度直接作差。

D 随 C 下降而减小；在空间算子中保留变系数的散度形式，不能用 $D\nabla^2 C$ 遗漏其梯度贡献。此问 D 本身不依赖 T，但表面气膜通量依赖饱和水汽压，温度与含水率仍通过蒸发边界双向耦合；不能把纯对流传热解直接代入。

4.3 统一有限体积与隐式时间积分

令 $\xi = r/R$，在轴心与表面均保留节点，并用

$$\xi_i = 1 - \left(1 - \frac{i}{N_r - 1}\right)^2, \quad i = 0, \ldots, N_r - 1 \tag{10}$$

加密表面低扩散区域。相邻节点中点为内部控制体界面，最内与最外界面分别为 0 和 1；令

$$V_i = \frac{\xi_{i+1/2}^2 - \xi_{i-1/2}^2}{2}. \tag{11}$$

将方程乘以 ξ 后在控制体积分，以相邻节点输运系数的算术平均构造界面通量：

$$F_{i+1/2}^T = \xi_{i+1/2} k_{i+1/2} \frac{T_{i+1} - T_i}{\xi_{i+1} - \xi_i}, \tag{12}$$

$$F_{i+1/2}^C = \xi_{i+1/2} D_{i+1/2} \frac{C_{i+1} - C_i}{\xi_{i+1} - \xi_i}. \tag{13}$$

于是

$$\frac{dT_i}{dt} = \frac{F_{i+1/2}^T - F_{i-1/2}^T}{R^2 V_i \rho_i c_p(C_i, T_i)}, \qquad \frac{dC_i}{dt} = \frac{F_{i+1/2}^C - F_{i-1/2}^C}{R^2 V_i}. \tag{14}$$

轴心界面通量直接置零，表面分别代入 $F_s^T = R[h(T_\infty - T_s) - \Delta H_{vap} j_v]$ 及 $F_s^C = -R j_v/\rho_d$。所有内部界面通量在总量审计中成对抵消，也避免在轴心直接计算 $1/r$。

表面加密使最小单元远小于交付网格，显式格式的稳定步长随最小长度平方缩小。本文采用适用于刚性问题的变阶 BDF 隐式积分与稀疏 Jacobian，在附件边界折点处分段重启，防止自适应步长跳过驱动变化。内部空间与时间精度设置、加密对照结果在第8节统一报告。最终以稠密时间输出及径向插值抽取每 1 s、每 0.1 cm 的交付网格，输出间隔不等于积分器实际步长。

4.4 预热阶段结果

表1、2分别给出题目指定的七个时刻。1800 s 时中心与表面温度为 31.5960 °C、34.0720 °C，含水率为 2.5500、2.4483 kg/kg。初期相变耗热超过对流补热，表面先短暂冷却，随后随环境升温回升。蒸发冷却抑制表面升温，内部水分向外扩散使表层比中心

表 1　30 分钟内药材的温度（°C；距离单位：cm）

时间/s	0	0.5	1	1.5	2
100.0000	27.9957	27.9806	27.8664	27.4303	26.6179
300.0000	27.6866	27.6145	27.4228	27.2174	27.2388
600.0000	27.4691	27.4978	27.6202	27.9424	28.6235
900.0000	28.0307	28.1370	28.4766	29.1077	30.1064
1200.0000	29.0585	29.2027	29.6450	30.4119	31.5600
1500.0000	30.3039	30.4648	30.9494	31.7543	32.8582
1800.0000	31.5960	31.7519	32.2180	32.9919	34.0720

表 2　30 分钟内药材的水分浓度（kg/kg；距离单位：cm）

时间/s	0	0.5	1	1.5	2
100.0000	2.5500	2.5500	2.5500	2.5500	2.5084
300.0000	2.5500	2.5500	2.5500	2.5499	2.5012
600.0000	2.5500	2.5500	2.5500	2.5481	2.4980
900.0000	2.5500	2.5500	2.5500	2.5451	2.4924
1200.0000	2.5500	2.5500	2.5498	2.5418	2.4821
1500.0000	2.5500	2.5500	2.5494	2.5383	2.4667
1800.0000	2.5500	2.5500	2.5488	2.5345	2.4483

更干；两种效应通过式(5)联系。对应工作簿保存 0 至 1800 s 的温度、含水率完整记录。

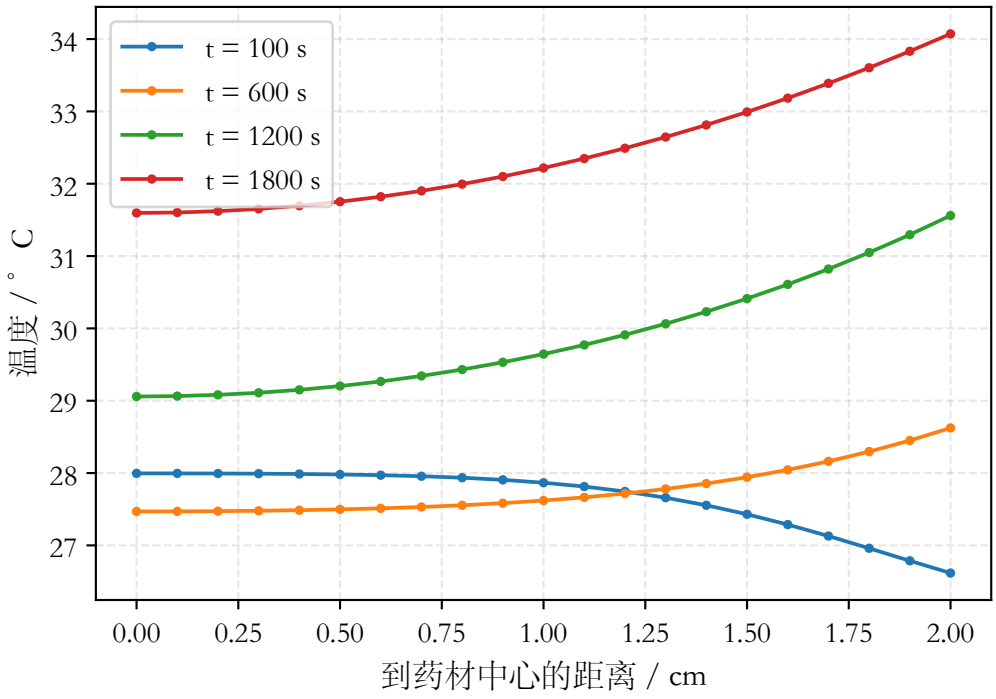


图 1　预热阶段温度的径向分布；由含潜热模型计算

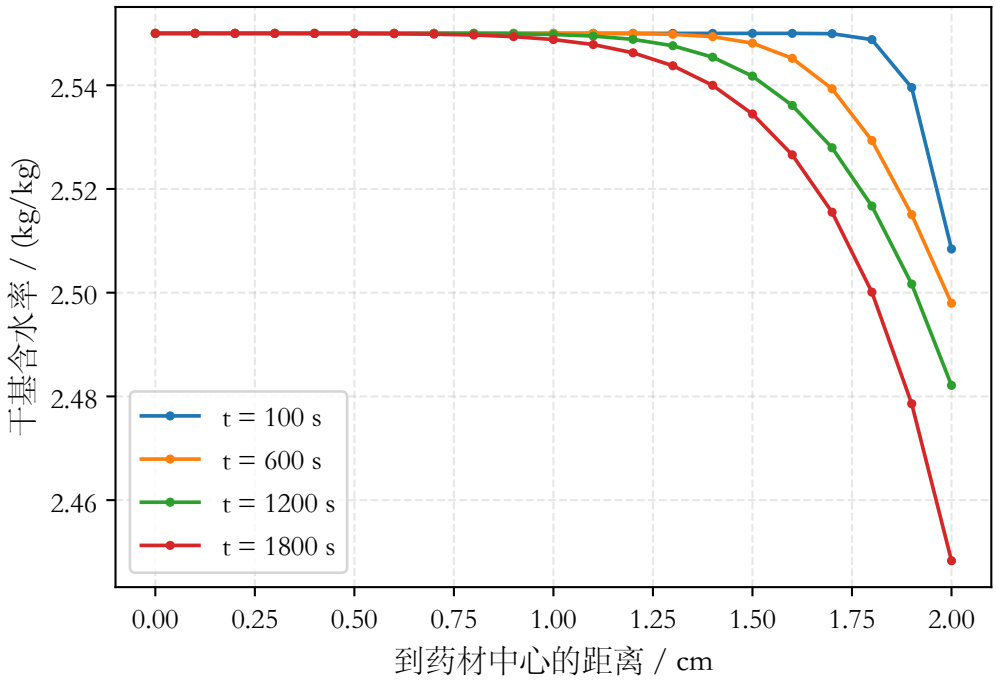


图 2　预热阶段干基含水率的径向分布

5　问题二：自定温湿度下的全程耦合模型

5.1　选取工况及其文献依据

Ju 等在山药片热风干燥研究中采用 60 °C，并设置 20% 相对湿度恒定工况以及分段湿度工况 [3]。该实验为块根类物料选择烘干温湿度提供可追溯参照，但其样品厚度、

风速和材料品种并非本题参数。因此本文自主选择后续恒温干燥工况为

$$T_\infty = 60\ ^\circ\mathrm{C}, \qquad \varphi_\infty = 20\%. \tag{15}$$

这是一组有公开研究依据的可计算工况，并非由该文献证明的本题最优值。题目未给风速、品质目标或能耗价格，故不额外建立缺乏数据支持的品质优化目标。

附件 1 覆盖前 4 h，该区间使用原始烘房温度与空气湿度比，时间点之间线性插值；4 h 以后切换为上述恒定温湿度。此处保留整个观测序列，未在 1800 s 后截断实际已提供的环境数据。温湿度切换时场变量连续，边界通量可改变，数值积分在接点处分段重启。

5.2 由附件升温曲线构造第二温度方案

为检验环境温度选择的影响，另对附件 1 的升温数据拟合具有有限渐近值的曲线。初始空气温度固定为附件首行 $T_{0,\mathrm{air}}$，采用

$$T_{\mathrm{fit}}(t) = S - (S - T_{0,\mathrm{air}})\frac{1+B}{1+B\exp(t/\tau)}, \quad S > T_{0,\mathrm{air}},\ B > 0,\ \tau > 0. \tag{16}$$

其中 S 为渐近温度，B 控制初始形状，τ 为时间尺度；该式严格通过初始温度且单调趋向有限平台。以附件前 3 h 作为训练段、最后 1 h 作为留出段，先比较外推误差，再用完整 0–4 h 数据估计最终参数，避免仅凭全数据拟合优度选择高自由度曲线。该留出段参与候选选择，因而是选模验证集，并非独立于选模的最终测试集。

四个候选曲线的比较见表8。选中模型的渐近温度为 50.0082 °C，形状参数 $B = 1.343000$，时间尺度 $\tau = 1369.1834$ s；完整数据均方根误差为 0.1366 K，前 3 h 训练后预测最后 1 h 的误差为 0.1530 K。采用 500 次、每块 5 个连续观测点的循环移动块残差重采样，在固定初温与所选函数条件下，平台温度的 95% 参数区间为 49.9853–50.0316 °C；它反映条件参数不确定性，不能当作未来烘房必然保持该温度的保证。第二方案仍在前 4 h 使用附件实测边界，之后取拟合渐近温度作为恒定设定值，相对湿度保持文献支持的 20%。

此方案回答“由已观测升温曲线外推后续恒温”的建模设想，主方案则回答“按已有干燥研究自主设置工况”；两者使用相同潜热、传质与收缩模型，以便比较温度输入的影响。

5.3 空气湿度与药材含水率的区分

将附件 1 的环境水分浓度解释为空气湿度比 Y_∞（kg 水/kg 干空气），药材 C 则以干物质为基准，二者不能直接相减。由水汽分压关系 [4]，

$$Y_\infty = 0.621945\frac{\varphi_\infty e_s(T_\infty)}{p - \varphi_\infty e_s(T_\infty)}, \qquad Y_{\mathrm{sat}} = 0.621945\frac{e_s(T_s)}{p - e_s(T_s)}, \tag{17}$$

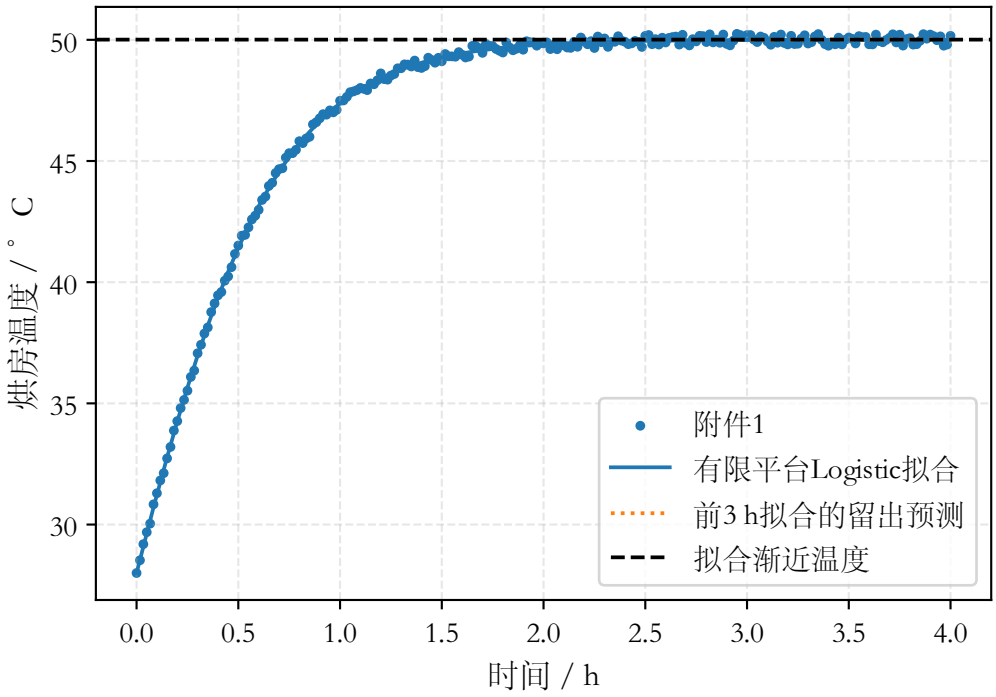


图 3　附件 1 温度观测、有限平台拟合及最后 1 h 的留出预测

其中相对湿度以 0 至 1 的比例输入，总压取标准大气压 $p = 101325$ Pa。饱和水汽压采用 FAO56 公式 [2]

$$e_s(T) = 610.8 \exp\left[\frac{17.27(T - 273.15)}{T - 35.85}\right] \quad \text{Pa}. \tag{18}$$

对应恒温工况的空气湿度比为 0.025473 kg/kg。相同相对湿度在不同温度下对应不同湿度比，敏感性试验改变温度时必须重新换算，不能把 Y_∞ 固定而仍声称相对湿度不变。

5.4　由供水与气膜共同约束的表面蒸发

题目给出干基内部扩散系数和传质系数，但没有药材吸附等温线。若直接令 j_v 正比于 $(C_s - Y_\infty)$ 并乘潜热，便把气固质量基准混为一谈，也没有用气相蒸发能力约束冷却。为闭合边界，分别计算内部供水上限与外部气膜能力：

$$j_s = \rho_d h_m C_s, \tag{19}$$

$$j_g = \frac{h}{c_{pm}} \mathrm{Le}^{-2/3} \left[Y_{\mathrm{sat}}(T_s) - Y_\infty\right], \tag{20}$$

$$j_v = \min(j_s, j_g), \tag{21}$$

其中以每千克干空气为基准的湿空气比热 $c_{pm} = 1006 + 1860 Y_\infty$ J/(kg K)[5]，Lewis 数 $\mathrm{Le} = 0.845$；气膜关系采用热质传递类比 [4]，取低传质率及稀混合物近似，忽略强蒸发吹出对边界层的修正。式(20)中 h/c_{pm} 的单位为 kg/(m^2 s)，故所得 j_v 可直接进入潜热通量。h_m 沿用附录 2 给定的有效供水系数 8×10^{-7} m/s，h 取 25 W/(m^2 K)，用于后续各问时明确视为同一烘房条件的模型设定。

若表面湿润且内部供水充分，气膜能力限制蒸发；若表层逐渐变干，供水上限限制蒸发，避免假定干表面始终有无限饱和水汽供给。当 $Y_{\mathrm{sat}} < Y_{\infty}$ 时，$j_g < 0$，式(21)允许冷凝及相应潜热释放。由于缺少吸附等温线，式(19)相当于以理想干燥界面估算供水上限，不能解释为已识别出该药材的平衡含水率；这是本文最主要的物理闭合不确定性。

将 j_v 统一代入式(4)和(5)，两场通过表面蒸发、温度相关扩散及变热物性相互耦合。采用最小值虽使切换点的一阶导数不连续，但通量本身连续；时间积分依靠自适应误差控制处理机制转换。

5.5 附录 3 变物性及结果

问题二、三在预热与恒温阶段均使用同一套附录 3 公式，而非在阶段间切换附录 2 与附录 3：

$$\rho = 650 + 128C, \qquad c_p = 1450 + 2736\frac{C}{1+C}, \tag{22}$$

$$k = 0.21 + 0.38\frac{C}{1+C}, \qquad D = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T}\right). \tag{23}$$

单位依次为 $\mathrm{kg/m^3}$、$\mathrm{J/(kg\,K)}$、$\mathrm{W/(m\,K)}$ 与 $\mathrm{m^2/s}$。干基 C 与湿基 $C/(1+C)$ 严格区分，Arrhenius 指数中的 T 始终为 K。初始参考干物质密度按本问物性取 $\rho_{d0} = (650 + 128 \times 2.55)/(1 + 2.55)\ \mathrm{kg/m^3}$。初始与轴心条件沿用问题一，表面使用实际水分通量控制的热质条件；内部守恒方程仍为式(1)、(2)。

表 3　3 小时内药材的温度（°C；距离单位：cm）

时间/h	0	0.5	1	1.5	2
0.5000	30.9783	31.1516	31.6717	32.5411	33.7666
1.0000	37.1509	37.2577	37.5696	38.0616	38.6817
1.5000	40.0220	40.0621	40.1787	40.3628	40.5741
2.0000	41.0667	41.0816	41.1257	41.1953	41.2476
2.5000	41.4202	41.4249	41.4382	41.4593	41.5048
3.0000	41.5644	41.5658	41.5724	41.5833	41.5617

题目所需前 3 h、每 0.5 h 分布列于表3、4。3 h 时中心与表面温度分别为 41.5644 °C、41.5617 °C，含水率分别为 2.4411、2.1310 kg/kg。第一问与第二问在相同早期边界下仍可得到不同状态，原因是两问按照题意使用了不同经验物性。完整 `result2.xlsx` 保存每 1 s、每 0.1 cm 的温度与含水率，覆盖前 3 h 并继续至全域达标后的首个整秒。全过程结果与问题三共用同一物理模型，避免由不同初值或不同边界导致两问不一致。

表 4　3 小时内药材的水分浓度（kg/kg；距离单位：cm）

时间/h	0	0.5	1	1.5	2
0.5000	2.5500	2.5499	2.5483	2.5371	2.5011
1.0000	2.5488	2.5471	2.5386	2.5093	2.4243
1.5000	2.5411	2.5355	2.5131	2.4561	2.3385
2.0000	2.5202	2.5095	2.4723	2.3952	2.2621
2.5000	2.4855	2.4707	2.4231	2.3343	2.1956
3.0000	2.4411	2.4238	2.3701	2.2749	2.1310

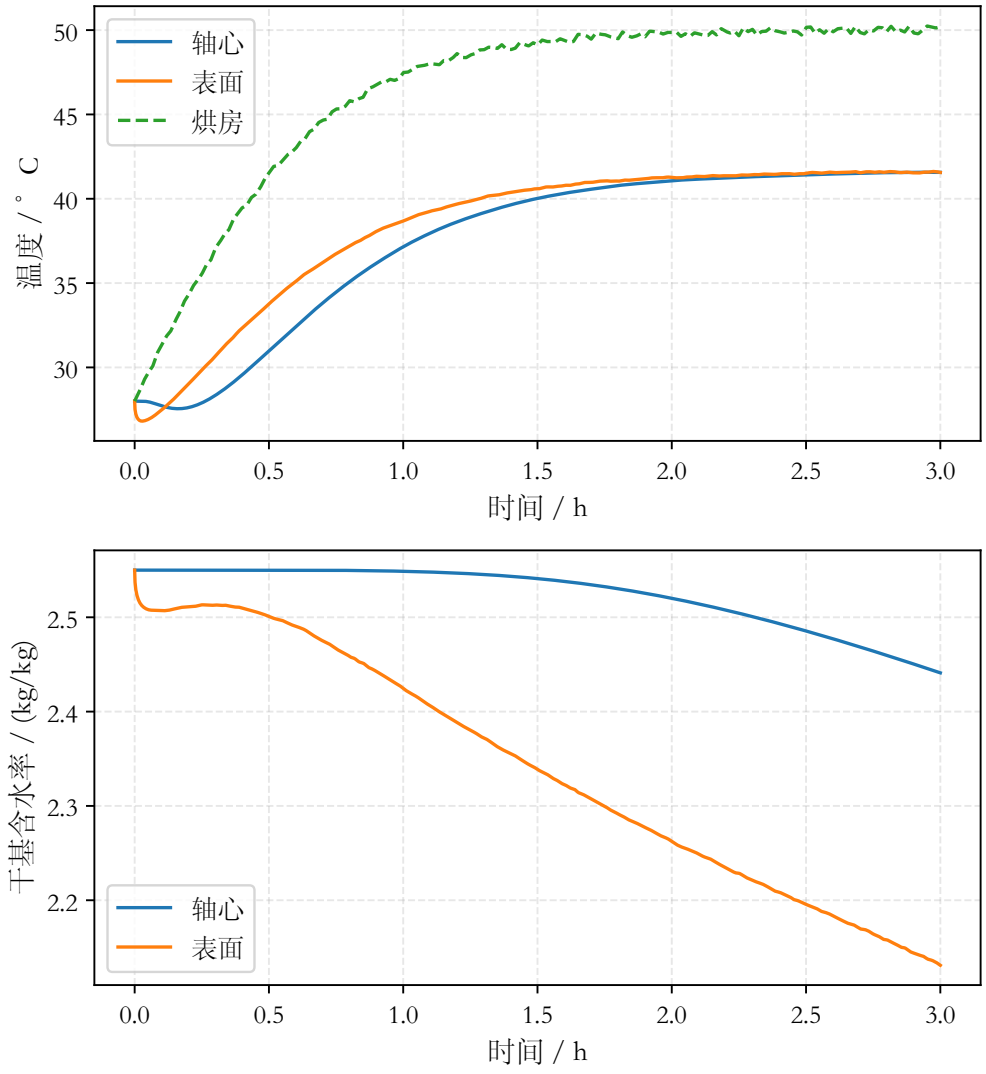


图 4　全程变物性模型在前 3 h 的温度和含水率响应

6 问题三：全域达标时间

6.1 以最大含水率定义终点

问题三沿用问题二的附录 3 参数、初始状态及整个温湿度历程。题意要求各处含水率低于 0.15 kg/kg，因此既不能用平均含水率，也不能只检查表面。定义

$$M(t) = \max_{0 \le r \le R_0} C(r,t), \qquad t_* = \inf\{t : M(t) < 0.15\}. \tag{24}$$

用积分器的连续插值定位 $M - 0.15$ 由正向负穿越的零点。对连续单调干燥过程，零点处恰等于阈值，t_* 是严格达标时间集合的下确界；实际加工需取其后时刻。程序检查全部内部节点，因而无需预先假定最慢位置一定是中心。

工作簿按固定 60 s 网格输出，为同时满足严格不等式与均匀时间列，将积分延续到事件之后的首个 60 s 整点，并以未舍入结果复核全域严格小于阈值。论文末行展示连续事件时刻，含水率按四位小数显示为 0.1500 并不表示程序用 $\le$ 替代了题意。

6.2 结果及干燥速率解释

在选定工况下，连续达标时长为 **46.9964 h**，首个严格合格的 60 s 输出时刻为 47.0000 h。题目要求的每 6 h 分布及结束状态见表5。 随着干燥推进，表面向外失水而

表 5　药材烘干过程的水分浓度（kg/kg；距离单位：cm）

时间/h	0	0.5	1	1.5	2
6.0000	2.0302	1.9695	1.7782	1.4393	0.9434
12.0000	0.6840	0.6631	0.5998	0.4895	0.2978
18.0000	0.3265	0.3181	0.2917	0.2421	0.0604
24.0000	0.2326	0.2277	0.2122	0.1820	0.0214
30.0000	0.1940	0.1904	0.1789	0.1559	0.0111
36.0000	0.1725	0.1695	0.1600	0.1410	0.0070
42.0000	0.1585	0.1559	0.1477	0.1310	0.0049
46.9964	0.1500	0.1477	0.1402	0.1249	0.0038

中心依赖内部扩散补给，径向水分差逐渐成为限制环节。附录 3 中的 $\exp(-0.45/C)$ 使后期扩散系数随含水率下降而迅速减小；同时蒸发消耗热量，温度通过 $\exp(-3850/T)$ 进一步影响传质。因此，前几小时的平均下降速率不能线性外推至终点。图5中的最大含水率轨迹用于判断达标，表面轨迹说明仅观察表面会过早判定完成。上述时长是所选温湿工况、有效传质系数及表面通量闭合共同决定的条件性预测，不把题干的“一般持续 2–3 天”当作拟合目标或强制截止值。

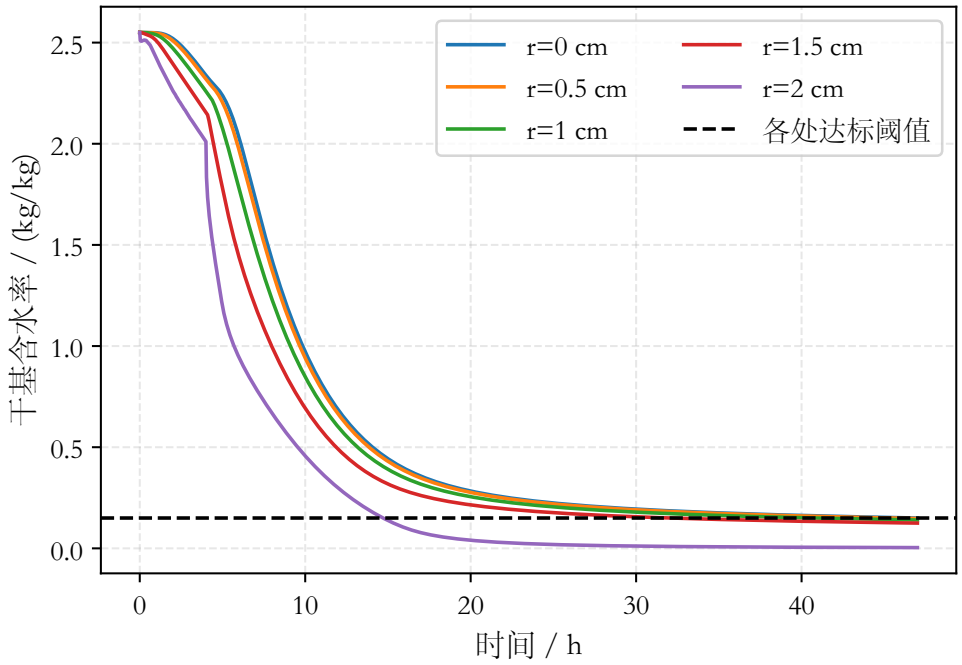


图 5　全域干燥过程及 0.15 kg/kg 终点阈值

7　问题四：失水收缩下的移动边界

7.1　半径数据与守恒的材料坐标

附件 2 给出各时刻半径，以分段线性插值构造 $R(t)$，不改动原始数据、不用高阶拟合制造半径回升。长度保持不变，取均匀径向收缩速度

$$v(r,t)=\frac{\dot R}{R}r,\qquad \rho_d(t)=\rho_d(0)\left(\frac{R_0}{R(t)}\right)^2. \tag{25}$$

于是 $\rho_d\pi R^2L$ 保持不变，干物质不会因网格缩小凭空消失。水分质量守恒使用对固体材料的导数，

$$\frac{\partial C}{\partial t}+v\frac{\partial C}{\partial r}=\frac{1}{r}\frac{\partial}{\partial r}\left(rD\frac{\partial C}{\partial r}\right). \tag{26}$$

令 $\xi=r/R(t)$，对 $\widetilde C(\xi,t)=C(\xi R(t),t)$ 有

$$\left.\frac{\partial C}{\partial t}\right|_r=\left.\frac{\partial \widetilde C}{\partial t}\right|_\xi-\frac{\xi\dot R}{R}\frac{\partial \widetilde C}{\partial \xi},\qquad v\frac{\partial C}{\partial r}=\frac{\xi\dot R}{R}\frac{\partial \widetilde C}{\partial \xi}. \tag{27}$$

两项抵消，得到固定材料域 $0<\xi<1$ 上的方程。下文省去波浪号：

$$\rho c_p\frac{\partial T}{\partial t}=\frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left(\xi k\frac{\partial T}{\partial \xi}\right), \tag{28}$$

$$\frac{\partial C}{\partial t}=\frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left(\xi D\frac{\partial C}{\partial \xi}\right). \tag{29}$$

温度采用随骨架运动的有效热容量模型，忽略应力功；式(28)不应再额外加入一次坐标对流项。仅把固定域扩散方程的半径改小而保留固定 ρ_d，将不能正确计算单位面积的实际蒸发潜热。

表面条件在 $\xi = 1$ 处为

$$-\frac{\rho_d D}{R}\frac{\partial C}{\partial \xi} = j_v, \tag{30}$$

$$\frac{k}{R}\frac{\partial T}{\partial \xi} = h(T_\infty - T_s) - \Delta H_{vap} j_v. \tag{31}$$

其中 j_v 仍由式(21)确定，内部供水能力中的 ρ_d 逐时更新；$\xi = 0$ 仍为零通量。该形式与第一问有限体积离散一致，只需逐时更新 R、ρ_d 和附录 4 物性。

7.2 附录 4 参数与移动区域的输出

第四问使用独立的附录 4 经验式：

$$\rho = 760 + 90C, \qquad c_p = 1850 + 2150\frac{C}{1+C}, \tag{32}$$

$$k = 0.12 + 0.20\frac{C}{1+C}, \qquad D = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T}\right). \tag{33}$$

本问 $\rho_{d0} = (760 + 90 \times 2.55)/(1 + 2.55)$ kg/m^3。各式依次采用 kg/m^3、J/(kg K)、W/(m K) 与 m^2/s，T 必须用 K。环境及换热、传质系数沿用前述自主工况，终点仍由全域最大含水率判断。

每个输出时刻，固定物理位置 r 映射为 $\xi = r/R(t)$ 后插值；当 $r > R(t)$ 时，该处已经位于药材外部，不能填入表面浓度或虚构药材状态。因此结果表中的相应单元格留空，末列“药材表面”始终取当前 $\xi = 1$ 的值。表6中空白只表示几何域外，内部有效位置均有有限数值。完整工作簿采用同一规则，前 20 列为 0 至 1.9 cm，最后一列为动态表面。

7.3 收缩干燥结果

考虑收缩与附录 4 物性后，连续达标时长为 **44.0360 h**，此时半径为 1.2000 cm；首个严格合格的 60 s 输出时刻为 44.0500 h。按每 6 h 及终点输出的分布见表6。 收缩使内部扩散距离缩短，同时提高单位当前体积的干物质量，后者通过式(5)增加给定干基表面通量对应的潜热需求。附录 4 的扩散与热物性又不同于附录 3，因此问题三、四时长差为这些作用的综合结果。本文不将两问的全部差值解释成单独的收缩加速效应。

表 6　考虑收缩的药材烘干过程的水分浓度（kg/kg；距离单位：cm）

时间/h	0	0.5	1	1.5	药材表面
6.0000	2.4209	2.3149	1.7826		0.7059
12.0000	1.2718	1.1023	0.6297		0.2357
18.0000	0.5322	0.4732	0.2908		0.0798
24.0000	0.3008	0.2745	0.1856		0.0266
30.0000	0.2181	0.2020	0.1447		0.0119
36.0000	0.1786	0.1669	0.1237		0.0067
42.0000	0.1557	0.1464	0.1111		0.0043
44.0360	0.1500	0.1412	0.1078		0.0037

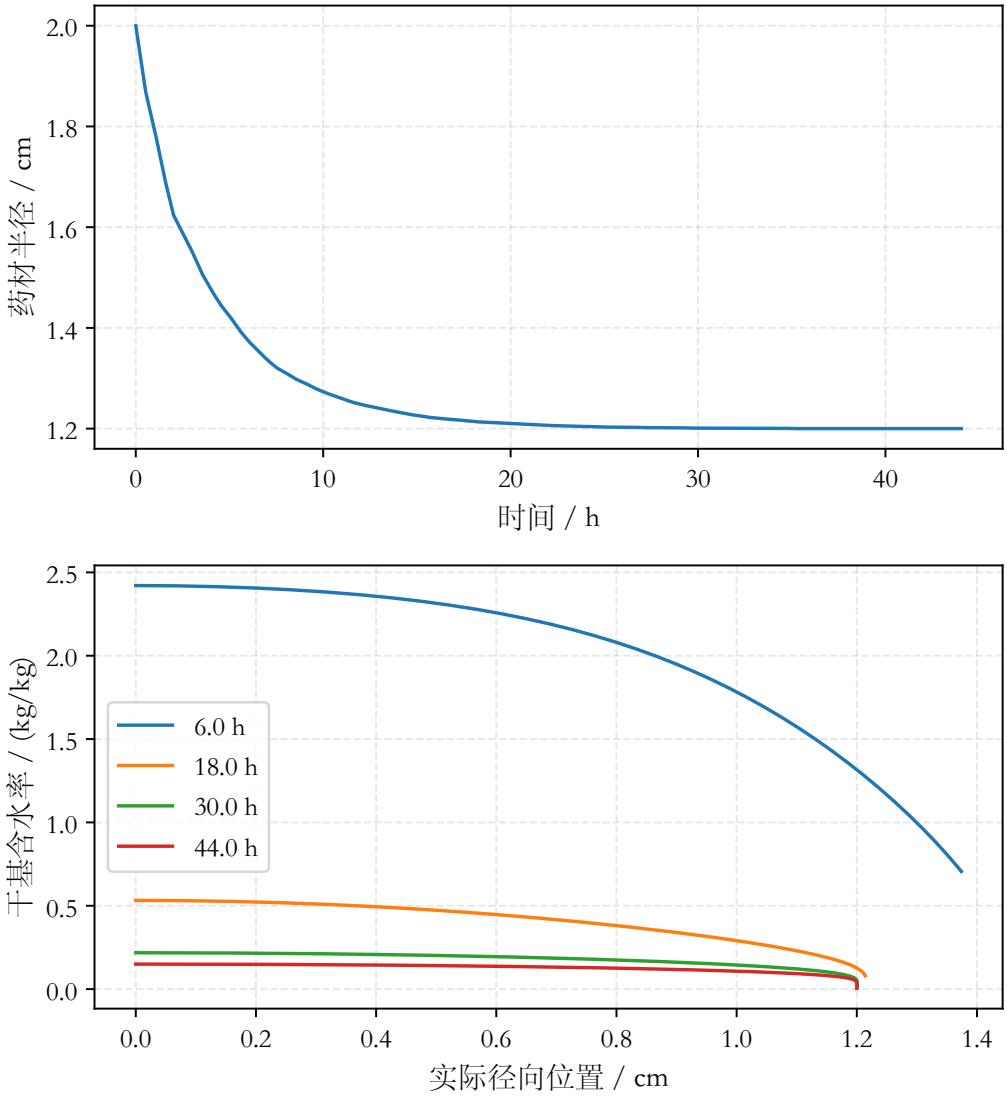


图 6　给定半径收缩历程及移动边界下的干燥响应

8　数值验证、敏感性与模型评价

8.1　独立于输出格式的质量与能量审计

离散内部通量成对抵消，累计表面真实水分通量可得到统一质量审计：

$$\sum_i V_i C_i(t) + \int_0^t \frac{j_v(s)}{\rho_d(s) R(s)}\, ds = \sum_i V_i C_i(0). \tag{34}$$

问题四中 $\rho_d R^2$ 为常数，故式(34)乘以 $2\pi L \rho_d R^2$ 即可还原真实水质量收支。累计通量与状态一起积分，核验在原始未舍入场上进行，不以表格四舍五入后的数值作守恒判断。

问题一的常热容量允许核验累计显热收支：

$$\rho c_p R_0^2 \sum_i V_i [T_i(t) - T_i(0)] = \int_0^t R_0 [h(T_\infty - T_s) - \Delta H_{vap} j_v]\, ds. \tag{35}$$

问题二至四的经验热容量随状态改变，应核验有效热方程的瞬时离散收支

$$R^2 \sum_i V_i \rho_i c_p(C_i, T_i) \frac{dT_i}{dt} = R[h(T_\infty - T_s) - \Delta H_{vap} j_v]. \tag{36}$$

不能把 $\rho c_p T$ 简单积分并宣称总焓守恒，因为 ρc_p 随 C 变化，且模型没有完整追踪水相显焓输运与材料收缩功。质量审计和式(36)检验的是已声明模型的正确实现。

8.2　空间加密与时间积分交叉验证

正式计算采用 1281 个表面加密节点，相对容差 `2.0e-10`、绝对容差 `2.0e-11`，最大内部步长 60 s；前 4 h 最大步长不超过 30 s，并在附件折点重启。针对本次含潜热和气膜限制模型重新执行验证，前一版无潜热模型的误差报告不作为本次依据。 加密试验对问题一、三在相同物理输出位置比较场值，对问题四在相同材料坐标通过插值对齐网格，比较温度、含水率及连续达标时间。场差采样包括问题一每 10 s，问题三、四前 3 h 每 60 s 及 4–39 h 每小时，并另比较连续事件；表中最大差只针对这些核验时空点，不宣称是全流程连续误差的严格上界。时间积分交叉试验在相同 81 节点网格、相同容差与步长上限下，分别采用 BDF 和 Radau 两种隐式方法；它检验时间积分的一致性，不等价于正式细网格上的严格时间误差上界。两方法在核验采样点的最大温度差为 `6.10e-08` K，最大含水率差为 `3.08e-09` kg/kg，连续终点差最大为 `5.45e-07` h。问题四在接近表面的极薄低扩散层中，材料域最大含水率差没有表现出统一二阶收敛，误差峰值随网格加密向表面移动；因此不能声称全内部场均达到四位小数精度。固定物理交付位置的网格差较小，最新对照最大含水率差为 `9.24e-06` kg/kg，仍应与四舍五入末位分开理解。终点是后期低扩散区的累计结果，除比较场变量外还必须检查终点变化，不能只用短时温度误差替代长时干燥精度。

式(34)对应的四问最大相对残差为 `5.05e-15`；第一问累计能量相对残差为 `1.02e-13`。对均匀平衡、热流方向、冷凝符号、潜热与质量使用同一通量以及移动边

表 7　相邻加密网格比较（最大场差与连续终点差）

题号	节点数	温度差/K	含水率差	终点差/s
Q1	641.0000	1.94e-05	2.63e-06	—
Q1	1281.0000	4.44e-06	6.57e-07	—
Q1	2561.0000	1.24e-06	1.64e-07	—
Q3	641.0000	5.17e-05	8.71e-06	2.9828
Q3	1281.0000	1.44e-05	2.15e-06	0.8319
Q3	2561.0000	3.85e-06	6.14e-07	0.2292
Q4	641.0000	1.14e-04	2.59e-03	1.6614
Q4	1281.0000	2.85e-05	2.20e-03	0.4637
Q4	2561.0000	7.14e-06	1.88e-03	0.1271

界外位置处理开展单独检查。另在关闭潜热的纯热极限下对照圆柱 Bessel 级数解，该解析测试只核验导热离散，不代替耦合边界验证。数值误差、物理闭合误差与四位小数展示要求分别报告，不由机器误差很小推断实际药材预测同样精确。

8.3　工况与收缩的对照

表 8　附件温度曲线的拟合候选比较

候选模型	平台/°C	全段 RMSE/K	留出 RMSE/K
自由初值指数	50.1533	0.2658	0.2921
固定初值指数	50.2121	0.3359	0.3719
有限平台 Logistic	50.0082	0.1366	0.1530
Weibull 饱和曲线	50.0164	0.1442	0.1557

整根西洋参研究覆盖 55 °C，并提出先高湿后低湿的策略 [6]。本文据此增加后续 55 °C、20% 相对湿度情形，与 60 °C 基准对照；前 4 h 保持同一附件输入，改变温度时重新换算湿度比。另保持附录 4 的全部物性和外部工况，仅固定半径为初始值，以区分几何收缩效应与两问参数集差异。 在同一 161 节点对照设置下，保持附录 4 物性而固定初始半径的时长为 98.1902 h；给定收缩历程使时长缩短 55.15%。这一比较只改变几何，因而比直接比较问题三、四更能分离收缩作用。 对拟合温度方案，641 与 1281 节点在每 60 s 交付位置上的含水率最大差为 `5.01e-04` kg/kg，连续终点最大差为 1.3273 s。第四问固定位置即将离开药材时穿过极薄表层，因此个别交付位置差值仍高于第四位小数，不能保证两方案所有单元格的末位稳定。附件拟合的第二温度方案对应问题三、四

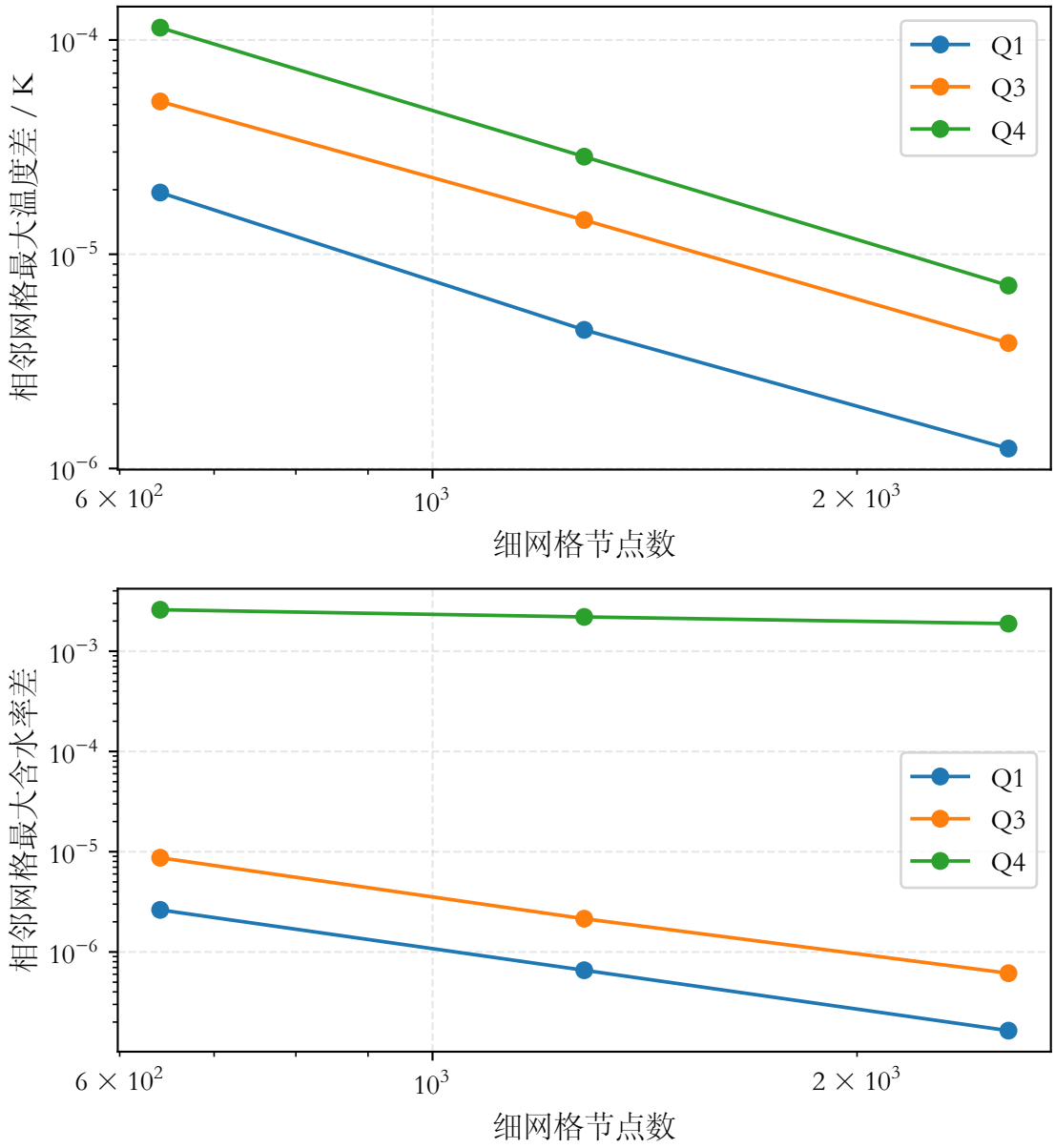


图 7　本次潜热模型的空间加密差值

达标时长分别为 62.7635 h 和 58.2928 h。该差异说明，对短时环境观测的外推方式能够显著影响长时干燥预测。平台参数区间仅约束已选函数内部的估计；即使区间很窄，真实设备在 4 h 以后改动设定或通风条件，仍会使该外推失效。这些对照是模型内部的敏感性计算，不能代替药材品质试验。若降低温度使时长延长，其原因同时包括温度相关扩散减弱、表面饱和蒸汽压变化以及潜热反馈。是否值得以更长时间换取成分保持率，仍需具体药材的实测品质指标。

8.4　模型的适用边界

模型的优点是热、质边界使用同一实际水分通量，质量基准和单位可追溯；三套附录物性分别管理，收缩采用材料坐标并保持干物质量守恒；题目要求的六张表、四个工

表 9　工况与结构假设敏感性（161 节点，全部为方案计算）

题号	比较条件	时间/h	相对基准差/h
Q3	55°C、20% RH	54.0426	7.0502
Q3	60°C、40% RH	48.1102	1.1178
Q3	Lewis 数为 1	47.0279	0.0355
Q4	55°C、20% RH	50.4376	6.4038
Q4	60°C、40% RH	46.1753	2.1415
Q4	Lewis 数为 1	44.0849	0.0511
Q3	同参数不计潜热	42.4122	-4.5802
Q4	同参数不计潜热	37.6779	-6.3560
Q4	附录 4 固定半径	98.1902	54.1564

表 10　两种恒温温度来源的比较（相对湿度均为 20%）

恒温方案	温度/°C	Q3 时间/h	Q4 时间/h
文献设定	60.0000	46.9964	44.0360
附件拟合渐近值	50.0082	62.7635	58.2928

作簿和论文关键数字由同一计算链生成。

局限集中在三方面：一维模型忽略端面与局部结构；给定半径只驱动收缩而不预测其力学机制；气膜和内部供水的互补闭合缺少该药材吸附等温线标定，低含水率下还忽略结合水的附加解吸热。已有山药解吸研究表明等温线和吸附热依赖具体材料与含水率[7]，故不能借用未经核验的异物种参数填补本题。后续应优先测量表面温度、质量损失与解吸等温线以检验闭合，再用轴向模型或多孔介质模型细化结构。

9　结论

本文在全部四问的表面热平衡中计入水分蒸发潜热，以气膜蒸发能力和内部供水上限共同限制水分通量，并严格区分空气湿度比与药材干基含水率。第一问得到前 1800 s 的径向温度和含水率，第二问用附录 3 变物性贯通预热与后续恒温过程，给出前 3 h 的指定结果及全程记录。

依据公开干燥研究选取后续 60 °C、20% 相对湿度；在此条件下，问题三的连续达标时长为 46.9964 h。问题四将附件 2 半径转化为材料坐标，保持干物质量守恒并采用附录 4 物性，得到达标时长 44.0360 h，终点半径 1.2000 cm。工作簿采用原模板要求的

表名、时间单位与距离网格，第四问动态表面单独记录，域外位置留空。

对附件升温曲线拟合得渐近温度 50.0082 °C，保持 20% 相对湿度的第二方案对应问题三、四时长为 62.7635 h、58.2928 h。该比较量化了后续温度设定对干燥时间的影响；拟合渐近值是模型外推结果，不能视为 4 h 后已实测平台。两问时长同时受物性、几何、温湿工况及表面闭合影响。缺少材料吸附曲线和实测内部场时，网格收敛与守恒只能证明数值实现可靠，不能消除模型结构误差；本文结果是条件清楚、能够重算的数值预测。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 A 题：药材的烘干问题及附件 1–3[Z]. 2026.

[2] Allen R G, Pereira L S, Raes D, Smith M. Crop evapotranspiration: Guidelines for computing crop water requirements[M]. FAO Irrigation and Drainage Paper 56. Rome: Food and Agriculture Organization of the United Nations, 1998. ISBN 92-5-104219-5. 公式 (11)、附录 3 式 (3-1). https://www.fao.org/4/x0490e/x0490e00.htm.

[3] Ju H Y, Zhang Q, Mujumdar A S, et al. Hot-air drying kinetics of yam slices under step change in relative humidity[J]. International Journal of Food Engineering, 2016, 12(8): 783–792. DOI: 10.1515/ijfe-2015-0340.

[4] ASHRAE. 2021 ASHRAE Handbook—Fundamentals, SI Edition[M/OL]. Atlanta: ASHRAE, 2021. Chapter 1: Psychrometrics; Chapter 6: Mass Transfer. https://handbook.ashrae.org/Handbooks/F21/SI/F21_Ch06/F21_Ch06_si.aspx（访问日期：2026-09-11）.

[5] Meyer D, Thevenard D. PsychroLib: A library of psychrometric functions to calculate thermodynamic properties of air[J]. Journal of Open Source Software, 2019, 4(33): 1137. DOI: 10.21105/joss.01137.

[6] Ju H Y, Zhao S H, Mujumdar A S, et al. Step-down relative humidity convective air drying strategy to enhance drying kinetics, efficiency, and quality of American ginseng root (*Panax quinquefolium*)[J]. Drying Technology, 2020, 38(7): 903–916. DOI: 10.1080/07373937.2019.1597373.

[7] Njie D N. Desorption isotherms of water in trifoliate yam (*Dioscorea dumetorum* Pax.)[J]. Drying Technology, 1999, 17(6): 1161–1171. DOI: 10.1080/07373939908917601.

A　支撑材料与复现说明

A.1　文件用途与运行方法

支撑材料包为 `cumcm2026A_support.zip`。主要文件包括：`src/`下的物性、空气湿度、边界、有限体积求解及导出模块；`scripts/`下的求解、验证和图表生成入口；`tests/`下的核验程序；`results/result1.xlsx` 至 `result4.xlsx`、数值摘要和验证报告；`paper/`下的论文源文件、自动生成表格与矢量插图；`docs/model/`下的模型规格和文献依据。具体清单随支撑包附带，原始题目附件按目录 `problem/`读取。

本次使用 Python、NumPy、SciPy、pandas、openpyxl 和 Matplotlib 完成求解与结果输出，以 XeLaTeX 编译中文论文。已配置环境时，在项目根目录执行：

```
conda run -n 26A python scripts/run_pipeline.py
```

该入口依次生成主方案、附件拟合方案、验证报告、论文表图、源码附录、PDF 与支撑包。单独计算四问可执行 `python run_all.py`；单独编译论文使用 `latexmk -xelatex`，输出目录指定为 `paper/build`。所有数值先以 SI 单位计算，写出时再换算；论文表格从结果工作簿或同一次计算的连续终点记录读取，摘要和正文关键数字通过自动生成的宏引用。原始题目数据不修改，失败的积分和格式校验会中止计算。

A.2　AI 辅助使用说明

本解题过程使用 AI 辅助核对题面与格式、检索文献、推导和审查模型、编写 Python 程序及组织 LaTeX 正文。论文中数值由本地程序实际运行得到，文献按可核验来源列示；AI 生成的数值建议不直接作为结果。此次生成不包含参赛身份、学校、签名或任何提交操作。

A.3　完整源程序

以下源程序从项目实际文件直接载入；相同代码也以可运行的文本文件提供。论文排版中的自动换行不改变支撑包内的文件内容。

`run_all.py`

```
"""四问正式复现入口：先预检依赖和模板，再计算及输出。"""

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent / "scripts"))

```

```
from run_q123_accuracy import main as compute_main


def main():
    code = compute_main(("q1", "q3", "q4", "q2"))
    return code

if __name__ == "__main__":
    raise SystemExit(main())
```

src/common/boundary.py

```
"""附件驱动边界；输入读取时统一转SI，外推假设见 D-013。"""

from dataclasses import dataclass

import numpy as np

from .io_excel import read_attachment
from .units import c_to_k, cm_to_m
from .psychrometrics import humidity_ratio


@dataclass
class Boundary:
    ambient_t: np.ndarray
    T_amb: np.ndarray
    C_amb: np.ndarray
    radius_t: np.ndarray
    R: np.ndarray
    late_T_shift: float = 0.0
    late_C_shift: float = 0.0
    late_T: float | None = None
    late_Y: float | None = None
    description: str = "attachment 1 humidity interpreted as kg water/kg dry air"

    @classmethod
    def for_problem(cls, problem, temperature_c=60.0, relative_humidity=0.20):
        if problem == "q1":
            return cls.from_attachments()
        T = float(c_to_k(temperature_c))
        return cls.from_attachments(late_T=T, late_Y=float(humidity_ratio(T,
    relative_humidity)),
                                    description=f"attachment 1 through 4 h; then {
```

```
    temperature_c:g} degC, {100*relative_humidity:g}% RH at 101325 Pa")

    @classmethod
    def from_attachments(cls, **kwargs):
        a = read_attachment(1)
        b = read_attachment(2)
        return cls(a.iloc[:, 0].to_numpy(float), c_to_k(a.iloc[:, 1].to_numpy(
    float)),
                   a.iloc[:, 2].to_numpy(float, copy=True), b.iloc[:, 0].to_numpy(
    float),
                   cm_to_m(b.iloc[:, 1].to_numpy(float)), **kwargs)

    def ambient(self, t):
        late = np.asarray(t) > self.ambient_t[-1]
        T = np.interp(t, self.ambient_t, self.T_amb)
        Y = np.interp(t, self.ambient_t, self.C_amb)
        if self.late_T is not None:
            T = np.where(late, self.late_T, T)
        if self.late_Y is not None:
            Y = np.where(late, self.late_Y, Y)
        return T + late * self.late_T_shift, Y + late * self.late_C_shift

    def radius(self, t):
        if np.any(np.asarray(t) > self.radius_t[-1] + 1e-7):
            raise ValueError("超过附件2的72小时范围；不得自动外推收缩半径")
        return np.interp(t, self.radius_t, self.R)
```

src/common/correlations.py

```
"""物性公式（附录 2 / 3 / 4 三套）。

铁律（AGENTS.md 铁律 5）：ρ、c_p、k、D 一律通过本模块获取。
在 src/q*/ 里直接写常数（如 0.36）属于严重错误。

⚠ 当前状态：**接口已冻结，实现待填**。
   公式请从 problem/A题.pdf 的附录逐字抄录到 docs/model/correlations.md，
   再由 @impl 实现于此。本文件不预先填入任何公式，防止与原文不一致。

接口约定（不要改）：
    - C 一律是**干基**含水率 (kg/kg)。公式里需要湿基 w = C/(1+C) 时，在函数内部换算，
      不要把这个负担丢给调用方 —— 否则必然有人在某处传错。
    - T 一律是**开尔文** (K)。附录 3/4 的 D 公式含 exp(-3850/T)。
    - 返回单位见各方法 docstring。
"""
```

```python

from __future__ import annotations

from typing import Protocol, runtime_checkable

import numpy as np
from numpy.typing import ArrayLike

from . import units

#: 附录 3/4 的 D 公式含 exp(-3850/T)，T 必须 > 200 K 才可能是开尔文
_KELVIN_CHECK = True


def wet_basis(C: ArrayLike) -> np.ndarray:
    """干基含水率 C (kg/kg) → 湿基质量分数 w = C/(1+C) (kg/kg)。

    附录 3/4 的 c_p、k 公式里出现的 C/(C+1) 就是这个量。
    见 docs/model/correlations.md §1。
    """
    C = np.asarray(C, dtype=float)
    return C / (1.0 + C)


def _prepare(C: ArrayLike, T: ArrayLike, *, where: str) -> tuple[np.ndarray, np.ndarray]:
    """统一入参检查。所有 PropertySet 方法都应先调用它。"""
    C_arr = np.asarray(C, dtype=float)
    T_arr = np.asarray(T, dtype=float)
    if np.any(C_arr < 0.0):
        raise ValueError(f"水分浓度 C 出现负值{_at(where)}")
    if _KELVIN_CHECK:
        units.assert_kelvin(T_arr, where=where)
    return C_arr, T_arr


@runtime_checkable
class PropertySet(Protocol):
    """一套物性公式。附录 2 / 3 / 4 各实现一个。

    三套公式**不可混用**（AGENTS.md §4.3）：
        Q1          -> Appendix2（常数物性）
        Q2, Q3      -> Appendix3（变物性）
        Q4          -> Appendix4（变物性 + 收缩）
    """
```

```
60
61      name: str
62      appendix: int
63
64      def rho(self, C: ArrayLike, T: ArrayLike) -> np.ndarray:
65          """含湿药材表观密度，kg/m^3。
66
67          ⚠ 这是**表观**密度（随 C 变化）。扩散方程需要的干基密度 ρ_d 与此不同，
68             换算关系见 docs/DECISIONS.md D-001（尚未定稿）。
69          """
70          ...
71
72      def cp(self, C: ArrayLike, T: ArrayLike) -> np.ndarray:
73          """定压比热容，J/(kg·K)。"""
74          ...
75
76      def k(self, C: ArrayLike, T: ArrayLike) -> np.ndarray:
77          """导热系数，W/(m·K)。"""
78          ...
79
80      def D(self, C: ArrayLike, T: ArrayLike) -> np.ndarray:
81          """水分扩散系数，m^2/s。"""
82          ...
83
84
85  class _NotImplementedPropertySet:
86      """骨架：三套公式集的公共占位实现。
87
88      子类只需填 `name` / `appendix` 和四个方法。填之前请先更新
89      docs/model/correlations.md，并完成量纲校验。
90      """
91
92      name: str = "UNSET"
93      appendix: int = 0
94
95      def _todo(self, method: str) -> NotImplementedError:
96          return NotImplementedError(
97              f"{type(self).__name__}.{method}() 尚未实现。\n"
98              f"  步骤 1: 从 problem/A题.pdf 附录 {self.appendix} 逐字抄录公式到 "
99              f"docs/model/correlations.md\n"
100             f"  步骤 2: 在 docs/model/correlations.md 完成量纲校验\n"
101             f"  步骤 3: 实现本方法，注意 C 是干基、T 是开尔文\n"
102             f"  见 AGENTS.md 铁律 5 与 docs/model/correlations.md"
103         )
104
```

```
    def rho(self, C, T):  # noqa: N803
        raise self._todo("rho")

    def cp(self, C, T):
        raise self._todo("cp")

    def k(self, C, T):  # noqa: N803
        raise self._todo("k")

    def D(self, C, T):  # noqa: N803
        raise self._todo("D")


class Appendix2(_NotImplementedPropertySet):
    """问题 1：常数物性。见 docs/model/correlations.md §2。"""

    name = "appendix2"
    appendix = 2

    def rho(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return np.full(np.broadcast_shapes(C.shape, T.shape), 820.0)

    def cp(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return np.full(np.broadcast_shapes(C.shape, T.shape), 2600.0)

    def k(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return np.full(np.broadcast_shapes(C.shape, T.shape), 0.36)

    def D(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return _diffusion(C, T, 7e-9, 0.89, 0.0)


class Appendix3(_NotImplementedPropertySet):
    """问题 2、3：变物性，ρ/c_p/k 依赖 C，D 依赖 C 与 T。见 docs/model/correlations.
    md §3。"""

    name = "appendix3"
    appendix = 3

    def rho(self, C, T):
        C, T = _prepare(C, T, where=self.name)
```

```
        return 650.0 + 128.0 * C

    def cp(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return 1450.0 + 2736.0 * wet_basis(C)

    def k(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return 0.21 + 0.38 * wet_basis(C)

    def D(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return _diffusion(C, T, 2.4e-3, 0.45, 3850.0)


class Appendix4(_NotImplementedPropertySet):
    """问题 4：变物性（与附录 3 数值不同）+ 尺寸收缩。见 docs/model/correlations.md §4
    。"""

    name = "appendix4"
    appendix = 4

    def rho(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return 760.0 + 90.0 * C

    def cp(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return 1850.0 + 2150.0 * wet_basis(C)

    def k(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return 0.12 + 0.20 * wet_basis(C)

    def D(self, C, T):
        C, T = _prepare(C, T, where=self.name)
        return _diffusion(C, T, 4.2e-4, 0.30, 3850.0)


def _diffusion(C, T, prefactor, moisture_scale, temperature_scale):
    inverse = np.full_like(C, np.inf)
    np.divide(moisture_scale, C, out=inverse, where=C > 0)
    return prefactor * np.exp(-inverse - temperature_scale / T)
```

```python
# --- 工况参数（不是"物性"，但同样集中在这里）--------------------------------


def h_conv(problem: str) -> float:
    """对流换热系数，W/(m^2·K)。

    问题 1 由附录 2 给出。问题 2/3/4 题目是否给出需核对 —— 若未给出，
    必须在 docs/DECISIONS.md 里记录取值与出处（AGENTS.md 铁律 2）。
    """
    if problem not in {"q1", "q2", "q3", "q4"}:
        raise ValueError(f"未知题号: {problem}")
    return 25.0


def h_mass(problem: str) -> float:
    """对流传质系数，m/s。问题 1 由附录 2 给出。"""
    if problem not in {"q1", "q2", "q3", "q4"}:
        raise ValueError(f"未知题号: {problem}")
    return 8e-7


def latent_heat_vaporization(T: ArrayLike) -> np.ndarray:
    """FAO56 Annex3式(3-1)，输入K、输出J/kg。

    Allen et al. (1998), https://www.fao.org/4/x0490e/x0490e0k.htm
    """
    units.assert_kelvin(T, where="latent_heat_vaporization")
    return 1e6 * (2.501 - 0.002361 * units.k_to_c(np.asarray(T, dtype=float)))


def _at(where: str) -> str:
    return f"（位置: {where}）" if where else ""
```

src/common/export.py

```python
"""Solution -> 交付物表格（result*.xlsx 的工作表 + 论文用展示网格）。

列结构一律取自 `io_excel.layout()`（`make probe` 实测结论），不自己发明。
"""

from __future__ import annotations

import numpy as np
import pandas as pd
```

```python

from . import io_excel, problem_spec, units
from .io_excel import SURFACE_HEADER, TIME_HEADER, ResultLayout
from .problem_spec import ProblemSpec
from .solution import Solution


def _sample_rows(t: np.ndarray, target: np.ndarray) -> np.ndarray:
    """对每个目标时刻，找最近的网格索引（不插值，避免引入额外误差）。"""
    return np.array([int(np.argmin(np.abs(t - x))) for x in target], dtype=int)


def _extract(field_2d: np.ndarray, r: np.ndarray, r_target_cm: np.ndarray) -> np.ndarray:
    """把 (n_t, n_r) 的场在指定物理半径上取值。

    若目标半径与网格对齐（常见情形）直接索引；否则按行线性插值。
    """
    r_cm = units.m_to_cm(r)
    # 注意：必须先比长度再 np.allclose —— 形状不同时 allclose 会直接抛 broadcast 错误
    if r_cm.size == r_target_cm.size and np.allclose(np.sort(r_target_cm), np.sort(r_cm)):
        order = np.argsort(r_cm)
        return field_2d[:, order]

    out = np.empty((field_2d.shape[0], r_target_cm.size))
    for i in range(field_2d.shape[0]):
        out[i] = np.interp(r_target_cm, r_cm, field_2d[i])
    return out


def result_sheets(sol: Solution, spec: ProblemSpec) -> dict[str, pd.DataFrame]:
    """构造 result*.xlsx 的工作表。

    工作表名、列数、表头、时间单位一律按 `io_excel.layout(spec.name)`：
        • A 列 = `时间\\到药材中心的距离`，单位 **秒**
        • 距离列 = 0, 0.1, 0.2, …, 2.0（共 21 列；问题 4 末列是「药材表面」）
    """
    lay: ResultLayout = io_excel.layout(spec.name)
    headers = lay.distance_headers()

    # 交付网格的目标半径：问题 1~3 是 0~2.0 的全 21 列；
    # 问题 4 前 20 列是固定物理位置 0~1.9，末列「药材表面」另行处理。
    target_cm = np.array([float(h) for h in headers if _is_numeric(h)], dtype=
```

```
    float)
    has_surface_col = lay.last_header == SURFACE_HEADER

    sheets: dict[str, pd.DataFrame] = {}
    for sheet_name, field in lay.sheets:
        df = pd.DataFrame({TIME_HEADER: sol.t})  # 单位：秒（模板如此）

        values = sol.T if field == "T" else sol.C
        if spec.uses_shrinkage and sol.meta.get("coordinate") == "material":
            vals = np.empty((len(sol.t), len(target_cm)))
            for i, R in enumerate(sol.meta["R"]):
                physical_r = sol.r / spec.geometry.radius0 * R
                vals[i] = np.interp(units.cm_to_m(target_cm), physical_r, values[i
    ],
                                    right=np.nan)
        else:
            vals = _extract(values, sol.r, target_cm)
        if field == "T":
            vals = units.k_to_c(vals)

        for j, head in enumerate(headers):
            if has_surface_col and head == SURFACE_HEADER:
                # 问题 4：末列 = 当时的表面节点值，而非固定 2 cm 处的值
                df[SURFACE_HEADER] = (
                    units.k_to_c(sol.T[:, -1]) if field == "T" else sol.C[:, -1]
                )
            else:
                df[head] = vals[:, j]

        sheets[sheet_name] = df
    return sheets


def _is_numeric(s: str) -> bool:
    try:
        float(s)
        return True
    except (TypeError, ValueError):
        return False


#: 论文表格的时间单位：问题 1 用秒（表 1/表 2），问题 2~4 用小时（表 3~表 6）
TABLE_TIME_UNIT: dict[str, str] = {"q1": "s", "q2": "h", "q3": "h", "q4": "h"}
```

```python
def table_rows(
    sol: Solution,
    spec: ProblemSpec,
    *,
    times_s: np.ndarray,
    distances_cm: tuple[float, ...] = problem_spec.TABLE_R_CM,
    field: str = "C",
    include_surface: bool = False,
) -> pd.DataFrame:
    """论文表 1~表 6 所需的展示网格。

    Parameters
    ----------
    times_s : 展示时刻，单位秒（内部按最近网格点取值，不插值）
    include_surface : 问题 4 的末列是「药材表面」而非固定距离

    返回的 DataFrame 索引单位由 `TABLE_TIME_UNIT` 决定（q1 秒，q2~q4 小时）。
    """
    field = field.upper()
    times_s = np.asarray(times_s, dtype=float)
    if np.any(times_s < sol.t[0]) or np.any(times_s > sol.t[-1]):
        raise ValueError("展示时间超出求解结果，不允许最近邻冒充")

    cols: dict[str, np.ndarray] = {}
    for d in distances_cm:
        cols[f"{d:g}"] = []
    if include_surface:
        cols[SURFACE_HEADER] = []
    values = sol.T if field == "T" else sol.C
    for t in times_s:
        event_time = sol.meta.get("event_time_s")
        if event_time is not None and abs(t-event_time) < 1e-6:
            vals = np.asarray(sol.meta[f"event_{field}"])
            R = sol.meta["event_R"]
            r = np.asarray(sol.meta["internal_xi"])*R
        else:
            vals = np.asarray([np.interp(t, sol.t, values[:,j]) for j in range(sol.n_r)])
            R = np.interp(t, sol.t, sol.meta["R"]) if spec.uses_shrinkage else spec.geometry.radius0
            r = sol.r/spec.geometry.radius0*R
        if field == "T":
            vals = units.k_to_c(vals)
        for d in distances_cm:
            cols[f"{d:g}"].append(np.interp(units.cm_to_m(d), r, vals, right=np.
```

```
    nan))
        if include_surface:
            cols[SURFACE_HEADER].append(vals[-1])

    index = np.asarray(times_s, dtype=float)
    if TABLE_TIME_UNIT.get(spec.name, "s") == "h":
        index = units.s_to_h(index)

    out = pd.DataFrame(cols, index=pd.Index(index, name="时间"))
    out.columns.name = "到药材中心的距离/cm"
    return out


def summarize(sol: Solution, spec: ProblemSpec) -> str:
    """跑完打印的一行摘要，便于在日志里追溯。"""
    return (
        f"[{spec.name}] {sol.summary()} | "
        f"max_r C 终值={sol.max_C_over_r()[-1]:.4f} kg/kg | "
        f"内部步长={sol.meta.get('dt_internal', '未记录')}"
    )
```

src/common/io_excel.py

```
"""交付物 xlsx 读写。

⚠ **模板是交付格式的唯一真源**（AGENTS.md 铁律 3）。
   本模块的布局常量是 `make probe` 实测 `problem/附件/附件3/*.xlsx` 得到的，
   **不是**从题目文字描述猜的。题目文字与模板冲突时，以模板为准。

实测结论（`make probe` 复核，2026 赛前）：

    ┌──────────┬──────────────────┬────────┬──────────────────────────┐
    │ 文件     │ 工作表           │ 时间步 │ 最后一列                 │
    ├──────────┼──────────────────┼────────┼──────────────────────────┤
    │ result1  │ 温度，水分浓度   │ 1 s    │ 2                        │
    │ result2  │ 温度，水分浓度   │ 1 s    │ 2                        │
    │ result3  │ Sheet1           │ 60 s   │ 2                        │
    │ result4  │ Sheet1           │ 60 s   │ 药材表面（文字，非数值） │
    └──────────┴──────────────────┴────────┴──────────────────────────┘

    • A1 是斜线表头 `时间\\到药材中心的距离`，其余表头是距离数值（cm）。
    • 距离列是**完整的 0.1 cm 网格**：0, 0.1, 0.2, …, 2.0 —— 共 **21 列**。
      模板里用 `…` 省略，那是示意，不是只有 4 列。
    • **时间单位是秒**，尽管题目表 3~表 6 的表头写的是 h。
```

```
      模板 A 列数据是 1 或 60（秒），论文表格才用小时。
    • result3/result4 **没有温度表**，且工作表名是 `Sheet1`，不是「水分浓度」。
"""

from __future__ import annotations

import math
import warnings
from dataclasses import dataclass

import numpy as np
import pandas as pd
from openpyxl import Workbook
from openpyxl.cell import WriteOnlyCell

from . import paths

#: 斜线表头（A1 原文）
TIME_HEADER = "时间\\到药材中心的距离"

#: result4 最后一列是文字
SURFACE_HEADER = "药材表面"

#: 题目要求：所有结果保留四位小数
DECIMALS = 4

#: 交付空间网格：0 ~ 2 cm，步长 0.1 cm -> 21 列
N_DISTANCE_COLUMNS = 21

#: 兼容旧名（论文/文档里引用过）
SHEET_TEMPERATURE = "温度"
SHEET_MOISTURE = "水分浓度"

#: 附件 1 / 附件 2 的列名（实测）
ATTACH1_COLUMNS = ("时间", "温度", "水分浓度")
ATTACH2_COLUMNS = ("时间", "半径")


class ResultFormatError(ValueError):
    """交付物格式不符合模板。**不要捕获它然后继续写出**——格式错了就是错了。"""


# --- 布局定义（实测，不是猜的）-----------------------------------------------


```

```python
@dataclass(frozen=True)
class ResultLayout:
    """一个 result 文件的真实结构。"""

    problem: str
    sheets: tuple[tuple[str, str], ...]  # (工作表名, 物理量 "T" | "C")
    dt_s: float  # 交付时间步长（秒）
    last_header: str = "2"  # 最后一列的表头

    @property
    def sheet_names(self) -> tuple[str, ...]:
        return tuple(name for name, _ in self.sheets)

    def distance_headers(self) -> list[str]:
        """距离列的表头。数值用 %g，例如 0, 0.1, ..., 2。"""
        headers = [f"{i * 0.1:g}" for i in range(N_DISTANCE_COLUMNS)]
        headers[-1] = self.last_header
        return headers

    def expected_columns(self) -> list[str]:
        return [TIME_HEADER, *self.distance_headers()]


LAYOUTS: dict[str, ResultLayout] = {
    "q1": ResultLayout("q1", (("温度", "T"), ("水分浓度", "C")), 1.0),
    "q2": ResultLayout("q2", (("温度", "T"), ("水分浓度", "C")), 1.0),
    "q3": ResultLayout("q3", (("Sheet1", "C"),), 60.0),
    "q4": ResultLayout("q4", (("Sheet1", "C"),), 60.0, last_header=SURFACE_HEADER)
    ,
}


def layout(problem: str) -> ResultLayout:
    key = str(problem).strip().lower()
    if key.startswith("result"):
        key = "q" + key.removeprefix("result")
    elif not key.startswith("q"):
        key = "q" + key
    if key not in LAYOUTS:
        raise ValueError(f"问题编号只能是 1~4（或 q1~q4 / result1~result4），收到 {
    problem!r}")
    return LAYOUTS[key]


# --- 读 ----------------------------------------------------------------------
```

```python


def read_attachment(n: int) -> pd.DataFrame:
    """读取 problem/附件/附件<N>.xlsx。

    n=1 -> 烘房温度与水分浓度的时间序列（0~14400 s，步长 60 s）
    n=2 -> 药材半径随时间变化（0~259200 s，步长 1800 s）

    返回的 DataFrame 列名与原件一致，**单位也与原件一致**（时间 s、半径 cm、温度 °C）。
    换算成 SI 是调用方的事（用 units 模块）。
    """
    path = paths.attachment_path(n)
    if not path.exists():
        raise FileNotFoundError(f"附件不存在：{paths.rel(path)}")

    df = pd.read_excel(path, sheet_name=0)
    df.columns = [str(c).strip() for c in df.columns]

    expected = ATTACH1_COLUMNS if n == 1 else ATTACH2_COLUMNS
    if tuple(df.columns) != expected:
        raise ResultFormatError(
            f"{paths.rel(path)} 的列名与预期不符。\n"
            f"  预期：{expected}\n"
            f"  实际：{tuple(df.columns)}\n"
            f"题目原件可能已更新，请核对后同步本模块的常量与 data/README.md。"
        )

    t = df["时间"].to_numpy(dtype=float)
    if t.size and not (pd.Series(t).diff().dropna() > 0).all():
        raise ResultFormatError(f"{paths.rel(path)} 的时间列非严格单调递增。")

    return df


def read_template(problem: str) -> dict[str, pd.DataFrame]:
    """读取 problem/附件/附件3/result<N>.xlsx 的原始内容（含 `…` 示意列）。

    用于 `make probe` 与交叉核对。**不要用它来推导列结构** ——
    用 `layout(problem)`（那是实测结论的结构化形式）。
    """
    path = paths.template_path(problem)
    if not path.exists():
        raise FileNotFoundError(f"交付模板不存在：{paths.rel(path)}")
    sheets = pd.read_excel(path, sheet_name=None)
    return {str(k).strip(): v for k, v in sheets.items()}
```

```python


def verify_layout_against_template(problem: str) -> list[str]:
    """把实测布局与实际模板文件对一遍。返回不一致项（空列表表示一致）。

    CI 里跑这个：如果赛方更新了模板，这里会立刻报出来。
    """
    lay = layout(problem)
    sheets = read_template(problem)
    problems: list[str] = []

    if set(sheets) != set(lay.sheet_names):
        problems.append(
            f"{problem} 工作表名不一致：模板={sorted(sheets)} 布局={sorted(lay.sheet_names)}"
        )

    for name, df in sheets.items():
        cols = [str(c).strip() for c in df.columns]
        if not cols:
            continue
        if cols[0] != TIME_HEADER:
            problems.append(f"{problem}/{name} A1 表头不一致：模板={cols[0]!r} 期望={TIME_HEADER!r}")
        last = cols[-1]
        if last != lay.last_header:
            problems.append(
                f"{problem}/{name} 末列表头不一致：模板={last!r} 期望={lay.last_header!r}"
            )

    # 时间步长：模板给的是示意数据（1 或 60），核对首行
    for name, df in sheets.items():
        t = pd.to_numeric(df.iloc[:, 0], errors="coerce").dropna()
        if t.size:
            step = float(t.iloc[1] - t.iloc[0]) if t.size > 1 else float(t.iloc[0])
            if not math.isclose(step, lay.dt_s):
                problems.append(
                    f"{problem}/{name} 时间步长不一致：模板={step:g} 期望={lay.dt_s:g}"
                )
    return problems


```

```python
# --- 写 ----------------------------------------------------------------------


def write_result(problem: str, sheets: dict[str, pd.DataFrame]) -> paths.Path:
    """写出 results/result<N>.xlsx，并与**实测布局**做一致性校验。

    校验不通过直接抛 ResultFormatError，**不写出文件**。
    数值统一四舍五入到 DECIMALS 位。
    """
    lay = layout(problem)
    _validate(problem, sheets, lay)

    rounded: dict[str, pd.DataFrame] = {}
    for name, df in sheets.items():
        out = df.copy()
        num_cols = out.select_dtypes(include="number").columns
        out[num_cols] = out[num_cols].round(DECIMALS)
        rounded[name] = out

    dest = paths.result_path(problem)
    dest.parent.mkdir(parents=True, exist_ok=True)
    # 全过程result2可能有数百万单元格，流式写入避免openpyxl常规模式内存膨胀。
    wb = Workbook(write_only=True)
    for name, df in rounded.items():
        ws = wb.create_sheet(name)
        ws.freeze_panes = "B2"
        ws.append(list(df.columns))
        for row in df.itertuples(index=False, name=None):
            cells = []
            for value in row:
                cell = WriteOnlyCell(ws, value=None if pd.isna(value) else float(
    value))
                cell.number_format = "0.0000"
                cells.append(cell)
            ws.append(cells)
    wb.save(dest)
    return dest


def _validate(problem: str, sheets: dict[str, pd.DataFrame], lay: ResultLayout) ->
     None:
    """交付前的硬校验。宁可在这里炸，也不要在提交后被发现格式错了。"""
    errors: list[str] = []

    if set(sheets) != set(lay.sheet_names):
```

```
        errors.append(
            f"工作表名不匹配。\n    模板/布局: {sorted(lay.sheet_names)}\n    待写: {sorted(sheets)}"
        )

    expected_cols = lay.expected_columns()
    for name in set(sheets) & set(lay.sheet_names):
        df = sheets[name]
        got = [str(c) for c in df.columns]
        if got != expected_cols:
            diff = _first_diff(got, expected_cols)
            errors.append(
                f"工作表「{name}」列结构不匹配（{diff}）。\n"
                f"    期望 {len(expected_cols)} 列: {expected_cols[:4]} ... {expected_cols[-1]!r}\n"
                f"    实际 {len(got)} 列: {got[:4]} ... {got[-1]!r}\n"
                f"    提示: 距离列是 0~2 cm 步长 0.1 cm 的完整 21 列网格。"
            )
            continue

        if df.empty:
            errors.append(f"工作表「{name}」为空。")
            continue

        t = pd.to_numeric(df.iloc[:, 0], errors="coerce").to_numpy(dtype=float)
        if not np.all(np.isfinite(t)):
            errors.append(f"工作表「{name}」时间列含非数值。")
        else:
            d = np.diff(t)
            if d.size and not np.allclose(d, lay.dt_s, rtol=0, atol=1e-6):
                errors.append(
                    f"工作表「{name}」时间步长不是均匀的 {lay.dt_s:g} s"
                    f"（实测 min={d.min():g}, max={d.max():g}）。"
                )
            if t.size and not math.isclose(t[0], lay.dt_s) and not math.isclose(t[0], 0.0):
                warnings.warn(
                    f"{problem}/{name} 时间列首值为 {t[0]:g}，模板示意数据从 "
                    f"{lay.dt_s:g} 开始。请确认是否要包含 t=0 这一行。",
                    stacklevel=3,
                )

        vals = df.iloc[:, 1:].to_numpy(dtype=float) if df.shape[1] > 1 else np.empty((0, 0))
        if lay.problem == "q4" and vals.size and np.all(np.isfinite(t)):
```

```
            # 用户确认：只允许实际药材域外留空；半径不信任调用方，直接核对附件2。
            radius_data = read_attachment(2)
            rt = radius_data.iloc[:, 0].to_numpy(float)
            rr = radius_data.iloc[:, 1].to_numpy(float)
            if t.min() < rt[0] or t.max() > rt[-1]:
                errors.append("第四问输出时间超出附件2覆盖范围")
            R_cm = np.interp(t, rt, rr)
            outside = np.zeros_like(vals, dtype=bool)
            outside[:, :-1] = np.arange(20)[None, :]*0.1 > R_cm[:, None]+1e-12
            if np.any(~np.isfinite(vals[~outside])):
                errors.append("第四问药材内部或表面含NaN/Inf；仅域外允许留空")
            if np.any(~np.isnan(vals[outside])):
                errors.append("第四问域外位置必须留空，不能填入虚构药材值")
        elif vals.size and not np.all(np.isfinite(vals)):
            bad = [c for c in df.columns[1:] if not _finite(df[c])]
            errors.append(f"工作表「{name}」以下列含 NaN/Inf: {bad}")

    if errors:
        raise ResultFormatError(
            f"result{problem[-1]} 交付校验未通过，未写出文件：\n  - "
            + "\n  - ".join(errors)
            + "\n\n模板见 problem/附件/附件3/。先跑 make probe 查看真实结构。"
            + "见 AGENTS.md 铁律 3。"
        )


def _first_diff(got: list[str], want: list[str]) -> str:
    if len(got) != len(want):
        return f"列数 {len(got)} != {len(want)}"
    for i, (a, b) in enumerate(zip(got, want, strict=False)):
        if a != b:
            return f"第 {i + 1} 列 {a!r} != {b!r}"
    return "未知"


def _finite(s: pd.Series) -> bool:
    try:
        return all(math.isfinite(float(v)) for v in s)
    except (TypeError, ValueError):
        return False


def write_table_csv(df: pd.DataFrame, name: str) -> paths.Path:
    """写论文用的展示网格数值表到 results/tables/<name>.csv。

```

```
    这是 `scripts/make_tables.py` 的输入。**不要手工编辑这些 csv**——
    它们是生成物，下次 make 就覆盖了。
    """
    paths.RESULTS_TABLES.mkdir(parents=True, exist_ok=True)
    dest = paths.RESULTS_TABLES / f"{name}.csv"
    df.round(DECIMALS).to_csv(dest, index=False)
    return dest


# --- 探查 ------------------------------------------------------------------


def probe_template(problem: str) -> str:
    """打印模板的真实结构。**开工第一件事就是跑这个**（AGENTS.md 铁律 3）。"""
    lay = layout(problem)
    path = paths.template_path(problem)
    lines = [f"{paths.rel(path)}"]
    sheets = read_template(problem)
    for name, df in sheets.items():
        lines.append(f"  工作表「{name}」  模板内列数={df.shape[1]}（含 '…' 示意列）
    ")
        lines.append(f"    表头原文: {[str(c) for c in df.columns]}")
        t = pd.to_numeric(df.iloc[:, 0], errors="coerce").dropna()
        if t.size:
            lines.append(f"    时间列示意值: {list(t)}")
    lines.append("  —— 实测布局（写交付物按这个）——")
    lines.append(f"    工作表: {list(lay.sheet_names)}  时间步长: {lay.dt_s:g} s")
    lines.append(
        f"    应为 {len(lay.expected_columns())} 列 = 1 列时间 + "
        f"{N_DISTANCE_COLUMNS} 列距离(0~2 cm 步长 0.1)"
    )
    lines.append(f"    A1 表头: {TIME_HEADER!r}  末列表头: {lay.last_header!r}")
    mism = verify_layout_against_template(problem)
    if mism:
        lines.append("    ⚠ 布局与模板文件不一致: ")
        lines += [f"      - {m}" for m in mism]
    else:
        lines.append("    ✓ 布局与模板文件一致")
    return "\n".join(lines)
```

src/common/numerical_settings.py

```
"""正式数值精度设置；物理模型与物性不在此处定义。"""

```

```
3
4  def production_options(problem):
5      if problem in {"q1", "q2", "q3", "q4"}:
6          return dict(n_internal=1281, rtol=2e-10, atol=2e-11,
7                      max_step=60.0, align_boundary_knots=True)
8      raise ValueError(f"未知题号: {problem}")
```

src/common/paths.py

```
1  """路径常量。任何模块都不许自己拼 "../..", 一律从这里取。"""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  # src/common/paths.py -> parents[2] == 仓库根
8  ROOT: Path = Path(__file__).resolve().parents[2]
9
10 # --- 只读冻结的题目原件 ------------------------------------------------------
11 PROBLEM_DIR: Path = ROOT / "problem"
12 PROBLEM_PDF: Path = PROBLEM_DIR / "A题.pdf"
13 ATTACHMENT_DIR: Path = PROBLEM_DIR / "附件"
14 TEMPLATE_DIR: Path = ATTACHMENT_DIR / "附件3"  # result1~4.xlsx 模板
15
16 # --- 数据 --------------------------------------------------------------------
17 DATA_DIR: Path = ROOT / "data"
18 DATA_RAW: Path = DATA_DIR / "raw"
19 DATA_INTERIM: Path = DATA_DIR / "interim"
20 DATA_PROCESSED: Path = DATA_DIR / "processed"
21
22 # --- 代码 --------------------------------------------------------------------
23 SRC_DIR: Path = ROOT / "src"
24 SCRIPTS_DIR: Path = ROOT / "scripts"
25 TESTS_DIR: Path = ROOT / "tests"
26
27 # --- 交付物（入库）----------------------------------------------------------
28 RESULTS_DIR: Path = ROOT / "results"
29 RESULTS_TABLES: Path = RESULTS_DIR / "tables"  # 论文用展示网格的 csv
30
31 # --- 论文 --------------------------------------------------------------------
32 PAPER_DIR: Path = ROOT / "paper"
33 FIGURES_DIR: Path = PAPER_DIR / "figs"  # 图片直接写到这里, 不要再复制
34 TABLES_DIR: Path = PAPER_DIR / "tab"  # 生成的 booktabs 片段
35 PAPER_BUILD: Path = PAPER_DIR / "build"
```

```python

# --- 可重建产物（gitignore）-------------------------------------------------
OUTPUTS_DIR: Path = ROOT / "outputs"
OUTPUTS_CACHE: Path = OUTPUTS_DIR / "cache"
OUTPUTS_LOGS: Path = OUTPUTS_DIR / "logs"

# --- 文档 -------------------------------------------------------------------
DOCS_DIR: Path = ROOT / "docs"
MODEL_DOCS: Path = DOCS_DIR / "model"


def rel(path: str | Path) -> str:
    """转为相对仓库根的字符串路径。用于日志、xlsx 溯源信息、错误消息。"""
    p = Path(path).resolve()
    try:
        return str(p.relative_to(ROOT))
    except ValueError:
        return str(p)


def ensure_output_dirs() -> None:
    """创建所有可写目录。脚本入口调用一次即可（首次 clone 后目录可能不存在）。"""
    for d in (
        DATA_RAW,
        DATA_INTERIM,
        DATA_PROCESSED,
        RESULTS_DIR,
        RESULTS_TABLES,
        FIGURES_DIR,
        TABLES_DIR,
        PAPER_BUILD,
        OUTPUTS_CACHE,
        OUTPUTS_LOGS,
    ):
        d.mkdir(parents=True, exist_ok=True)


def template_path(problem: str) -> Path:
    """交付模板路径。problem 取 'q1'..'q4' 或 'result1'..'result4'。"""
    n = _problem_number(problem)
    return TEMPLATE_DIR / f"result{n}.xlsx"


def result_path(problem: str) -> Path:
    """交付物路径。"""
```

```
    n = _problem_number(problem)
    return RESULTS_DIR / f"result{n}.xlsx"


def attachment_path(n: int) -> Path:
    """附件路径。n=1 -> 烘房条件；n=2 -> 半径随时间变化。"""
    if n not in (1, 2):
        raise ValueError(f"附件编号只支持 1 或 2，收到 {n}")
    return ATTACHMENT_DIR / f"附件{n}.xlsx"


def _problem_number(problem: str) -> int:
    key = str(problem).strip().lower().removeprefix("result").removeprefix("q")
    if key not in {"1", "2", "3", "4"}:
        raise ValueError(f"问题编号只能是 1~4 或 q1~q4 或 result1~result4，收到 {
    problem!r}")
    return int(key)
```

src/common/plotting.py

```
"""论文插图。

规则（AGENTS.md §2 / docs/INTERFACES.md §7）：
    - 输出一律 **PDF（矢量）**，不要 PNG。
    - 图片直接写到 paper/figs/，论文用 \\includegraphics{figs/xxx.pdf} 引用。
      **不要再往论文目录复制图片。**
    - 轴标签必须含单位；图里的单位是 **cm 和 °C**（展示单位），
      因此本模块负责从 SI 转换，调用方传进来的一律还是 SI。
"""

from __future__ import annotations

import os
from collections.abc import Mapping, Sequence
from pathlib import Path

from . import paths


def _ensure_cache_dir(env_var: str, default_dir: Path, repo_dir: Path) -> None:
    """给 matplotlib / fontconfig 找一个可写的缓存目录。

    默认位置在受限环境（容器、CI、只读 HOME）里常常不可写，matplotlib 会退回
    临时目录并**每次重新扫描字体**——中文环境下这个开销很明显，fontconfig 还会
```

```
    直接报 "No writable cache directories"。

    策略：先试默认位置；**不可写时**才回退到仓库内（outputs/ 已被 gitignore）。
    不无条件覆盖环境变量，以免擅自搬动别人机器上的缓存。

    ⚠ 必须在 `import matplotlib` **之前**调用 —— matplotlib 在自身导入期间就会
       解析配置目录，之后再设环境变量已经太晚了（实测确认）。
    """
    if os.environ.get(env_var):
        return
    probe = default_dir / ".write_test"
    try:
        default_dir.mkdir(parents=True, exist_ok=True)
        probe.touch()
        probe.unlink()
        return  # 默认位置可写，不动它
    except OSError:
        pass

    fallback = repo_dir / "mplconfig"
    fallback.mkdir(parents=True, exist_ok=True)
    os.environ[env_var] = str(fallback)


# 必须在 import matplotlib 之前设置
_ensure_cache_dir("MPLCONFIGDIR", Path.home() / ".matplotlib", paths.OUTPUTS_CACHE
    )
# fontconfig 用的是 XDG_CACHE_HOME（Linux/容器里常见报错源）
_ensure_cache_dir("XDG_CACHE_HOME", Path.home() / ".cache", paths.OUTPUTS_CACHE)

import matplotlib  # noqa: E402

# 必须在 import pyplot 之前设置，否则在无头环境（CI / 服务器）会失败
matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np

from . import units  # noqa: E402
from .solution import Solution  # noqa: E402

#: 中文字体候选。macOS 优先，回退到常见 Linux/Windows 字体，避免"豆腐块"。
_CJK_FONTS = (
    "Songti SC",
    "Heiti SC",
    "PingFang SC",
```

```
    "STSong",
    "SimSun",
    "Microsoft YaHei",
    "Noto Sans CJK SC",
    "Noto Serif CJK SC",
    "WenQuanYi Zen Hei",
    "DejaVu Sans",
)

_FIGSIZE = (6.0, 4.0)
_DPI = 300


def apply_style() -> None:
    """全局设置。每个脚本入口调用一次即可。"""
    plt.rcParams.update(
        {
            "font.sans-serif": list(_CJK_FONTS),
            "font.family": "sans-serif",
            "axes.unicode_minus": False,  # 负号用 ASCII，避免中文字体缺字形
            "font.size": 9,
            "axes.labelsize": 10,
            "axes.titlesize": 10,
            "legend.fontsize": 9,
            "xtick.labelsize": 9,
            "ytick.labelsize": 9,
            "figure.figsize": _FIGSIZE,
            "figure.dpi": 120,
            "savefig.dpi": _DPI,
            "savefig.bbox": "tight",
            "savefig.pad_inches": 0.02,
            "axes.grid": True,
            "grid.alpha": 0.3,
            "grid.linestyle": "--",
            "lines.linewidth": 1.4,
            "pdf.fonttype": 42,  # 嵌入 TrueType，投稿/打印不出问题
            "ps.fonttype": 42,
        }
    )


def _out_path(out: str | Path) -> Path:
    """把相对名转成 paper/figs/<name>.pdf。"""
    p = Path(out)
    if p.suffix != ".pdf":
```

```
        p = p.with_suffix(".pdf")
    if p.parent == Path("."):
        p = paths.FIGURES_DIR / p.name
    p.parent.mkdir(parents=True, exist_ok=True)
    return p


def _cjk_warning() -> str | None:
    """若没有任何中文字体可用，返回警告文本（论文里出现豆腐块是致命的）。"""
    from matplotlib import font_manager

    available = {f.name for f in font_manager.fontManager.ttflist}
    if not available & set(_CJK_FONTS):
        return (
            "⚠ 未找到任何候选中文字体，图中中文会显示为方块。\n"
            f"   候选：{_CJK_FONTS}\n"
            "   请安装中文字体（macOS 自带 Songti SC；Linux：fonts-noto-cjk）。"
        )
    return None


# --- 图 1：某一时刻的径向分布 ------------------------------------------------


def plot_profiles(
    sol: Solution,
    *,
    field: str,
    times: Sequence[float],
    out: str | Path,
    title: str | None = None,
    r_scale: float | None = None,
) -> Path:
    """画若干时刻的径向分布。

    Parameters
    ----------
    field : "T" 或 "C"
    times : 要画的时刻，单位 **s**（内部会找最近的网格点）
    r_scale : 若给定，用 r/r_scale 作横坐标（问题 4 的无量纲坐标 ξ 用）
    """
    field = field.upper()
    if field not in {"T", "C"}:
        raise ValueError(f"field 只能是 'T' 或 'C'，收到 {field!r}")

```

```
    label = "温度" if field == "T" else "水分浓度"
    ylabel = "温度 / °C" if field == "T" else "水分浓度 / (kg/kg, 干基)"

    fig, ax = plt.subplots()
    for t_req in times:
        i = int(np.argmin(np.abs(sol.t - t_req)))
        if field == "T":
            y = units.k_to_c(sol.T[i])
            t_show = f"{sol.t[i] / 60:.0f} min" if sol.t[i] < 3600 else f"{sol.t[i] / 3600:.1f} h"
        else:
            y = sol.C[i]
            t_show = f"{sol.t[i] / 3600:.2f} h"

        if r_scale is None:
            x = units.m_to_cm(sol.r)
            xlabel = "到药材中心的距离 / cm"
        else:
            x = sol.r / r_scale
            xlabel = r"无量纲径向坐标 $\xi = r/R(t)$"

        ax.plot(x, y, marker="o", markersize=2.5, label=f"t = {t_show}")

    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title or f"{label}的径向分布")
    ax.legend()
    fig.savefig(_out_path(out))
    plt.close(fig)
    return _out_path(out)


# --- 图 2: 若干位置的时间历程 ------------------------------------------------


def plot_history(
    sol: Solution,
    *,
    locations_cm: Sequence[float],
    field: str = "C",
    out: str | Path,
    title: str | None = None,
) -> Path:
    """画若干径向位置的时间历程。locations_cm 单位 **cm**（展示单位）。"""
    field = field.upper()
```

```
    label = "温度" if field == "T" else "水分浓度"
    ylabel = "温度 / °C" if field == "T" else "水分浓度 / (kg/kg, 干基)"

    t_h = units.s_to_h(sol.t)
    fig, ax = plt.subplots()
    for r_cm in locations_cm:
        i = int(np.argmin(np.abs(units.m_to_cm(sol.r) - r_cm)))
        y = units.k_to_c(sol.T[:, i]) if field == "T" else sol.C[:, i]
        ax.plot(t_h, y, label=f"r = {units.m_to_cm(sol.r[i]):.1f} cm")

    ax.set_xlabel("时间 / h")
    ax.set_ylabel(ylabel)
    ax.set_title(title or f"{label}的时间历程")
    ax.legend()
    fig.savefig(_out_path(out))
    plt.close(fig)
    return _out_path(out)


# --- 图 3：网格收敛性（log-log）-----------------------------------------------


def plot_grid_convergence(
    errors: Mapping[float, float],
    *,
    out: str | Path,
    expected_order: float | None = None,
    title: str = "网格收敛性验证",
) -> Path:
    """画误差随步长的变化（双对数），用于论文的模型验证一节。

    Parameters
    ----------
    errors : {步长 h: 误差 E(h)}
    expected_order : 若给定，画一条斜率参考线，便于目视判断实测阶数
    """
    if len(errors) < 2:
        raise ValueError("至少需要两个步长点才能画收敛曲线。")

    h = np.array(sorted(errors), dtype=float)
    e = np.array([errors[k] for k in h], dtype=float)
    if np.any(e <= 0):
        raise ValueError("误差必须为正，否则无法在双对数图上表示。")

    # 实测收敛阶：相邻点斜率的中位数
```

```
    slopes = np.diff(np.log(e)) / np.diff(np.log(h))
    order_est = float(np.median(slopes))

    fig, ax = plt.subplots()
    ax.loglog(h, e, "o-", label=f"实测（阶数 ≈ {order_est:.2f}）")
    if expected_order is not None:
        ref = e[0] * (h / h[0]) ** expected_order
        ax.loglog(h, ref, "k--", alpha=0.6, label=f"参考斜率 {expected_order}")

    ax.set_xlabel("步长 h")
    ax.set_ylabel("误差 E(h)")
    ax.set_title(title)
    ax.legend()
    fig.savefig(_out_path(out))
    plt.close(fig)
    return _out_path(out)


def check_fonts(verbose: bool = True) -> bool:
    """检查中文字体。返回 True 表示可以画中文图。"""
    msg = _cjk_warning()
    if verbose:
        print(msg if msg else "✓ 中文字体可用")
    return msg is None


# 便利常量：供脚本引用，避免拼字符串
FIGS = paths.FIGURES_DIR
if os.environ.get("MPLBACKEND") is None:
    apply_style()
```

src/common/problem_spec.py

```
"""题目常量与四个问题的静态定义。

把"题目给的东西"集中成结构化对象，避免散落在代码里（AGENTS.md 铁律 5 的延伸）。
只放**题目明确的给定量**，不放任何需要建模决定的量。
"""

from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np
```

```python

from .correlations import Appendix2, Appendix3, Appendix4, PropertySet
from .paths import rel

# --- 题目给定量（来源：problem/A题.pdf）-----------------------------------

#: 药材长度 L = 25 cm（题目：形状大致呈圆柱形，长为 25 cm）
LENGTH_M: float = 0.25

#: 初始半径 R0 = 2 cm
RADIUS0_M: float = 0.02

#: 初始温度 28 °C
INITIAL_T_C: float = 28.0

#: 初始水分浓度（干基）2.55 kg/kg
INITIAL_C_DRY: float = 2.55

#: 问题 3 的烘干终点判据：各处水分浓度应低于 0.15 kg/kg
TARGET_C_DRY: float = 0.15

#: 问题 1 的求解时段：1800 s
Q1_T_END_S: float = 1800.0

#: 问题 2 的表 3/表 4 展示时段：3 h
Q2_TABLE_T_END_S: float = 3.0 * 3600.0

#: 交付完整结果的时间步长（题目要求"每隔 1 s"）
DT_OUTPUT_DEFAULT_S: float = 1.0

#: 问题 3/4 交付完整结果的时间步长（题目要求"每隔 60 s"）
DT_OUTPUT_Q34_S: float = 60.0

#: 交付空间步长：0.1 cm
DR_OUTPUT_M: float = 0.1 * 1e-2

#: 论文表格的展示距离网格：0, 0.5, 1, 1.5, 2 cm
TABLE_R_CM: tuple[float, ...] = (0.0, 0.5, 1.0, 1.5, 2.0)

#: 附件 1 的覆盖时长（实测 14400 s = 4 h）—— 问题 2/3/4 需要外推，见 D-009
AMBIENT_COVERAGE_S: float = 14400.0

#: 附件 2 的覆盖时长（实测 259200 s = 72 h）
RADIUS_COVERAGE_S: float = 259200.0

```

```python

@dataclass(frozen=True)
class Geometry:
    """几何。单位：m。"""

    length: float = LENGTH_M
    radius0: float = RADIUS0_M

    def __post_init__(self) -> None:
        if not (self.length > 0 and self.radius0 > 0):
            raise ValueError(f"几何参数必须为正：L={self.length}, R0={self.radius0}")
        # 长径比：用于论证一维径向简化的合理性（docs/DECISIONS.md D-007）
        if self.length / self.radius0 < 5.0:
            raise ValueError(
                f"长径比仅 {self.length / self.radius0:.2f}，一维径向简化（D-007）可能不成立。"
                f"若确实要这样建模，请先更新 docs/DECISIONS.md 的论证。"
            )

    @property
    def aspect_ratio(self) -> float:
        """长径比 L/(2R0)。用于论文里论证轴向均匀假设。"""
        return self.length / (2.0 * self.radius0)


@dataclass(frozen=True)
class InitialState:
    """初始条件。T 用 K，C 用干基 kg/kg。"""

    T: float = INITIAL_T_C + 273.15  # noqa: N815
    C: float = INITIAL_C_DRY  # noqa: N815

    def __post_init__(self) -> None:
        if self.T < 200.0:
            raise ValueError(f"初始温度 {self.T} 看起来不是开尔文。请写 °C + 273.15。")
        if self.C < 0.0:
            raise ValueError(f"初始水分浓度不能为负：{self.C}")


@dataclass(frozen=True)
class Grid:
    """交付网格（不是内部计算网格）。

```

```
    内部网格可能更细，那时求解器负责插值到本网格，并必须在
    docs/DECISIONS.md 里记录 + 给出步长无关性验证（AGENTS.md §4.2）。
    """

    dt: float  # s
    t_end: float  # s
    dr: float = DR_OUTPUT_M  # m
    radius: float = RADIUS0_M  # m，问题 4 中会随时间变化

    def __post_init__(self) -> None:
        if self.dt <= 0 or self.t_end <= 0 or self.dr <= 0:
            raise ValueError("网格参数必须为正")
        if self.t_end < self.dt:
            raise ValueError(f"总时长 {self.t_end} 小于一个时间步 {self.dt}")

    @property
    def t(self) -> np.ndarray:
        """时间节点，(n_t,) s。"""
        n = int(round(self.t_end / self.dt)) + 1
        return np.linspace(0.0, self.t_end, n)

    @property
    def r(self) -> np.ndarray:
        """径向节点，(n_r,) m。i=0 对应轴心 r=0。"""
        n = int(round(self.radius / self.dr)) + 1
        return np.linspace(0.0, self.radius, n)

    @property
    def shape(self) -> tuple[int, int]:
        return (self.t.size, self.r.size)


@dataclass(frozen=True)
class ProblemSpec:
    """一个问题的完整静态定义。"""

    name: str  # "q1" | "q2" | "q3" | "q4"
    title: str
    geometry: Geometry
    initial: InitialState
    grid: Grid
    properties: PropertySet
    uses_shrinkage: bool = False  # 问题 4 为 True
    extra: dict = field(default_factory=dict)

```

```
144     def describe(self) -> str:
145         n_t, n_r = self.grid.shape
146         return (
147             f"[{self.name}] {self.title}\n"
148             f"  几何    : L={self.geometry.length} m, R0={self.geometry.radius0} m,
    "
149             f"长径比={self.geometry.aspect_ratio:.1f}\n"
150             f"  初值    : T={self.initial.T:.2f} K, C={self.initial.C} kg/kg(干基)\n
    "
151             f"  交付网格: {n_t} 时间步 x {n_r} 空间节点 "
152             f"(dt={self.grid.dt}s, dr={self.grid.dr * 100}cm, t_end={self.grid.
    t_end}s)\n"
153             f"  物性集  : {self.properties.name} (附录 {self.properties.appendix})\n
    "
154             f"  收缩    : {'是' if self.uses_shrinkage else '否'}"
155         )
156
157
158 # --- 四个问题的定义 ------------------------------------------------------------
159 # t_end 中，问题 2/3/4 的"整个烘干过程"时长本身就是待求结果（问题 3/4），
160 # 或是需要先估算的上界（问题 2）。因此这里只给一个**占位上界**，
161 # 真正取值必须在 docs/DECISIONS.md 定稿后由 @impl 更新。
162
163 #: 占位上界：附件 2 覆盖 72 h。⚠ 这是数据覆盖时长，不是题目给的答案。
164 _T_END_PLACEHOLDER_S: float = 259200.0
165
166 Q1 = ProblemSpec(
167     name="q1",
168     title="预热平衡阶段温度与水分浓度变化规律",
169     geometry=Geometry(),
170     initial=InitialState(),
171     grid=Grid(dt=DT_OUTPUT_DEFAULT_S, t_end=Q1_T_END_S),
172     properties=Appendix2(),
173     uses_shrinkage=False,
174 )
175
176 Q2 = ProblemSpec(
177     name="q2",
178     title="整个烘干过程的温度与水分浓度变化规律",
179     geometry=Geometry(),
180     initial=InitialState(),
181     grid=Grid(dt=DT_OUTPUT_DEFAULT_S, t_end=_T_END_PLACEHOLDER_S),
182     properties=Appendix3(),
183     uses_shrinkage=False,
184     extra={"table_t_end": Q2_TABLE_T_END_S},
```

```
)

Q3 = ProblemSpec(
    name="q3",
    title="确定药材烘干所需的时间",
    geometry=Geometry(),
    initial=InitialState(),
    grid=Grid(dt=DT_OUTPUT_Q34_S, t_end=_T_END_PLACEHOLDER_S),
    properties=Appendix3(),
    uses_shrinkage=False,
    extra={"target_C": TARGET_C_DRY},
)

Q4 = ProblemSpec(
    name="q4",
    title="考虑失水收缩的烘干时长",
    geometry=Geometry(),
    initial=InitialState(),
    grid=Grid(dt=DT_OUTPUT_Q34_S, t_end=_T_END_PLACEHOLDER_S),
    properties=Appendix4(),
    uses_shrinkage=True,  # R(t) 来自 problem/附件/附件2.xlsx
    extra={"target_C": TARGET_C_DRY},
)

ALL: dict[str, ProblemSpec] = {"q1": Q1, "q2": Q2, "q3": Q3, "q4": Q4}


def get(problem: str) -> ProblemSpec:
    key = str(problem).strip().lower()
    if key.startswith("result"):
        key = "q" + key.removeprefix("result")
    if key not in ALL:
        raise KeyError(f"未知问题 {problem!r}，可选：{sorted(ALL)}")
    return ALL[key]


def describe_all() -> str:
    return "\n\n".join(spec.describe() for spec in ALL.values())


def source_note() -> str:
    """溯源信息。写进 xlsx 的元数据或日志，便于回答「这个数从哪来的」。"""
    return (
        f"spec 定义：{rel(__file__)}\n"
        f"题目原件：problem/A题.pdf\n"
```

```
        f"模板原件: problem/附件/附件3/resultN.xlsx\n"
        f"环境数据: problem/附件/附件1.xlsx (0~{AMBIENT_COVERAGE_S:.0f} s)\n"
        f"半径数据: problem/附件/附件2.xlsx (0~{RADIUS_COVERAGE_S:.0f} s)"
    )
```

src/common/psychrometrics.py

```
"""FAO56饱和汽压及ASHRAE热质类比；SI输入，湿含量为kg水/kg干空气。

来源及适用限制：docs/model/references-research.md、docs/model/numerics.md。
"""

import numpy as np

from .units import assert_kelvin, k_to_c

STANDARD_PRESSURE_PA = 101325.0
DRY_AIR_CP = 1006.0
WATER_VAPOR_CP = 1860.0
LEWIS_NUMBER = 0.845
MOLECULAR_MASS_RATIO = 0.621945


def saturation_pressure(T):
    """FAO56式(11)，输出Pa。"""
    assert_kelvin(T, where="saturation_pressure")
    T_C = k_to_c(np.asarray(T, dtype=float))
    return 610.8 * np.exp(17.27 * T_C / (T_C + 237.3))


def humidity_ratio(T, relative_humidity, pressure=STANDARD_PRESSURE_PA):
    """相对湿度用0—1；输出气相湿含量Y，不是药材平衡含水率。"""
    phi = np.asarray(relative_humidity, dtype=float)
    if np.any((phi < 0) | (phi > 1)):
        raise ValueError("相对湿度必须在0—1范围")
    vapor = phi * saturation_pressure(T)
    if np.any(vapor >= pressure):
        raise ValueError("蒸汽分压必须小于总压")
    return MOLECULAR_MASS_RATIO * vapor / (pressure - vapor)


def vapor_pressure(Y, pressure=STANDARD_PRESSURE_PA):
    return pressure * np.asarray(Y) / (MOLECULAR_MASS_RATIO + np.asarray(Y))

```

```
38
39 def relative_humidity(T, Y, pressure=STANDARD_PRESSURE_PA):
40     return vapor_pressure(Y, pressure) / saturation_pressure(T)
41
42
43 def gas_evaporation_capacity(T_s, Y_amb, h, lewis=LEWIS_NUMBER):
44     """由Lewis类比得到气膜输水能力kg/(m²s)，保留冷凝负号。"""
45     moist_air_cp = DRY_AIR_CP + WATER_VAPOR_CP * np.asarray(Y_amb)
46     return h / moist_air_cp * lewis**(-2.0/3.0) * (humidity_ratio(T_s, 1.0) -
    Y_amb)
```

src/common/radial.py

```
1  """圆柱节点中心有限体积+BDF；干基材料坐标模型见 docs/model/numerics.md。"""
2
3  import time
4
5  import numpy as np
6  from scipy.integrate import solve_ivp
7  from scipy.sparse import coo_matrix, lil_matrix
8
9  from .boundary import Boundary
10 from .correlations import h_conv, h_mass, latent_heat_vaporization
11 from .solution import Solution
12 from .psychrometrics import gas_evaporation_capacity, LEWIS_NUMBER
13
14
15 def outward_flux(value_surface, value_ambient, coefficient):
16     """向外为正的Robin通量；加热时热通量为负。"""
17     return coefficient * (value_surface - value_ambient)
18
19
20 def grid_weights(n):
21     # 表面二次加密，解析干燥后期低D区域的薄层。
22     xi = 1.0-(1.0-np.linspace(0.0, 1.0, n))**2
23     faces = np.r_[0.0, (xi[1:] + xi[:-1]) / 2.0, 1.0]
24     weights = np.diff(faces**2) / 2.0
25     return xi, faces, weights
26
27
28 class RadialModel:
29     def __init__(self, spec, n=161, boundary=None, fixed_radius=False, latent_heat
    =True):
30         self.spec = spec
```

```
        self.n = n
        self.xi, self.faces, self.weights = grid_weights(n)
        self.boundary = boundary or Boundary.for_problem(spec.name)
        self.moving = spec.uses_shrinkage and not fixed_radius
        self.h_conv = h_conv(spec.name)
        self.h_mass = h_mass(spec.name)
        self.latent_heat = latent_heat
        self.moisture_boundary = "supply_gas"
        self.lewis = LEWIS_NUMBER
        self.equilibrium_C = 0.0
        self.rho_d0 = float(spec.properties.rho(spec.initial.C, spec.initial.T)) /
 (1 + spec.initial.C)

    def dry_density(self, t):
        return self.rho_d0 * (self.spec.geometry.radius0 / self.radius(t))**2

    def surface_fluxes(self, t, T_s, C_s):
        T_amb, C_amb = self.boundary.ambient(t)
        if self.moisture_boundary == "legacy_effective":
            moisture = outward_flux(C_s, C_amb, self.h_mass)
        else:
            supply = self.dry_density(t) * self.h_mass * (C_s-self.equilibrium_C)
            gas = gas_evaporation_capacity(T_s, C_amb, self.h_conv, self.lewis)
            # 物理互补边界，允许气相凝结；绝不裁剪数值状态T或C。
            moisture = np.minimum(supply, gas) / self.dry_density(t)
        convective = self.h_conv * (T_amb - T_s)
        latent = self.dry_density(t) * moisture * latent_heat_vaporization(T_s) if
 self.latent_heat else 0.0
        return moisture, convective, latent

    def radius(self, t):
        return self.boundary.radius(t) if self.moving else self.spec.geometry.
radius0

    def initial(self):
        # 最后两项是边界累计质量损失（归一化干物质基准）和加热量。
        return np.r_[np.full(self.n, self.spec.initial.T),
                     np.full(self.n, self.spec.initial.C), 0.0, 0.0]

    def rhs(self, t, y):
        n = self.n
        T, C = y[:n], y[n:2*n]
        p = self.spec.properties
        R = self.radius(t)
        k = p.k(C, T)
```

```
        D = p.D(C, T)
        capacity = p.rho(C, T) * p.cp(C, T)
        # q = r * coefficient * field_r, 正号表示向内输运贡献。
        qT, qC = np.zeros(n+1), np.zeros(n+1)
        qT[1:-1] = self.faces[1:-1] * (k[1:]+k[:-1])/2 * np.diff(T)/np.diff(self.
    xi)
        qC[1:-1] = self.faces[1:-1] * (D[1:]+D[:-1])/2 * np.diff(C)/np.diff(self.
    xi)
        moisture, convective, latent = self.surface_fluxes(t, T[-1], C[-1])
        qT[-1] = R * (convective - latent)
        qC[-1] = -R * moisture
        dT = np.diff(qT) / (R*R*self.weights*capacity)
        dC = np.diff(qC) / (R*R*self.weights)
        loss_rate = -qC[-1] / (R*R)
        heat_rate = qT[-1]
        return np.r_[dT, dC, loss_rate, heat_rate]

    def sparsity(self):
        n = self.n
        s = lil_matrix((2*n+2, 2*n+2), dtype=int)
        for i in range(n):
            near = list(range(max(0,i-1), min(n,i+2)))
            s[i, near] = 1
            s[i, [j+n for j in near]] = 1
            s[i+n, near] = 1
            s[i+n, [j+n for j in near]] = 1
        s[2*n, 2*n-1] = 1
        s[2*n+1, n-1] = 1
        s[2*n+1, 2*n-1] = 1
        return s.tocsr()

    def jacobian(self, t, y):
        """六色差分Jacobian，累计量对应的零列不作自适应扰动。

        防止SciPy默认差分反复放大严格零导数列的扰动因子而溢出。
        累计通量行使用解析导数；不屏蔽运行时警告。
        """
        n = self.n
        baseline = self.rhs(t, y)
        steps = np.sqrt(np.finfo(float).eps)*np.maximum(np.abs(y), 1.0)
        rows, cols, data = [], [], []
        for field in range(2):
            for color in range(3):
                ids = np.arange(color, n, 3)+field*n
                perturbed = y.copy()
```

```
                perturbed[ids] += steps[ids]
                difference = self.rhs(t, perturbed)-baseline
                for j in ids:
                    i = j % n
                    for row_field in range(2):
                        for k in range(max(0,i-1),min(n,i+2)):
                            row = row_field*n+k
                            rows.append(row)
                            cols.append(j)
                            data.append(difference[row]/steps[j])
        for j in (n-1, 2*n-1):
            perturbed = y.copy()
            perturbed[j] += steps[j]
            difference = self.rhs(t, perturbed) - baseline
            for row in (2*n, 2*n+1):
                rows.append(row)
                cols.append(j)
                data.append(difference[row] / steps[j])
        return coo_matrix((data,(rows,cols)),shape=(2*n+2,2*n+2)).tocsc()


def solve_radial(spec, **opts):
    start = time.perf_counter()
    n = int(opts.get("n_internal", 321))
    if n < 3:
        raise ValueError("至少需要三个空间节点")
    boundary = opts.get("boundary") or Boundary.for_problem(spec.name)
    model = RadialModel(spec, n, boundary, opts.get("fixed_radius", False), opts.
    get("latent_heat", True))
    model.moisture_boundary = opts.get("moisture_boundary", "supply_gas")
    model.lewis = opts.get("lewis", LEWIS_NUMBER)
    model.equilibrium_C = opts.get("equilibrium_C", 0.0)
    rtol = float(opts.get("rtol", 2e-9))
    atol = float(opts.get("atol", 2e-10))
    max_step = float(opts.get("max_step", 120.0))
    method = opts.get("method", "BDF")
    if method not in {"BDF", "Radau"}:
        raise ValueError("刚性积分方法必须为BDF或Radau")
    align_boundary_knots = bool(opts.get("align_boundary_knots", False))
    horizon = float(opts.get("t_end", spec.grid.t_end))
    target = spec.extra.get("target_C", 0.15)
    seek_end = opts.get("seek_end", spec.name != "q1")

    def event(t, y):
        return np.max(y[n:2*n]) - target
```

```
160
161     event.direction = -1
162     event.terminal = True
163     # 在4小时边界外推接点重启，避免跳过敏感性算例的参数跳变。
164     split = boundary.ambient_t[-1]
165     knots = boundary.ambient_t if align_boundary_knots else np.array([split])
166     if align_boundary_knots and model.moving:
167         knots = np.unique(np.r_[knots, boundary.radius_t])
168     edges = [0.0] + [float(x) for x in knots if 0 < x < horizon] + [horizon]
169     pieces = []
170     y0 = model.initial()
171     event_time = None
172
173     def starting_step(t0, t1, state, step_limit):
174         if not align_boundary_knots:
175             return None
176         # 默认启动估计含显式试探；极细表面单元会使其越过C=0。
177         # 限制初始试探为局部浓度变化时间尺度的1%，不裁剪状态。
178         rate = model.rhs(t0, state)[n:2*n]
179         active = np.abs(rate) > 0
180         scale = np.min(state[n:2*n][active] / np.abs(rate[active])) if active.any
    () else 1.0
181         return min(1.0, step_limit, t1-t0, 0.01*scale)
182
183     for a, b in zip(edges[:-1], edges[1:]):
184         step = min(max_step, 30.0) if a < split else max_step
185         result = solve_ivp(model.rhs, (a,b), y0, method=method, rtol=rtol, atol=
    atol,
186                            max_step=step, jac=model.jacobian, dense_output=True,
187                            first_step=starting_step(a, b, y0, step),
188                            events=event if seek_end else None)
189         if not result.success:
190             raise RuntimeError(f"{spec.name} {method}失败: {result.message}")
191         pieces.append(result)
192         y0 = result.y[:, -1]
193         if seek_end and result.t_events[0].size:
194             event_time = float(result.t_events[0][0])
195             # 正式xlsx保留均匀输出，继续到首个严格合格输出点。
196             output_end = (np.floor(event_time/spec.grid.dt)+1)*spec.grid.dt
197             if output_end > horizon:
198                 raise ValueError("终点后的首个输出时刻超出可计算范围")
199             tail = solve_ivp(model.rhs, (event_time, output_end), y0, method=
    method,
200                              rtol=rtol, atol=atol, max_step=step,
201                              first_step=starting_step(event_time, output_end, y0,
```

```
    step),
                              jac=model.jacobian, dense_output=True)
            if not tail.success:
                raise RuntimeError(tail.message)
            pieces.append(tail)
            horizon = output_end
            break
    if seek_end and event_time is None:
        raise ValueError(f"{spec.name} 在{horizon/3600:g} h内未达到全场<{target}; "
                         f"末值={np.max(y0[n:2*n]):.8f}, 需复核或延长获准范围")

    t = np.asarray(opts.get("sample_times", np.arange(0.0, horizon+spec.grid.dt/2,
                                                      spec.grid.dt)), dtype=float)
    if np.any(t < 0) or np.any(t > horizon+1e-8):
        raise ValueError("采样时刻超出已求解区间")
    r = (model.xi*spec.geometry.radius0
         if model.moving else spec.grid.r)
    T, C = np.empty((len(t), len(r))), np.empty((len(t),len(r)))
    radii = np.asarray([model.radius(x) for x in t])
    max_mass_residual = 0.0
    max_energy_residual = 0.0
    initial_mass = model.weights.sum()*spec.initial.C
    initial_capacity = float(np.asarray(spec.properties.rho(spec.initial.C,spec.
    initial.T)
                                        *spec.properties.cp(spec.initial.C,spec.
    initial.T)))
    for piece in pieces:
        raw_mass_error = np.abs(model.weights@piece.y[n:2*n] + piece.y[2*n]-
    initial_mass)
        max_mass_residual = max(max_mass_residual, float(raw_mass_error.max()/
    initial_mass))
        indexes = np.flatnonzero((t >= piece.t[0]-1e-8) & (t <= piece.t[-1]+1e-8))
        for chunk in np.array_split(indexes, max(1, int(np.ceil(len(indexes)/1024)
    ))):
            if not len(chunk):
                continue
            states = piece.sol(t[chunk])
            mass_error = np.abs(model.weights@states[n:2*n] + states[2*n] -
    initial_mass)
            max_mass_residual = max(max_mass_residual, float(mass_error.max()/
    initial_mass))
            if spec.name == "q1":
                energy = initial_capacity*spec.geometry.radius0**2*(
                    model.weights@(states[:n]-spec.initial.T))
                scale = max(1.0, float(np.max(np.abs(states[2*n+1]))))
```

```
                max_energy_residual = max(max_energy_residual,
                    float(np.max(np.abs(energy-states[2*n+1]))/scale))
            for j, idx in enumerate(chunk):
                # q4保存材料坐标场；r是参考半径，实际半径由meta.R逐行给出。
                xi_target = r/spec.geometry.radius0
                T[idx] = np.interp(xi_target, model.xi, states[:n,j])
                C[idx] = np.interp(xi_target, model.xi, states[n:2*n,j])
    if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C)) or np.min(C)<0:
        raise RuntimeError("非有限值或负浓度；拒绝写出")
    final_raw = pieces[-1].y[:, -1]
    event_state = pieces[-2].y[:, -1] if event_time is not None else None
    meta = {"n_internal": n, "dt_internal": f"adaptive {method}, max_step={max_step}s",
            "theta": None, "rtol": rtol, "atol": atol,
            "n_iterations": sum(x.nfev for x in pieces),
            "residual": max_mass_residual, "mass_balance_relative": max_mass_residual,
            "q1_energy_balance_relative": max_energy_residual if spec.name=="q1" else None,
            "wall_time_s": time.perf_counter()-start, "event_time_s": event_time,
            "first_output_below_s": horizon if seek_end else None,
            "coordinate": "material" if model.moving else "physical",
            "R": radii.tolist(), "method": f"nodal finite volume + {method}",
            "align_boundary_knots": align_boundary_knots,
            "max_C_final_internal": float(np.max(final_raw[n:2*n])),
            "radial_C_monotone_final": bool(np.all(np.diff(final_raw[n:2*n]) <= 1e-9)),
            "event_C": event_state[n:2*n].tolist() if event_state is not None else None,
            "event_T": event_state[:n].tolist() if event_state is not None else None,
            "event_R": float(model.radius(event_time)) if event_time is not None else None,
            "internal_xi": model.xi.tolist(), "mesh": "surface_graded_power_2"}
    meta.update(latent_heat=model.latent_heat, rho_d0_kg_m3=model.rho_d0,
                min_T_K=float(T.min()), min_C=float(C.min()),
                boundary_description=getattr(boundary, "description", "attachment effective moisture boundary"))
    return Solution(t, r, T, C, meta)
```

src/common/solution.py

```
"""求解结果容器。

```

```
放在 common/ 而不是各题的 solver 里，是为了避免 plotting / io 反向依赖 src/qN/。
"""

from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np


@dataclass
class Solution:
    """一个问题的数值解。

    形状约定（**不要改**，下游的 io/plotting 都依赖它）：
        T, C : (n_t, n_r)
        t    : (n_t,)
        r    : (n_r,)

    ⚠ 单位：内部一律 SI —— t 用 s，r 用 m，T 用 K，C 用干基 kg/kg。
       转成 cm / °C / h 只发生在 io_excel.write_result 与 plotting 里。
    """

    t: np.ndarray  # (n_t,)     s
    r: np.ndarray  # (n_r,)     m
    T: np.ndarray  # (n_t, n_r) K          # noqa: N815
    C: np.ndarray  # (n_t, n_r) kg/kg 干基  # noqa: N815
    meta: dict = field(default_factory=dict)

    def __post_init__(self) -> None:
        self.t = np.asarray(self.t, dtype=float)
        self.r = np.asarray(self.r, dtype=float)
        self.T = np.asarray(self.T, dtype=float)
        self.C = np.asarray(self.C, dtype=float)
        n_t, n_r = self.T.shape
        if self.C.shape != (n_t, n_r):
            raise ValueError(f"T 与 C 形状不一致：{self.T.shape} vs {self.C.shape}")
        if self.t.size != n_t or self.r.size != n_r:
            raise ValueError(
                f"网格与场形状不匹配：t={self.t.size}, r={self.r.size}, T={self.T.
    shape}"
            )

    @property
    def shape(self) -> tuple[int, int]:
```

```
        return self.T.shape

    @property
    def n_t(self) -> int:
        return self.t.size

    @property
    def n_r(self) -> int:
        return self.r.size

    @property
    def surface_index(self) -> int:
        """表面节点索引。"""
        return self.n_r - 1

    def center_series(self) -> np.ndarray:
        """轴心 (r=0) 的时间序列，形状 (n_t,)。"""
        return self.C[:, 0]

    def surface_series(self) -> np.ndarray:
        """表面节点的时间序列，形状 (n_t,)。"""
        return self.C[:, self.surface_index]

    def max_C_over_r(self) -> np.ndarray:
        """各时刻全场最大水分浓度，形状 (n_t,)。问题 3/4 的终点判据用。"""
        return self.C.max(axis=1)

    def first_time_below(self, target: float, *, strict: bool = True) -> float:
        """首次满足「全场水分浓度低于 target」的时刻，单位 s。

        问题 3/4 的终点判据：题目要求「药材各处的水分浓度应低于 0.15 kg/kg」，
        因此取 max_r C 而非平均值（最慢干燥的是轴心）。
        """
        m = self.max_C_over_r()
        cond = m < target if strict else m <= target
        idx = np.flatnonzero(cond)
        if idx.size == 0:
            raise ValueError(
                f"在 {self.t[-1]:.0f} s 内全场水分浓度从未低于 {target}。"
                f"当前终值 max_r C = {m[-1]:.4f}。"
                f"要么延长求解时段，要么先检查模型（AGENTS.md 铁律 10：不许静默降级）。
"
            )
        return float(self.t[idx[0]])

```

```
    def summary(self) -> str:
        return (
            f"Solution({self.n_t} x {self.n_r}) "
            f"t: {self.t[0]:.1f}~{self.t[-1]:.1f} s | "
            f"r: {self.r[0] * 100:.2f}~{self.r[-1] * 100:.2f} cm | "
            f"T: {self.T.min():.2f}~{self.T.max():.2f} K | "
            f"C: {self.C.min():.4f}~{self.C.max():.4f} kg/kg"
        )
```

src/common/units.py

```
"""单位换算。

铁律（AGENTS.md §4.1）：代码内部一律 SI —— 长度用 m，温度用 K。
只有在「写入 result*.xlsx」和「画图」的最后一步才转成 cm / °C / h。

禁止在 src/q*/ 里出现 0.01 或 273.15 字面量，用本模块的常量。
"""

from __future__ import annotations

import numpy as np
from numpy.typing import ArrayLike, NDArray

# --- 常量 ---------------------------------------------------------------------
CM_TO_M: float = 1.0e-2
M_TO_CM: float = 1.0e2
MM_TO_M: float = 1.0e-3

S_TO_H: float = 1.0 / 3600.0
H_TO_S: float = 3600.0

C_TO_K: float = 273.15  # T[K] = T[°C] + C_TO_K
K_TO_C: float = -273.15  # T[°C] = T[K] + K_TO_C

#: 绝对零度。用于合理性断言。
T_ABS_ZERO_K: float = 0.0
#: 防呆阈值：附录 3/4 的 D 公式含 exp(-3850/T)，T 必须是开尔文。
#: 传摄氏度进来会静默产生看似合理的结果，因此设这个下限做断言。
T_KELVIN_SANITY_MIN: float = 200.0


# --- 换算函数（标量或 ndarray 均可）-------------------------------------------
def cm_to_m(x: ArrayLike) -> NDArray | float:
```

```
    """厘米 → 米。"""
    return np.asarray(x) * CM_TO_M if isinstance(x, (np.ndarray, list, tuple))
    else x * CM_TO_M


def m_to_cm(x: ArrayLike) -> NDArray | float:
    """米 → 厘米。"""
    return np.asarray(x) * M_TO_CM if isinstance(x, (np.ndarray, list, tuple))
    else x * M_TO_CM


def c_to_k(x: ArrayLike) -> NDArray | float:
    """摄氏度 → 开尔文。"""
    return np.asarray(x) + C_TO_K if isinstance(x, (np.ndarray, list, tuple)) else
     x + C_TO_K


def k_to_c(x: ArrayLike) -> NDArray | float:
    """开尔文 → 摄氏度。"""
    return np.asarray(x) + K_TO_C if isinstance(x, (np.ndarray, list, tuple)) else
     x + K_TO_C


def s_to_h(x: ArrayLike) -> NDArray | float:
    """秒 → 小时。"""
    return np.asarray(x) * S_TO_H if isinstance(x, (np.ndarray, list, tuple)) else
     x * S_TO_H


def h_to_s(x: ArrayLike) -> NDArray | float:
    """小时 → 秒。"""
    return np.asarray(x) * H_TO_S if isinstance(x, (np.ndarray, list, tuple)) else
     x * H_TO_S


# --- 断言 ---------------------------------------------------------------------
def assert_kelvin(T: ArrayLike, *, where: str = "") -> None:
    """防呆：确认温度是开尔文。

    附录 3/4 的扩散系数公式含 exp(-3850/T)。若把 60 °C 当成 60 传进去，
    数值会错得离谱却「看起来还在合理范围」——这是本项目最危险的一类错误。
    """
    arr = np.asarray(T, dtype=float)
    if not np.all(np.isfinite(arr)):
        raise ValueError(f"温度含 NaN/Inf{_at(where)}")
```

```
    if arr.size and float(np.min(arr)) < T_KELVIN_SANITY_MIN:
        raise ValueError(
            f"温度最小值为 {float(np.min(arr)):.3f}，低于防呆阈值 "
            f"{T_KELVIN_SANITY_MIN} K。看起来你传的是摄氏度而不是开尔文"
            f"{_at(where)}。请用 units.c_to_k() 转换。"
        )


def assert_positive(x: ArrayLike, *, name: str, where: str = "") -> None:
    """防呆：物理量必须为正（半径、密度、扩散系数等）。"""
    arr = np.asarray(x, dtype=float)
    if not np.all(np.isfinite(arr)):
        raise ValueError(f"{name} 含 NaN/Inf{_at(where)}")
    if arr.size and float(np.min(arr)) <= 0.0:
        raise ValueError(f"{name} 出现非正值（最小 {float(np.min(arr)):.6g}）{_at(
    where)}")


def _at(where: str) -> str:
    return f"（位置：{where}）" if where else ""
```

src/q1/solver.py

```
"""问题1入口；物理和数值规格见 docs/model/numerics.md。"""
from common.radial import solve_radial


def solve(spec, **opts):
    return solve_radial(spec, **opts)
```

src/q2/solver.py

```
"""问题2入口；物理和数值规格见 docs/model/numerics.md。"""
from common.radial import solve_radial


def solve(spec, **opts):
    return solve_radial(spec, **opts)
```

src/q3/solver.py

```
"""问题3入口；物理和数值规格见 docs/model/numerics.md。"""
```

```python
from common.radial import solve_radial


def solve(spec, **opts):
    return solve_radial(spec, **opts)
```

src/q4/solver.py

```python
"""问题4入口；物理和数值规格见 docs/model/numerics.md。"""
from common.radial import solve_radial


def solve(spec, **opts):
    return solve_radial(spec, **opts)
```

scripts/_bootstrap.py

```python
"""脚本引导：把仓库根的 src/ 加入 sys.path。

用法（每个 scripts/*.py 的第一行导入）：
    from _bootstrap import ROOT, setup  # noqa: F401
    setup()

这样无需 `pip install -e .`，`python scripts/run_q1.py` 也能直接跑。
tests/ 走的是 pyproject.toml 里的 pythonpath 配置，不需要这个。
"""

from __future__ import annotations

import sys
from pathlib import Path

ROOT: Path = Path(__file__).resolve().parents[1]
SRC: Path = ROOT / "src"


def setup() -> None:
    """幂等地把 src/ 放到 sys.path 最前面，并确保输出目录存在。"""
    s = str(SRC)
    if s not in sys.path:
        sys.path.insert(0, s)

    from common import paths
```

```

    paths.ensure_output_dirs()
```

scripts/_runner.py

```
"""run_q1~q4 的公共执行逻辑。

流程: spec -> solver.solve -> 交付表格 -> 写 xlsx -> 打印摘要

⚠ 求解器尚未实现（见 src/qN/solver.py）。本文件负责的是**流程与交付**，
   流程本身是通的：一旦 solver.solve 有实现，`make qN` 立刻能出交付物。
"""

from __future__ import annotations

import sys
import time
import traceback
import json
import hashlib
import platform
import numpy as np
import scipy

from _bootstrap import setup

setup()

from common import export, io_excel, paths, problem_spec  # noqa: E402
from common.numerical_settings import production_options


def run(problem: str, *, boundary=None) -> int:
    spec = problem_spec.get(problem)
    print("=" * 72)
    print(spec.describe())
    print("=" * 72)

    # 1) 求解
    t0 = time.perf_counter()
    try:
        solver = __import__(f"{spec.name}.solver", fromlist=["solve"])
    except ImportError as e:
        print(f"!! 无法导入 {spec.name}.solver: {e}")
```

```
        return 1

    try:
        options = production_options(problem)
        if boundary is not None:
            options["boundary"] = boundary
        sol = solver.solve(spec, **options)
    except NotImplementedError as e:
        print()
        print(f"⏸  {spec.name} 的求解器尚未实现。这不是故障，是框架的预期状态。")
        print()
        print(str(e))
        print()
        print("下一步：")
        print(f"  1. @model 完成 docs/model/{spec.name}.md（控制方程 + 离散格式 + 边界条件）")
        print(f"  2. @impl 实现 src/{spec.name}/solver.py")
        print(f"  3. 重跑 make {spec.name}")
        return 2
    except Exception:
        print("!! 求解过程抛出异常：")
        traceback.print_exc()
        return 1

    wall = time.perf_counter() - t0
    sol.meta.setdefault("wall_time_s", round(wall, 3))
    print(f"  求解完成，耗时 {wall:.2f} s")
    print("  " + export.summarize(sol, spec))
    if sol.meta.get("event_time_s") is not None and sol.meta["max_C_final_internal"] >= spec.extra.get("target_C", 0.15):
        raise RuntimeError("下一输出时刻未严格低于阈值，拒绝误标达标结果")
    print("  正在流式写入结果文件……", flush=True)

    # 2) 交付物
    try:
        sheets = export.result_sheets(sol, spec)
        dest = io_excel.write_result(spec.name, sheets)
    except io_excel.ResultFormatError as e:
        print()
        print("!! 交付格式校验未通过，**没有写出文件**：")
        print(str(e))
        return 1

    print(f"  交付物 {paths.rel(dest)}  ({dest.stat().st_size} B)")
    for name, df in sheets.items():
```

```
        print(f"    工作表「{name}」 {df.shape[0]} 行 x {df.shape[1]} 列")

    # 3) 论文用展示网格
    _write_table_csvs(sol, spec)
    metadata = {k:v for k,v in sol.meta.items() if k not in {"R","event_T","event_C"}}
    (paths.RESULTS_DIR / f"{problem}_summary.json").write_text(
        json.dumps(metadata, indent=2, ensure_ascii=False), encoding="utf-8")
    paths.OUTPUTS_CACHE.mkdir(parents=True, exist_ok=True)
    np.savez_compressed(paths.OUTPUTS_CACHE / f"{problem}.npz", t=sol.t, r=sol.r,
                        T=sol.T,C=sol.C,meta=json.dumps(sol.meta))

    if problem in {"q1", "q2", "q3", "q4"}:
        inputs = [paths.ROOT / "problem" / "A题.pdf",
                  paths.ROOT / "problem" / "附件" / "附件1.xlsx",
                  paths.ROOT / "problem" / "附件" / "附件3" / f"result{problem[-1]}.xlsx"]
        if problem == "q4":
            inputs.append(paths.ROOT / "problem" / "附件" / "附件2.xlsx")
        manifest = {
            "question_id": problem, "model_version": "latent-v2",
            "numerical_version": "latent-validated-v1", "status": "computed",
            "run_command": f"python scripts/run_{problem}.py",
            "parameters": production_options(problem), "python": platform.python_version(),
            "packages": {"numpy": np.__version__, "scipy": scipy.__version__},
            "input_hashes": {str(p.relative_to(paths.ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
                             for p in inputs},
            "event_time_s": sol.meta["event_time_s"],
            "mass_balance_relative": sol.meta["mass_balance_relative"],
            "validation": "results/validation/latent.json",
            "submission_file": str(dest.relative_to(paths.ROOT)),
            "submission_sha256": hashlib.sha256(dest.read_bytes()).hexdigest(),
            "boundary_description": sol.meta["boundary_description"],
            "latent_heat": sol.meta["latent_heat"],
            "source_hashes": {str(p.relative_to(paths.ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
                              for p in sorted((paths.ROOT / "src").rglob("*.py"))},
            "limitations": ["One-dimensional radial model; end effects omitted",
                            "Prescribed empirical heat capacity and shrinkage"],
        }
        (paths.RESULTS_DIR / f"{problem}_manifest.json").write_text(
            json.dumps(manifest, indent=2, ensure_ascii=False), encoding="utf-8")
```

```
122
123     print()
124     print("  提示：别忘了 make tables figs，以及更新 docs/TASKS.md")
125     return 0
126
127
128 def _write_table_csvs(sol, spec) -> None:
129     """把论文要用的展示网格写成 csv，供 make tables 消费。"""
130     import numpy as np
131
132     from common import problem_spec as ps
133
134     made = []
135     if spec.name == "q1":
136         times = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
137         made.append(("q1_temp", export.table_rows(sol, spec, times_s=times, field=
    "T")))
138         made.append(("q1_moist", export.table_rows(sol, spec, times_s=times, field
    ="C")))
139     elif spec.name == "q2":
140         times = np.arange(0.5, 3.01, 0.5) * 3600.0
141         made.append(("q2_temp", export.table_rows(sol, spec, times_s=times, field=
    "T")))
142         made.append(("q2_moist", export.table_rows(sol, spec, times_s=times, field
    ="C")))
143     else:
144         # 问题 3/4：每隔 6 h 一行，最后一行是烘干结束时间
145         t_end = float(sol.meta["event_time_s"])
146         times = list(np.arange(6.0, t_end / 3600.0, 6.0) * 3600.0) + [t_end]
147         # 问题 4 表 6 的末列是「药材表面」而非固定 2 cm，故只取 0~1.5 的固定列
148         dists = ps.TABLE_R_CM[:-1] if spec.uses_shrinkage else ps.TABLE_R_CM
149         made.append(
150             (
151                 f"{spec.name}_moist",
152                 export.table_rows(
153                     sol,
154                     spec,
155                     times_s=np.array(times),
156                     distances_cm=dists,
157                     include_surface=spec.uses_shrinkage,
158                 ),
159             )
160         )
161
162     for name, df in made:
```

```
        p = io_excel.write_table_csv(df.reset_index(), name)
        print(f"  展示表格 {paths.rel(p)}  ({df.shape[0]} 行)")


def main(problem: str) -> int:
    return run(problem)


if __name__ == "__main__":
    sys.exit(main(sys.argv[1] if len(sys.argv) > 1 else "q1"))
```

scripts/audit_q4_boundary_layer.py

```
"""不重算：分别核验Q4全部材料节点与题目指定的固定物理输出位置。"""
import json
import numpy as np
from _bootstrap import setup
setup()
from common import export, paths
from common.problem_spec import Q4
from common.solution import Solution


def main():
    report=json.loads((paths.RESULTS_DIR/"validation/latent.json").read_text(encoding="utf-8"))
    rows=[];previous=None
    for row in [g for g in report['grid'] if g['q']=='q4']:
        n=row['n'];s=None
        for path in (paths.OUTPUTS_CACHE/"latent_validation").glob(f"q4_grid{n}_*.npz"):
            with np.load(path) as f:
                meta=json.loads(str(f['meta']))
                if abs(meta['event_time_s']-row['event_s'])<1e-7:
                    s=Solution(f['t'],f['r'],f['T'],f['C'],meta)
                    break
        if s is None:
            raise FileNotFoundError(f"缺少当前验证q4 {n}的精确缓存")
        if previous is not None:
            material=np.array([np.interp(previous.r,s.r,a) for a in s.C])
            error=abs(material-previous.C);i,j=np.unravel_index(np.argmax(error),error.shape)
            output=export.result_sheets(s,Q4)['Sheet1'].iloc[:,1:].to_numpy()
            old=export.result_sheets(previous,Q4)['Sheet1'].iloc[:,1:].to_numpy()
```

```
            rows.append({'coarse_n':previous.n_r,'fine_n':n,'material_max_C_difference':float(error[i,j]),
                         'at_time_s':float(s.t[i]),'at_xi':float(previous.r[j]/Q4.geometry.radius0),
                         'surface_distance_m':float((1-previous.r[j]/Q4.geometry.radius0)*s.meta['R'][i]),
                         'physical_output_max_C_difference':float(np.nanmax(abs(output-old))),
                         'surface_max_C_difference':float(np.max(abs(s.C[:,-1]-previous.C[:,-1])))})
        previous=s
    out={'rows':rows,'limitation':'Maximum material-field differences occur in an unresolved submicron surface layer. The delivered fixed physical positions and explicit surface value converge separately; no uniform four-decimal field accuracy is claimed.'}
    (paths.RESULTS_DIR/'validation/q4_boundary_layer.json').write_text(json.dumps(out,indent=2),encoding='utf-8')
    print(json.dumps(out,indent=2),flush=True)


if __name__=='__main__':
    main()
```

scripts/build_paper_assets.py

```
"""从正式结果、缓存、验证报告生成论文表图和全部数值宏。"""
import json
import numpy as np
import pandas as pd
import openpyxl
from _bootstrap import setup
setup()
from common import paths, units, plotting
from common.boundary import Boundary
from common.psychrometrics import humidity_ratio
from common.numerical_settings import production_options
from common.solution import Solution


def load(q):
    with np.load(paths.OUTPUTS_CACHE/f"{q}.npz") as f:
        return Solution(f["t"],f["r"],f["T"],f["C"],json.loads(str(f["meta"])))


```

```
def report():
    return json.loads((paths.RESULTS_DIR/"validation/latent.json").read_text(encoding="utf-8"))


def write_table(df, caption, label, name):
    from make_tables import to_booktabs
    (paths.TABLES_DIR/f"{name}.tex").write_text(to_booktabs(df,caption,label),encoding="utf-8")


def tables():
    from make_tables import TABLES
    paths.TABLES_DIR.mkdir(parents=True,exist_ok=True)
    # 规则时刻直接来自正式xlsx；非整步事件行来自同次求解事件CSV。
    for key,(caption,label) in TABLES.items():
        df=pd.read_csv(paths.RESULTS_TABLES/f"{key}.csv")
        q=key[:2]
        scale=1. if q=="q1" else 3600.
        requested=df.iloc[:,0].to_numpy()*scale
        wanted={int(round(t)):i for i,t in enumerate(requested) if abs(t-round(t))<1e-5 and (q in ('q1','q2') or i<len(requested)-1)}
        book=openpyxl.load_workbook(paths.result_path(q),read_only=True,data_only=True)
        sheet=("温度" if key.endswith('temp') else "水分浓度") if q in ('q1','q2') else "Sheet1"
        iterator=book[sheet].iter_rows(values_only=True)
        headers=list(next(iterator)); positions=[headers.index(str(c)) for c in df.columns[1:]]
        for row in iterator:
            if row[0] in wanted:
                df.iloc[wanted[row[0]],1:]=[np.nan if row[j] is None else row[j] for j in positions]
            if row[0]>=max(wanted):
                break
        book.close()
        df=df.rename(columns={df.columns[0]:"时间/s" if key.startswith("q1") else "时间/h"})
        unit="°C" if key.endswith("temp") else "kg/kg"
        write_table(df,caption+f"（{unit}；距离单位：cm）",label,key)
    summaries={q:json.loads((paths.RESULTS_DIR/f"{q}_summary.json").read_text(encoding="utf-8")) for q in ("q1","q2","q3","q4")}
    v=report()
    s1,s2=load("q1"),load("q2")
    i2=int(np.searchsorted(s2.t,10800.))
```

```
    scalars={}
    for name,s,i in (("QOne",s1,-1),("QTwo",s2,i2)):
        for suffix,field,j in (("CenterT","T",0),("SurfaceT","T",-1),("CenterC","C",0),("SurfaceC","C",-1)):
            value=getattr(s,field)[i,j]
            scalars[name+suffix]=f"{units.k_to_c(value) if field=='T' else value:.4f}"
    for q,name in (("q3","QThree"),("q4","QFour")):
        scalars[name+"EndH"]=f"{summaries[q]['event_time_s']/3600.:.4f}"
        scalars[name+"StrictEndH"]=f"{summaries[q]['first_output_below_s']/3600.:.4f}"
    scalars["QFourEndRadiusCm"]=f"{100*summaries['q4']['event_R']:.4f}"
    scalars.update(EnvTempC="60",EnvRHPercent="20",EnvC=f"{humidity_ratio(333.15,.20):.6f}")
    options=production_options("q1")
    scalars.update(ProductionNodes=str(options["n_internal"]),ProductionRtol=f"{options['rtol']:.1e}",ProductionAtol=f"{options['atol']:.1e}",ProductionMaxStep=f"{options['max_step']:g}",
                   QOneMassError=f"{summaries['q1']['mass_balance_relative']:.2e}",QOneEnergyError=f"{summaries['q1']['q1_energy_balance_relative']:.2e}",
                   MaxMassError=f"{max(s['mass_balance_relative'] for s in summaries.values()):.2e}")
    from common.radial import RadialModel
    from common.problem_spec import Q1,Q3,Q4
    audit=[]
    for spec in (Q1,Q3,Q4):
        m=RadialModel(spec,41)
        for t in (0.,7200.,86400.):
            y=m.initial(); y[:41]+=2*m.xi**2; y[41:82]-=.2*m.xi**2
            dy=m.rhs(t,y)
            capacity=spec.properties.rho(y[41:82],y[:41])*spec.properties.cp(y[41:82],y[:41])
            value=abs(m.radius(t)**2*np.dot(m.weights*capacity,dy[:41])-dy[83])
            audit.append({"q":spec.name,"t_s":t,"absolute_residual_W_per_m":float(value)})
    (paths.RESULTS_DIR/"validation/energy_audit.json").write_text(json.dumps(audit,indent=2),encoding="utf-8")
    scalars["MaxInstantEnergyError"]=f"{max(x['absolute_residual_W_per_m'] for x in audit):.2e}"
    fine=[g for g in v["grid"] if g["n"]==2561]
    scalars.update(MaxGridTempError=f"{max(g['delta_T_K'] for g in fine):.2e}",MaxGridMoistError=f"{max(g['delta_C'] for g in fine):.2e}",MaxGridEndErrorH=f"{max(g['delta_event_s'] or 0 for g in fine)/3600.:.2e}",
                   MaxTimeTempError=f"{max(g['delta_T_K'] for g in v['time']):.2e}",MaxTimeMoistError=f"{max(g['delta_C'] for g in v['time']):.2e}",
```

```
    MaxTimeEndErrorH=f"{max(g['delta_event_s'] or 0 for g in v['time'])/3600.:.2e}"
    )
    fixed=next(a for a in v["ablation"] if a["case"]=="fixed_radius_same_appendix4
    ")
    scalars["QFourShrinkReductionPercent"]=f"{100*(1-fixed['base_h']/fixed['
    event_h']):.2f}"
    scalars["QFourFixedEndH"]=f"{fixed['event_h']:.4f}"
    layer=json.loads((paths.RESULTS_DIR/"validation/q4_boundary_layer.json").
    read_text(encoding="utf-8"))
    scalars["OutputGridCError"]=f"{layer['rows'][-1]['
    physical_output_max_C_difference']:.2e}"
    fitted_check=json.loads((paths.RESULTS_DIR/"validation/fitted_numerics.json").
    read_text(encoding="utf-8"))
    scalars["FitOutputGridCError"]=f"{max(r['delta_output_C'] for r in
    fitted_check['rows']):.2e}"
    scalars["FitGridEndErrorS"]=f"{max(r['delta_event_s'] for r in fitted_check['
    rows']):.4f}"
    rows=[]
    for g in v["grid"]:
        if "delta_T_K" in g:
            rows.append([g["q"].upper(),g["n"],f"{g['delta_T_K']:.2e}",f"{g['
    delta_C']:.2e}","—" if g["delta_event_s"] is None else f"{g['delta_event_s
    ']:.4f}"])
    write_table(pd.DataFrame(rows,columns=["题号","节点数","温度差/K","含水率差","终
    点差/s"]),"相邻加密网格比较（最大场差与连续终点差）","tab:all-validation","
    all_validation")
    label={"55C_20RH":"55°C、20\\% RH","60C_40RH":"60°C、40\\% RH","Lewis_one":"
    Lewis数为1","equilibrium_C_005":"供水平衡含水率0.05"}
    rows=[[x["q"].upper(),label[x["case"]],x["event_h"],x["delta_h"]] for x in v["
    sensitivity"]]
    for x in v["ablation"]:
        rows.append([x["q"].upper(),"同参数不计潜热" if x["case"]=="no_latent" else
    "附录4固定半径",x["event_h"],x["event_h"]-x["base_h"]])
    write_table(pd.DataFrame(rows,columns=["题号","比较条件","时间/h","相对基准差/h"
    ]),"工况与结构假设敏感性（161节点，全部为方案计算）","tab:sensitivity","sensitivity
    ")
    fit_path=paths.RESULTS_DIR/"validation/ambient_fit.json"
    if fit_path.exists():
        fit=json.loads(fit_path.read_text(encoding="utf-8"))
        selected=fit["selected"]
        scalars.update(FitTempC=f"{selected['T_inf_K']-273.15:.4f}",FitTempLowC=f"
    {selected['T_inf_ci95_K'][0]-273.15:.4f}",FitTempHighC=f"{selected['
    T_inf_ci95_K'][1]-273.15:.4f}",FitRMSE=f"{selected['rmse_K']:.4f}")
        scalars["FitHoldoutRMSE"]=f"{selected['holdout_rmse_K']:.4f}"
        params=fit['candidates']['logistic_fixed_initial']['parameters']
```

```
        scalars["FitShapeB"]=f"{params['B_dimensionless']:.6f}"
        scalars["FitTauS"]=f"{params['tau_s']:.4f}"
        modelnames={'exponential_free':'自由初值指数','exponential_fixed_initial':'固定初值指数','logistic_fixed_initial':'有限平台Logistic','weibull_fixed_initial':'Weibull饱和曲线'}
        rows=[[modelnames[k],item['T_asymptote_C'],item['full_fit']['rmse_K'],item['holdout']['rmse_K']] for k,item in fit['candidates'].items()]
        write_table(pd.DataFrame(rows,columns=['候选模型','平台/°C','全段RMSE/K','留出RMSE/K']),"附件温度曲线的拟合候选比较","tab:ambient-models","ambient_models")
        scenario_path=paths.RESULTS_DIR/"scenarios/fitted_temperature/comparison.json"
        if scenario_path.exists():
            scenario=json.loads(scenario_path.read_text(encoding="utf-8"))
            for q,name in (("q3","FitQThreeEndH"),("q4","FitQFourEndH")):
                scalars[name]=f"{scenario[q]['event_time_s']/3600:.4f}"
            rows=[["文献设定",60.,float(scalars['QThreeEndH']),float(scalars['QFourEndH'])],["附件拟合渐近值",selected['T_inf_K']-273.15,float(scalars['FitQThreeEndH']),float(scalars['FitQFourEndH'])]]
            write_table(pd.DataFrame(rows,columns=["恒温方案","温度/°C","Q3时间/h","Q4时间/h"]),"两种恒温温度来源的比较（相对湿度均为20\\%）","tab:temperature-scenarios","temperature_scenarios")
    (paths.TABLES_DIR/"all_values.tex").write_text("% 自动生成，不得手抄结果\n"+"\n".join("\\newcommand{\\"+k+"}{"+val+"}" for k,val in scalars.items())+"\n",encoding="utf-8")
    notation=(paths.ROOT/"docs/NOTATION.md").read_text(encoding="utf-8")
    section=notation.split("## 0.",1)[1].split("## 1.",1)[0]
    rows=[]
    for line in section.splitlines():
        cells=[x.strip() for x in line.strip().strip("|").split("|")]
        if len(cells)==4 and cells[1].startswith("`\\"):
            macro=cells[1].replace("`","").replace(",",r",\,")
            rows.append("$"+macro+"$ & "+cells[3].replace("_",r"\_")+r" \\")
    (paths.TABLES_DIR/"notation.tex").write_text("\\begin{longtable}{lp{10cm}}\n\\toprule 符号 & 含义 \\\\ \\midrule\n"+"\n".join(rows)+"\n\\bottomrule\\end{longtable}\n",encoding="utf-8")
    print("Generated all paper tables and result macros.",flush=True)
    return 0


def figures():
    from matplotlib import pyplot as plt
    plotting.apply_style()
    plt.rcParams.update({'font.size':11,'axes.labelsize':12,'axes.titlesize':12,
                         'legend.fontsize':11,'xtick.labelsize':11,'ytick.labelsize':11})
```

```
    def save(fig,name):
        fig.savefig(paths.FIGURES_DIR/f"{name}.pdf",bbox_inches="tight")
        plt.close(fig)
    s1,s2,s3,s4=(load(f"q{i}") for i in range(1,5))
    for field in ("T","C"):
        fig,ax=plt.subplots(figsize=(6.0,4.0))
        for t in (100,600,1200,1800):
            i=int(np.searchsorted(s1.t,t))
            y=s1.T[i]-273.15 if field=="T" else s1.C[i]
            ax.plot(s1.r*100,y,marker='o',markersize=2.5,label=f"t = {t} s")
        ax.set(xlabel="到药材中心的距离 / cm",ylabel="温度 / °C" if field=="T" else
    "干基含水率 / (kg/kg)")
        ax.legend();save(fig,f"fig1-{field}-profiles")
    b=Boundary.for_problem("q2")
    fig,ax=plt.subplots(2,1,figsize=(6.0,6.5))
    mask=s2.t<=10800
    for j,label in ((0,"轴心"),(-1,"表面")):
        ax[0].plot(s2.t[mask]/3600,s2.T[mask,j]-273.15,label=label)
        ax[1].plot(s2.t[mask]/3600,s2.C[mask,j],label=label)
    ax[0].plot(s2.t[mask]/3600,b.ambient(s2.t[mask])[0]-273.15,"--",label="烘房")
    for a,y in zip(ax,("温度 / °C","干基含水率 / (kg/kg)")):
        a.set(xlabel="时间 / h",ylabel=y);a.legend()
    fig.tight_layout();save(fig,"fig2-history")
    fig,ax=plt.subplots(figsize=(6.0,4.0))
    for j in (0,5,10,15,20):
        ax.plot(s3.t/3600,s3.C[:,j],label=f"r={j/10:g} cm")
    ax.axhline(.15,color="black",ls="--",label="各处达标阈值")
    ax.set(xlabel="时间 / h",ylabel="干基含水率 / (kg/kg)");ax.legend(ncol=2);save(
    fig,"fig3-drying")
    fig,ax=plt.subplots(2,1,figsize=(6.0,6.5))
    ax[0].plot(s4.t/3600,np.array(s4.meta['R'])*100)
    ax[0].set(xlabel="时间 / h",ylabel="药材半径 / cm")
    for t in (6*3600,18*3600,30*3600,s4.t[-1]):
        i=int(np.argmin(abs(s4.t-t)))
        ax[1].plot(s4.r/s4.r[-1]*s4.meta['R'][i]*100,s4.C[i],label=f"{s4.t[i
    ]/3600:.1f} h")
    ax[1].set(xlabel="实际径向位置 / cm",ylabel="干基含水率 / (kg/kg)");ax[1].legend
    ()
    fig.tight_layout();save(fig,"fig4-shrinkage")
    v=report();fig,ax=plt.subplots(2,1,figsize=(6.0,6.5))
    for q in ('q1','q3','q4'):
        rows=[g for g in v['grid'] if g['q']==q and 'delta_T_K' in g]
        for a,key in zip(ax,('delta_T_K','delta_C')):
            a.loglog([g['n'] for g in rows],[g[key] for g in rows],'o-',label=q.
    upper())
```

```
    for a,y in zip(ax,('相邻网格最大温度差 / K','相邻网格最大含水率差')):
        a.set(xlabel="细网格节点数",ylabel=y);a.legend()
    fig.tight_layout();save(fig,"fig-validation")
    path=paths.DATA_PROCESSED/"ambient_fit.csv"
    if path.exists():
        df=pd.read_csv(path)
        fig,ax=plt.subplots(figsize=(6.0,4.0))
        ax.scatter(df['t_s']/3600,df['T_observed_K']-273.15,s=7,label="附件1")
        ax.plot(df['t_s']/3600,df['T_fitted_K']-273.15,label="有限平台Logistic拟合")
        held=df['t_s']>10800
        ax.plot(df.loc[held,'t_s']/3600,df.loc[held,'T_train3h_logistic_fixed_initial_K']-273.15,':',label="前3 h拟合的留出预测")
        fit=json.loads((paths.RESULTS_DIR/"validation/ambient_fit.json").read_text(encoding="utf-8"))
        ax.axhline(fit['selected']['T_inf_K']-273.15,ls="--",color="black",label="拟合渐近温度")
        ax.set(xlabel="时间 / h",ylabel="烘房温度 / °C");ax.legend();save(fig,"fig-ambient-fit")
    print("Generated all paper figures.",flush=True)
    return 0


if __name__=="__main__":
    import sys
    if '--figures-only' not in sys.argv:
        tables()
    figures()
```

scripts/fit_ambient_temperature.py

```
"""Fit an asymptotic oven temperature to attachment 1, using K and seconds.

This estimates a conditional setpoint from the four-hour record. It does not
claim that an uncontrolled oven will follow the fitted asymptote indefinitely.
Only results/validation/ambient_fit.json and data/processed/ambient_fit.csv
are written. No PDE simulation or Git operation is performed.
"""

from __future__ import annotations

import argparse
import hashlib
import json
```

```
import platform
import warnings

import numpy as np
import pandas as pd
import scipy
from scipy.optimize import OptimizeWarning, curve_fit
from scipy.special import expit
from scipy.stats import linregress, t as student_t

from _bootstrap import setup

setup()

from common.io_excel import read_attachment  # noqa: E402
from common.paths import ROOT, attachment_path  # noqa: E402
from common.units import c_to_k, k_to_c  # noqa: E402


def candidate_models(T0):
    """Monotone heating candidates with finite asymptotes, all in SI units."""
    def exponential_free(t, S, A, tau):
        return S - A * np.exp(-t / tau)

    def exponential_fixed_initial(t, S, tau):
        return S - (S - T0) * np.exp(-t / tau)

    def logistic_fixed_initial(t, S, B, tau):
        # Algebraically: S-(S-T0)*(1+B)/(1+B*exp(t/tau)).
        # expit prevents overflow for the broad exploratory tau bounds.
        return S - (S - T0) * (1 + B) * expit(-np.log(B) - t / tau)

    def weibull_fixed_initial(t, S, tau, beta):
        return S - (S - T0) * np.exp(-(t / tau) ** beta)

    return {
        "exponential_free": {
            "function": exponential_free,
            "parameter_names": ["S_K", "A_K", "tau_s"],
            "formula": "T(t)=S-A*exp(-t/tau)",
            "initial": [T0 + 22, 22, 1800],
            "bounds": ([T0, 0, 1], [T0 + 200, 300, 144000]),
            "initial_temperature_fixed": False,
        },
        "exponential_fixed_initial": {
```

```
            "function": exponential_fixed_initial,
            "parameter_names": ["S_K", "tau_s"],
            "formula": "T(t)=S-(S-T0)*exp(-t/tau)",
            "initial": [T0 + 22, 1800],
            "bounds": ([T0, 1], [T0 + 200, 144000]),
            "initial_temperature_fixed": True,
        },
        "logistic_fixed_initial": {
            "function": logistic_fixed_initial,
            "parameter_names": ["S_K", "B_dimensionless", "tau_s"],
            "formula": "T(t)=S-(S-T0)*(1+B)/(1+B*exp(t/tau))",
            "initial": [T0 + 22, 2, 1800],
            "bounds": ([T0, 0.001, 100], [T0 + 200, 1000, 144000]),
            "initial_temperature_fixed": True,
        },
        "weibull_fixed_initial": {
            "function": weibull_fixed_initial,
            "parameter_names": ["S_K", "tau_s", "beta_dimensionless"],
            "formula": "T(t)=S-(S-T0)*exp(-(t/tau)**beta)",
            "initial": [T0 + 22, 1800, 1.1],
            "bounds": ([T0, 1, 0.25], [T0 + 200, 144000, 3]),
            "initial_temperature_fixed": True,
        },
    }


def fit_model(model, t, T, initial=None):
    with warnings.catch_warnings():
        warnings.simplefilter("error", OptimizeWarning)
        params, covariance = curve_fit(
            model["function"], t, T,
            p0=model["initial"] if initial is None else initial,
            bounds=model["bounds"], maxfev=20000,
            ftol=1e-12, xtol=1e-12, gtol=1e-12,
        )
    if not np.all(np.isfinite(params)) or not np.all(np.isfinite(covariance)):
        raise RuntimeError("Nonfinite fitted parameters or covariance")
    return params, covariance


def diagnostics(residual, parameter_count):
    residual = np.asarray(residual)
    n = residual.size
    sse = float(residual @ residual)
    aic = n * np.log(sse / n) + 2 * parameter_count
```

```
    return {
        "sample_count": n,
        "rmse_K": float(np.sqrt(sse / n)),
        "mae_K": float(np.mean(np.abs(residual))),
        "max_abs_error_K": float(np.max(np.abs(residual))),
        "mean_observed_minus_fitted_K": float(np.mean(residual)),
        "sse_K2": sse,
        "aicc": float(aic + 2 * parameter_count * (parameter_count + 1)
                      / (n - parameter_count - 1)),
        "lag1_residual_correlation": float(np.corrcoef(residual[:-1], residual
    [1:])[0, 1]),
        "durbin_watson": float(np.diff(residual) @ np.diff(residual) / sse),
    }


def param_mapping(model, parameters):
    return dict(zip(model["parameter_names"], map(float, parameters)))


def covariance_report(model, parameters, covariance, sample_count):
    se = np.sqrt(np.diag(covariance))
    quantile = float(student_t.ppf(0.975, sample_count - len(parameters)))
    interval = np.column_stack((parameters - quantile * se, parameters + quantile
    * se))
    correlation = covariance / np.outer(se, se)
    return {
        "parameter_order": model["parameter_names"],
        "covariance": covariance.tolist(),
        "parameter_correlation": correlation.tolist(),
        "standard_errors": param_mapping(model, se),
        "conditional_95pct_intervals": dict(zip(model["parameter_names"], interval
    .tolist())),
        "S_95pct_C": k_to_c(interval[0]).tolist(),
        "assumptions": "Local nonlinear least-squares covariance; independent
    constant-variance residuals; candidate and fixed T0 treated as known; not a
    future-temperature prediction interval.",
    }


def bootstrap_asymptote(model, t, T, parameters, count, seed, block_length):
    prediction = model["function"](t, *parameters)
    residual = T - prediction
    # The observed initial value anchors fixed-initial candidates; do not
    # resample that exactly constrained observation as independent noise.
    start = 1 if model["initial_temperature_fixed"] else 0
```

```
    pool = residual[start:] - np.mean(residual[start:])
    rng = np.random.default_rng(seed)
    samples = []
    failures = []
    for iteration in range(count):
        block_starts = rng.integers(0, len(pool), size=int(np.ceil(len(pool) / block_length)))
        indexes = (block_starts[:, None] + np.arange(block_length)) % len(pool)
        noise = pool[indexes.ravel()[:len(pool)]]
        simulated = prediction.copy()
        simulated[start:] += noise
        if start:
            simulated[0] = T[0]
        try:
            fitted, _ = fit_model(model, t, simulated, initial=parameters)
            samples.append(float(fitted[0]))
        except (RuntimeError, ValueError, OptimizeWarning) as error:
            failures.append({"iteration": iteration, "error": str(error)})
    if len(samples) < 0.95 * count:
        raise RuntimeError(f"Bootstrap failed for {len(failures)}/{count} replicates")
    return {
        "method": "circular moving-block residual bootstrap" if block_length > 1 else "iid centered residual bootstrap",
        "replicates_requested": count,
        "replicates_successful": len(samples),
        "seed": seed,
        "block_length_observations": block_length,
        "block_duration_s": float(block_length * np.median(np.diff(t))),
        "S_95pct_K": np.quantile(samples, [0.025, 0.975]).tolist(),
        "S_95pct_C": k_to_c(np.quantile(samples, [0.025, 0.975])).tolist(),
        "S_standard_deviation_K": float(np.std(samples, ddof=1)),
        "failures": failures,
        "interpretation": "Conditional parameter uncertainty with measured T0 fixed. Model choice, sensor bias, future controller changes and model-form error are not included.",
    }


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--bootstrap", type=int, default=500)
    args = parser.parse_args()
    if args.bootstrap < 100:
        raise ValueError("At least 100 bootstrap replicates are required")
```

```
    raw = read_attachment(1)
    t = raw["时间"].to_numpy(dtype=float)
    T = c_to_k(raw["温度"].to_numpy(dtype=float))
    if not np.all(np.isfinite(t)) or not np.all(np.isfinite(T)) or np.any(np.diff(t) <= 0):
        raise ValueError("Attachment time and temperature must be finite and time strictly increasing")
    if t[0] != 0.0 or t[-1] != 14400.0:
        raise ValueError("Expected attachment 1 to cover 0–14400 s")
    train = t <= 10800.0
    validation = ~train
    T0 = float(T[0])
    candidates = candidate_models(T0)
    summaries = {}
    fitted = {}
    frame = {"t_s": t, "T_observed_K": T, "T_observed_C": k_to_c(T),
             "holdout_3_to_4h": validation.astype(int)}
    for name, model in candidates.items():
        p, cov = fit_model(model, t, T)
        pt, covt = fit_model(model, t[train], T[train])
        prediction = model["function"](t, *p)
        cv_prediction = model["function"](t, *pt)
        summaries[name] = {
            "formula": model["formula"],
            "parameter_count": len(p),
            "initial_temperature_fixed": model["initial_temperature_fixed"],
            "parameters": param_mapping(model, p),
            "T_asymptote_C": float(k_to_c(p[0])),
            "full_fit": diagnostics(T - prediction, len(p)),
            "holdout": diagnostics(T[validation] - cv_prediction[validation], len(p)),
            "training_fit": diagnostics(T[train] - cv_prediction[train], len(p)),
            "training_parameters": param_mapping(model, pt),
            "training_covariance": covt.tolist(),
            "parameter_bounds": [list(x) for x in model["bounds"]],
            "uncertainty": covariance_report(model, p, cov, len(t)),
        }
        fitted[name] = (p, cov)
        frame[f"T_{name}_K"] = prediction
        frame[f"residual_{name}_K"] = T - prediction
        frame[f"T_train3h_{name}_K"] = cv_prediction
        print(f"{name}: S={k_to_c(p[0]):.8f} C, RMSE={summaries[name]['full_fit']['rmse_K']:.8f} K, holdout RMSE={summaries[name]['holdout']['rmse_K']:.8f} K", flush=True)

```

```
    selected = min(summaries, key=lambda name: summaries[name]["holdout"]["rmse_K"])
    model = candidates[selected]
    parameters, covariance = fitted[selected]
    selected_prediction = model["function"](t, *parameters)
    frame["T_fitted_K"] = selected_prediction
    # Window stability and late-record slope are identifiability diagnostics,
    # not additional holdouts used to retune the selected functional form.
    window_fits = []
    for end in (7200.0, 9000.0, 10800.0, 12600.0, 14400.0):
        mask = t <= end
        p, cov = fit_model(model, t[mask], T[mask], initial=parameters)
        window_fits.append({"fit_end_s": end, "sample_count": int(mask.sum()),
                            "parameters": param_mapping(model, p), "S_C": float(k_to_c(p[0])),
                            "S_standard_error_K": float(np.sqrt(cov[0, 0]))})
    late_slope = linregress(t[validation], T[validation])
    slope_quantile = student_t.ppf(0.975, int(validation.sum()) - 2)
    lower, upper = np.asarray(model["bounds"], dtype=float)
    span = upper - lower
    scaled_distance = np.minimum((parameters - lower) / span, (upper - parameters) / span)
    iid = bootstrap_asymptote(model, t, T, parameters, args.bootstrap, 20260911, 1)
    blocked = bootstrap_asymptote(model, t, T, parameters, args.bootstrap, 20260912, 5)
    pressure_note = "The drying comparison should use this fitted constant only after the measured 4 h record, with literature RH 0.20 held independently; humidity is not fitted by this script."
    report = {
        "source": {"path": str(attachment_path(1).relative_to(ROOT)),
                   "sha256": hashlib.sha256(attachment_path(1).read_bytes()).hexdigest(),
                   "temperature_column": "温度", "input_temperature_unit": "degC",
                   "fit_temperature_unit": "K", "fit_time_unit": "s",
                   "sample_count": len(t), "time_range_s": [float(t[0]), float(t[-1])],
                   "initial_temperature_K": T0},
        "validation_protocol": {"training_range_s": [0, 10800], "training_count": int(train.sum()),
                                "holdout_range_s": [float(t[validation][0]), 14400],
                                "holdout_count": int(validation.sum()),
                                "selection_rule": "minimum last-hour holdout RMSE across four predeclared monotone asymptotic candidates, followed by refit on
```

```
    all data",
257                              "warning": "The holdout selects the candidate and
    is not an untouched final test set; parameter intervals condition on the
    selected model."},
258         "candidates": summaries,
259         "selected_model": selected,
260         "selected": {"model": selected, "T_inf_K": float(parameters[0]),
261                      "T_inf_ci95_K": blocked["S_95pct_K"],
262                      "rmse_K": summaries[selected]["full_fit"]["rmse_K"],
263                      "holdout_rmse_K": summaries[selected]["holdout"]["rmse_K"],
264                      "interval_method": f"{args.bootstrap}-replicate circular
    moving-block residual bootstrap, block length 5 observations; conditional on
    selected model and fixed T0",
265                      "formula": model["formula"],
266                      "parameters": param_mapping(model, parameters)},
267         "selected_formula": model["formula"],
268         "selected_parameters": param_mapping(model, parameters),
269         "recommended_temperature_K": float(parameters[0]),
270         "recommended_temperature_C": float(k_to_c(parameters[0])),
271         "recommended_relative_humidity": 0.20,
272         "relative_humidity_source": "Ju et al. 2016, doi:10.1515/ijfe-2015-0340;
    this script does not estimate RH",
273         "full_fit_rmse_K": summaries[selected]["full_fit"]["rmse_K"],
274         "holdout_rmse_K": summaries[selected]["holdout"]["rmse_K"],
275         "covariance_uncertainty": covariance_report(model, parameters, covariance,
     len(t)),
276         "bootstrap_iid": iid,
277         "bootstrap_block5": blocked,
278         "recommended_temperature_95pct_K": blocked["S_95pct_K"],
279         "recommended_temperature_95pct_C": blocked["S_95pct_C"],
280         "identifiability": {"window_fits": window_fits,
281                             "scaled_distance_to_parameter_bounds": param_mapping(
    model, scaled_distance),
282                             "asymptote_minus_fitted_4h_K": float(parameters[0] -
    selected_prediction[-1]),
283                             "last_hour_observed_mean_K": float(np.mean(T[
    validation])),
284                             "last_hour_observed_mean_C": float(k_to_c(np.mean(T[
    validation]))),
285                             "last_hour_observed_slope_K_per_s": float(late_slope.
    slope),
286                             "last_hour_observed_slope_95pct_K_per_s": [float(
    late_slope.slope - slope_quantile * late_slope.stderr), float(late_slope.slope
    + slope_quantile * late_slope.stderr)],
287                             "description": "A late-record plateau, finite
```

```
    covariance and stability across truncated windows support a locally
    identifiable asymptote under the fitted model; none guarantees the unobserved
    long-term controller setting."},
288         "decision_note": pressure_note,
289         "limitations": [
290             "The record covers only 4 hours and does not prove indefinite
    asymptotic behavior.",
291             "Confidence intervals quantify conditional parameter uncertainty, not
    future observed temperature or material-model uncertainty.",
292             "Fixed T0 treats the first reading as known; thermometer calibration
    error and uncertainty in T0 are excluded.",
293             "The fitted approximately 50 C setpoint is an independent design
    comparison, not a claim of optimal medicinal quality.",
294             "Neither material moisture diffusivity nor shrinkage is reidentified
    from temperature data.",
295         ],
296         "software": {"python": platform.python_version(), "numpy": np.__version__,
     "scipy": scipy.__version__},
297         "reproduction": "D:/anaconda/Scripts/conda.exe run -p D:/anaconda/envs/26A
     --no-capture-output python scripts/fit_ambient_temperature.py --bootstrap 500"
    ,
298     }
299     dest = ROOT / "results" / "validation" / "ambient_fit.json"
300     csv = ROOT / "data" / "processed" / "ambient_fit.csv"
301     dest.parent.mkdir(parents=True, exist_ok=True)
302     csv.parent.mkdir(parents=True, exist_ok=True)
303     dest.write_text(json.dumps(report, ensure_ascii=False, indent=2, allow_nan=
    False) + "\n", encoding="utf-8")
304     pd.DataFrame(frame).to_csv(csv, index=False)
305     print(json.dumps({"selected_model": selected, "temperature_K": report["
    recommended_temperature_K"],
306                       "temperature_C": report["recommended_temperature_C"],
307                       "block_bootstrap_95pct_C": blocked["S_95pct_C"],
308                       "holdout_rmse_K": report["holdout_rmse_K"]}, ensure_ascii=
    False), flush=True)
309     return 0
310
311
312 if __name__ == "__main__":
313     raise SystemExit(main())
```

scripts/make_figs.py

```
1 """从当前正式计算与验证报告生成四问论文插图。"""
```

```python
from _bootstrap import setup
setup()

def main():
    from build_paper_assets import figures
    return figures()

if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/make_tables.py

```python
"""results/tables/*.csv  ->  paper/tab/*.tex（booktabs 三线表片段）

运行:  make tables

设计意图（AGENTS.md §2）:
    论文里的表 1~表 6 **不允许手抄数字**。数字永远只有一处来源 —— results/。
    论文用 \\input{tab/tabN} 引入本脚本生成的片段。

⚠ 不要手工编辑 paper/tab/*.tex，下次 make 就覆盖了。
   格式不对请提需求改本脚本（@impl 所有）。
"""

from __future__ import annotations

import sys
from pathlib import Path

import pandas as pd

from _bootstrap import setup

setup()

from common import paths  # noqa: E402

#: 表格列名 -> 论文表头的映射规则:
#: 生成的 csv 第一列是时间，其余列是「到药材中心的距离/cm」。
#: 表头文字与 docs/NOTATION.md 保持一致。


def _fmt(v: float) -> str:
    """保留四位小数（题目要求）。NaN 输出为空。"""
```

```
    if isinstance(v, str):
        return v
    try:
        f = float(v)
    except (TypeError, ValueError):
        return str(v)
    if f != f:  # NaN
        return ""
    return f"{f:.4f}"


def _num_columns(df: pd.DataFrame) -> list[str]:
    return [c for c in df.columns if c != df.columns[0]]


def to_booktabs(df: pd.DataFrame, caption: str, label: str) -> str:
    """生成一个 booktabs 三线表片段（无竖线，符合数模论文规范）。"""
    head, *body = df.columns.tolist()
    ncols = len(df.columns)

    lines = [
        "% 本文件由 scripts/make_tables.py 自动生成，请勿手工编辑",
        "\\begin{table}[htbp]",
        "  \\centering",
        f"  \\caption{{{caption}}}",
        f"  \\label{{{label}}}",
        "  \\begin{tabular}{" + "c" * ncols + "}",
        "    \\toprule",
        f"    {head} & " + " & ".join(str(c) for c in body) + " \\\\",
        "    \\midrule",
    ]
    for _, row in df.iterrows():
        cells = [_fmt(row[head])] + [_fmt(row[c]) for c in body]
        lines.append("    " + " & ".join(cells) + " \\\\")
    lines += [
        "    \\bottomrule",
        "  \\end{tabular}",
        "\\end{table}",
        "",
    ]
    return "\n".join(lines)


#: csv 文件名 -> (论文表标题, 标签)。表号由论文正文引用决定，这里给的是默认值。
TABLES: dict[str, tuple[str, str]] = {
```

```
    "q1_temp": ("30 分钟内药材的温度", "tab:q1-temperature"),
    "q1_moist": ("30 分钟内药材的水分浓度", "tab:q1-moisture"),
    "q2_temp": ("3 小时内药材的温度", "tab:q2-temperature"),
    "q2_moist": ("3 小时内药材的水分浓度", "tab:q2-moisture"),
    "q3_moist": ("药材烘干过程的水分浓度", "tab:q3-moisture"),
    "q4_moist": ("考虑收缩的药材烘干过程的水分浓度", "tab:q4-moisture"),
}


def main() -> int:
    from build_paper_assets import tables
    return tables()


if __name__ == "__main__":
    sys.exit(main())
```

scripts/package_delivery.py

```
"""Create and verify deterministic local review ZIPs using the standard library.

Run only after both scenarios, validation and the final manuscript are complete.
This script never modifies problem/, installs dependencies or publishes files.
"""

from __future__ import annotations

import argparse
import ast
import hashlib
import json
import os
from pathlib import Path
import shutil
import sys
import zipfile


ROOT = Path(__file__).resolve().parents[1]
DELIVERY = ROOT / "outputs" / "delivery"
FIXED_ZIP_TIME = (2026, 9, 11, 0, 0, 0)
LIMIT_BYTES = 20_000_000
MANIFEST_NAME = "PACKAGE_MANIFEST.json"
BUFFER_SIZE = 1024 * 1024
```

```
EXCLUDED_COMPONENTS = {
    ".git", ".github", ".agents", ".codex", "__pycache__", ".pytest_cache",
    ".ruff_cache", ".mypy_cache", ".ipynb_checkpoints", ".devtools",
    "node_modules", "build", "interim", "outputs",
}
LEGACY_FILES = {
    "scripts/make_accuracy_report.py",
    "scripts/validate_numerics.py",
    "scripts/validate_q4_accuracy.py",
    "scripts/verify_accuracy_reproduction.py",
    "results/validation/accuracy.json",
    "results/validation/accuracy-q4.json",
    "results/validation/accuracy-reproduction.json",
    "results/validation/accuracy-methods.md",
    "results/validation/numerics.json",
    "paper/figs/fig1-accuracy.pdf",
    "paper/figs/fig1-C-history.pdf",
    "paper/tab/q1_validation.tex",
    "paper/tab/q1_values.tex",
}
TEMPORARY_SUFFIXES = {
    ".pyc", ".pyo", ".aux", ".log", ".out", ".fls", ".fdb_latexmk",
    ".synctex", ".toc", ".nav", ".snm", ".vrb", ".tmp", ".bak",
    ".bbl", ".blg", ".bcf", ".dvi", ".xdv",
}
ROOT_FILES = (
    "AGENTS.md", "README.md", "Makefile", "environment.yml", "pyproject.toml",
    ".gitignore", ".gitattributes", "REPRODUCE.md", "run_all.py",
)


def sha256_file(path):
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(BUFFER_SIZE), b""):
            digest.update(chunk)
    return digest.hexdigest()


def load_json(path):
    return json.loads(path.read_text(encoding="utf-8"))


def relative(path):
    return path.relative_to(ROOT).as_posix()
```

```
71
72
73  def excluded(path):
74      rel = path.relative_to(ROOT)
75      if path.is_symlink() or any(part in EXCLUDED_COMPONENTS for part in rel.parts)
        :
76          return True
77      if path.name.startswith(("~$", ".#")) or path.name in {".DS_Store", "Thumbs.db
        ", "desktop.ini"}:
78          return True
79      if path.suffix.lower() in TEMPORARY_SUFFIXES or ".synctex" in path.name:
80          return True
81      if relative(path) in LEGACY_FILES:
82          return True
83      return False
84
85
86  def collect(directory):
87      parent = ROOT / directory
88      if not parent.is_dir():
89          raise FileNotFoundError(f"Required directory is missing: {directory}")
90      return [path for path in parent.rglob("*") if path.is_file() and not excluded(
        path)]
91
92
93  def unique_sorted(paths):
94      return sorted(set(paths), key=relative)
95
96
97  def audit_entrypoint_dependencies(planned_files):
98      """AST-check local imports and literal Python script calls, without runs.
99
100     External package imports are left to environment.yml. Dynamic qN solver
101     imports are covered by retaining all of src/. This audit follows the main
102     entrypoints and current tests, including historical helper modules still
103     used by current tests; filenames alone never determine historical status.
104     """
105     planned = {path.resolve() for path in planned_files}
106     seeds = [ROOT / "run_all.py", ROOT / "scripts/run_pipeline.py",
107              ROOT / "tests/test_accuracy.py", ROOT / "tests/test_numerics.py"]
108     visited = set()
109     edges = []
110     pending = list(seeds)
111
112     def resolve_module(module, caller, level=0):
```

```
        if not module:
            return []
        if level:
            base = caller.parent
            for _ in range(level - 1):
                base = base.parent
            bases = [base]
        else:
            bases = [caller.parent, ROOT, ROOT / "scripts", ROOT / "src"]
        module_path = Path(*module.split("."))
        for base in bases:
            options = [base / module_path.with_suffix(".py"), base / module_path / "__init__.py"]
            for option in options:
                if option.is_file() and option.resolve().is_relative_to(ROOT.resolve()):
                    return [option.resolve()]
        return []

    while pending:
        path = pending.pop().resolve()
        if path in visited:
            continue
        if not path.is_file() or path not in planned:
            raise RuntimeError("Entrypoint dependency is absent from ZIP plan: " + relative(path))
        visited.add(path)
        tree = ast.parse(path.read_text(encoding="utf-8-sig"), filename=str(path))
        dependencies = []
        for node in ast.walk(tree):
            if isinstance(node, ast.Import):
                for alias in node.names:
                    dependencies += resolve_module(alias.name, path)
            elif isinstance(node, ast.ImportFrom):
                dependencies += resolve_module(node.module, path, node.level)
                for alias in node.names:
                    if alias.name != "*":
                        child = f"{node.module}.{alias.name}" if node.module else alias.name
                        dependencies += resolve_module(child, path, node.level)
            elif isinstance(node, ast.Call):
                function_name = (node.func.id if isinstance(node.func, ast.Name)
                                 else node.func.attr if isinstance(node.func, ast.Attribute) else "")
                if function_name not in {"python", "run_path", "run", "Popen", "
```

```
    check_call", "check_output"}:
153                     continue
154                 script_literals = [value.value for argument in node.args for value
     in ast.walk(argument)
155                                    if isinstance(value, ast.Constant) and
    isinstance(value.value, str)
156                                    and value.value.endswith(".py")]
157                 for script in script_literals:
158                     raw = Path(script)
159                     if raw.is_absolute() or "\n" in script:
160                         continue
161                     for base in (ROOT, path.parent, ROOT / "scripts", ROOT / "
    tests"):
162                         candidate = base / raw
163                         if candidate.is_file():
164                             dependencies.append(candidate.resolve())
165                             break
166                     else:
167                         raise RuntimeError(f"Called script does not exist: {
    relative(path)} -> {script}")
168         for dependency in set(dependencies):
169             if dependency not in planned:
170                 raise RuntimeError(f"ZIP would break local dependency: {relative(
    path)} -> {relative(dependency)}")
171             edges.append({"caller": relative(path), "dependency": relative(
    dependency)})
172             pending.append(dependency)
173     return {"method": "AST local import and literal .py path dependency closure",
174             "entrypoints": [relative(path) for path in seeds],
175             "checked_python_files": sorted(relative(path) for path in visited),
176             "dependencies": sorted(edges, key=lambda row: (row["caller"], row["
    dependency"])),
177             "status": "all resolved local dependencies are present in ZIP source
    plan"}
178
179
180 def require_files(paths):
181     missing = [relative(path) for path in paths if not path.is_file()]
182     empty = [relative(path) for path in paths if path.is_file() and path.stat().
    st_size == 0]
183     if missing or empty:
184         raise RuntimeError(f"Delivery is unfinished. Missing: {missing}; empty: {
    empty}")
185
186
```

```
def verify_workbook_manifest(directory, q):
    file = directory / f"result{q}.xlsx"
    manifest_file = directory / f"q{q}_manifest.json"
    summary_file = directory / f"q{q}_summary.json"
    require_files([file, manifest_file, summary_file])
    manifest, summary = load_json(manifest_file), load_json(summary_file)
    if manifest.get("model_version") != "latent-v2" or manifest.get("latent_heat")
     is not True:
        raise RuntimeError(f"Historical or non-latent result is not deliverable: {
    relative(file)}")
    if summary.get("latent_heat") is not True:
        raise RuntimeError(f"Summary does not confirm latent heat: {relative(
    summary_file)}")
    if sha256_file(file) != manifest.get("submission_sha256"):
        raise RuntimeError(f"Workbook does not match its result manifest: {
    relative(file)}")
    input_hashes = {name.replace("\\", "/"): digest
                    for name, digest in manifest.get("input_hashes", {}).items()}
    for name, digest in input_hashes.items():
        if not (ROOT / name).is_file() or sha256_file(ROOT / name) != digest:
            raise RuntimeError(f"Original input does not match the run manifest: {
    name}")
    if q != 1 and (summary.get("event_time_s") is None
                   or summary.get("max_C_final_internal", 1.0) >= 0.15):
        raise RuntimeError(f"Strict drying threshold is not confirmed: {relative(
    summary_file)}")
    with zipfile.ZipFile(file) as workbook:
        bad = workbook.testzip()
        if bad is not None:
            raise RuntimeError(f"Damaged XLSX member: {relative(file)} :: {bad}")
    # Flag any final code drift instead of presenting unverifiable provenance.
    recorded = {name.replace("\\", "/"): digest
                for name, digest in manifest.get("source_hashes", {}).items()}
    if not recorded:
        raise RuntimeError(f"Missing source hashes: {relative(manifest_file)}")
    changed = [name for name, digest in recorded.items()
               if not (ROOT / name).is_file() or sha256_file(ROOT / name) !=
    digest]
    changed += [relative(path) + " (new source file)"
                for path in sorted((ROOT / "src").rglob("*.py"))
                if relative(path) not in recorded]
    return {"result": relative(file), "bytes": file.stat().st_size,
            "submission_sha256": manifest["submission_sha256"],
            "source_hashes_differ_from_current": changed}

```

```
225
226 def preflight():
227     required = [ROOT / name for name in ROOT_FILES]
228     required += [ROOT / name for name in (
229         "problem/A题.pdf", "problem/附件/附件1.xlsx", "problem/附件/附件2.xlsx",
230         "paper/main.tex", "paper/preamble.tex", "paper/refs.bib", "paper/build/
    main.pdf",
231         "paper/tab/all_values.tex", "paper/tab/temperature_scenarios.tex",
232         "results/validation/latent.json", "results/validation/ambient_fit.json",
233         "results/validation/fitted_numerics.json",
234         "data/processed/ambient_fit.csv", "scripts/README_delivery.md",
235         "results/scenarios/fitted_temperature/comparison.json",
236     )]
237     require_files(required)
238     validation = load_json(ROOT / "results/validation/latent.json")
239     if validation.get("model_version") != "latent-v2" or not validation.get("
    settings"):
240         raise RuntimeError("Latent-heat validation is missing or still being
    written")
241     if not all(validation.get(name) for name in ("grid", "time", "sensitivity", "
    ablation")):
242         raise RuntimeError("Latent validation does not contain every required
    analysis")
243     for q in ("q1", "q3", "q4"):
244         if not any(row.get("q") == q and row.get("n") == 2561 for row in
    validation["grid"]):
245             raise RuntimeError(f"Finest validation grid is missing for {q}")
246     fitted = ROOT / "results/scenarios/fitted_temperature"
247     comparison = load_json(fitted / "comparison.json")
248     if not all(q in comparison for q in ("q2", "q3", "q4")):
249         raise RuntimeError("Fitted-temperature scenario has not finished all three
     questions")
250     records = [verify_workbook_manifest(ROOT / "results", q) for q in (1, 2, 3, 4)
    ]
251     records += [verify_workbook_manifest(fitted, q) for q in (2, 3, 4)]
252     return records
253
254
255 def zip_info(name, readonly=False):
256     info = zipfile.ZipInfo(name, FIXED_ZIP_TIME)
257     info.compress_type = zipfile.ZIP_DEFLATED
258     info._compresslevel = 9  # ZipInfo streaming compression setting on Python
    3.12.
259     info.create_system = 3
260     info.external_attr = ((0o100444 if readonly else 0o100644) << 16)
```

```
    if readonly:
        info.external_attr |= 0x01  # DOS read-only flag; source filesystem is untouched.
    return info


def verify_archive(path):
    with zipfile.ZipFile(path) as archive:
        bad = archive.testzip()
        if bad is not None:
            raise RuntimeError(f"ZIP CRC check failed: {path.name} :: {bad}")
        manifest = json.loads(archive.read(MANIFEST_NAME))
        expected = {row["path"] for row in manifest["files"]} | {MANIFEST_NAME}
        names = archive.namelist()
        if len(names) != len(set(names)) or set(names) != expected:
            raise RuntimeError(f"ZIP membership mismatch: {path.name}")
        for row in manifest["files"]:
            digest = hashlib.sha256()
            count = 0
            with archive.open(row["path"]) as handle:
                for chunk in iter(lambda: handle.read(BUFFER_SIZE), b""):
                    digest.update(chunk)
                    count += len(chunk)
            if count != row["bytes"] or digest.hexdigest() != row["sha256"]:
                raise RuntimeError(f"ZIP member hash mismatch: {path.name} :: {row['path']}")
    size = path.stat().st_size
    return {"path": relative(path), "bytes": size, "MB_decimal": size / 1_000_000,
            "MiB_binary": size / 1024**2, "limit_20MB_bytes": LIMIT_BYTES,
            "exceeds_20MB": size > LIMIT_BYTES, "sha256": sha256_file(path),
            "member_count": len(names), "verified": "CRC plus every member SHA-256 and byte count"}


def write_archive(name, files, purpose, provenance_warnings):
    target = DELIVERY / name
    temporary = target.with_suffix(".zip.tmp")
    records = [{"path": relative(path), "bytes": path.stat().st_size, "sha256": sha256_file(path)}
               for path in files]
    manifest = {
        "format_version": 1, "purpose": purpose,
        "local_review_only": True, "files": records,
        "provenance_warnings": provenance_warnings,
        "excluded_historical_files": sorted(LEGACY_FILES),
```

```
        "exclusions": "Repository metadata, output caches, temporary files, compiled LaTeX files, and explicitly enumerated historical reports; main.pdf is separate. Current accuracy-output-check.json and imported validate_accuracy.py helpers are retained. Original problem PDFs and current figure PDFs remain for source reproducibility.",
        "reproducibility": "Sorted members, fixed ZIP timestamps and permissions, DEFLATE level 9. Identical input bytes and the same Python/zlib runtime produce identical archive bytes.",
    }
    with zipfile.ZipFile(temporary, "w", compression=zipfile.ZIP_DEFLATED,
                         compresslevel=9, allowZip64=True) as archive:
        for path in files:
            info = zip_info(relative(path), readonly=path.relative_to(ROOT).parts[0] == "problem")
            with path.open("rb") as source, archive.open(info, "w", force_zip64=True) as destination:
                shutil.copyfileobj(source, destination, BUFFER_SIZE)
        archive.writestr(zip_info(MANIFEST_NAME),
                         json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2).encode("utf-8"))
    # Verify before replacing a previously completed package. Any source file
    # changed while packaging is caught because hashes were computed first.
    verify_archive(temporary)
    os.replace(temporary, target)
    result = verify_archive(target)
    print(json.dumps(result, ensure_ascii=False), flush=True)
    return result


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--verify-only", action="store_true", help="Verify existing delivery ZIPs without rebuilding")
    parser.add_argument("--allow-source-drift", action="store_true",
                        help="Explicitly preserve and report source-hash differences; never suppress them")
    args = parser.parse_args()
    if args.verify_only:
        archives = sorted(DELIVERY.glob("*.zip"))
        if not archives:
            raise FileNotFoundError("No delivery ZIPs exist")
        for archive in archives:
            print(json.dumps(verify_archive(archive), ensure_ascii=False), flush=True)
        return 0
```

```
    provenance = preflight()
    drift = [row for row in provenance if row["source_hashes_differ_from_current"]]
    if drift and not args.allow_source_drift:
        raise RuntimeError("Result source hashes differ from current files. Resolve the discrepancy or explicitly retain it with --allow-source-drift; no archive was created. "
                           + json.dumps(drift, ensure_ascii=False))
    source_files = [ROOT / name for name in ROOT_FILES]
    for directory in ("problem", "src", "scripts", "tests", "docs", "data", "paper"):
        source_files += collect(directory)
    source_files = unique_sorted(source_files)
    dependency_audit = audit_entrypoint_dependencies(source_files)
    main_results = unique_sorted(path for path in collect("results")
                                 if "scenarios" not in path.relative_to(ROOT).parts)
    fitted_files = collect("results/scenarios/fitted_temperature")
    fitted_files += [ROOT / name for name in (
        "results/validation/ambient_fit.json", "results/validation/latent.json",
        "results/validation/fitted_numerics.json", "results/validation/README.md",
        "data/processed/ambient_fit.csv", "scripts/README_delivery.md",
        "scripts/fit_ambient_temperature.py", "scripts/validate_fitted_scenario.py",
        "docs/model/environment-fit.md",
    )]
    # Additional final verification reports (for example full-cell workbook
    # readback) are retained with the scenario too; only individually named
    # historical reports are excluded by collect().
    fitted_files += collect("results/validation")
    require_files(fitted_files)
    fitted_files = unique_sorted(fitted_files)
    DELIVERY.mkdir(parents=True, exist_ok=True)
    packages = [write_archive("cumcm2026A_support.zip", unique_sorted(source_files + main_results),
                              "Main 60 C / 20% RH scenario: source, frozen problem copies, validation, results and LaTeX sources", drift),
                write_archive("fitted_temperature_results.zip", fitted_files,
                              "Additional fitted-temperature / 20% RH scenario; use alongside the main support/source ZIP", drift)]
    if packages[0]["exceeds_20MB"]:
        print("NOTICE: Main support ZIP exceeds 20 MB; preserving it and additionally creating source/results ZIPs. No results are omitted.", flush=True)
        packages += [write_archive("cumcm2026A_source.zip", source_files,
```

```
                                    "Source-only companion; combine with main_results ZIP at the same extracted directory", drift),
                    write_archive("cumcm2026A_main_results.zip", main_results,
                                    "Complete main result files and current validation; may itself exceed 20 MB", drift)]
    pdf = ROOT / "paper/build/main.pdf"
    report = {"local_review_only": True, "submission_performed": False,
              "size_limit_interpretation": "Conservatively 20,000,000 bytes, with MiB also reported; over-limit files are preserved and disclosed, not silently truncated or split internally.",
              "packages": packages, "workbook_provenance": provenance,
              "source_dependency_audit": dependency_audit,
              "separate_final_pdf": {"path": relative(pdf), "bytes": pdf.stat().st_size,
                                     "sha256": sha256_file(pdf),
                                     "exceeds_20MB": pdf.stat().st_size > LIMIT_BYTES},
              "all_packages_within_20MB": all(not row["exceeds_20MB"] for row in packages)}
    (DELIVERY / "package_report.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    over = [row["path"] for row in packages if row["exceeds_20MB"]]
    if report["separate_final_pdf"]["exceeds_20MB"]:
        over.append(relative(pdf))
    if over:
        print("OVER 20 MB; local review artifacts remain complete: " + ", ".join(over), flush=True)
    print("Package report: " + relative(DELIVERY / "package_report.json"), flush=True)
    return 0


if __name__ == "__main__":
    try:
        raise SystemExit(main())
    except (OSError, RuntimeError, ValueError, zipfile.BadZipFile) as error:
        print(f"Delivery packaging failed: {error}", file=sys.stderr)
        raise SystemExit(1) from error
```

scripts/parse_attachments.py

```
"""解析 problem/附件 的附件 1、2 -> data/raw 与 data/processed。

运行:  make data
```

```
4
5   产出:
6       data/raw/attachment1.csv        附件 1 的SI单位副本
7       data/raw/attachment2.csv        附件 2 的SI单位副本
8       data/processed/ambient.csv      烘房条件 T_inf(t), C_inf(t)   [SI: K, kg/kg, s]
9       data/processed/radius.csv       药材半径 R(t)                 [SI: m, s]
10
11  本脚本只转换单位，不额外插值或外推。运行时边界由boundary.py负责，
12  四问潜热与两种后4h工况见当前docs/DECISIONS.md。
13  """
14
15  from __future__ import annotations
16
17  import sys
18
19  import pandas as pd
20
21  from _bootstrap import setup
22
23  setup()
24
25  from common import io_excel, paths, units  # noqa: E402
26
27
28  def parse_attachment1() -> pd.DataFrame:
29      """烘房环境条件。单位转为 SI（s, K, kg/kg）。"""
30      raw = io_excel.read_attachment(1)
31      out = pd.DataFrame(
32          {
33              "t_s": raw["时间"].to_numpy(dtype=float),
34              "T_inf_K": units.c_to_k(raw["温度"].to_numpy(dtype=float)),
35              "C_inf": raw["水分浓度"].to_numpy(dtype=float),
36          }
37      )
38      return out
39
40
41  def parse_attachment2() -> pd.DataFrame:
42      """药材半径随时间变化。单位转为 SI（s, m）。"""
43      raw = io_excel.read_attachment(2)
44      out = pd.DataFrame(
45          {
46              "t_s": raw["时间"].to_numpy(dtype=float),
47              "R_m": raw["半径"].to_numpy(dtype=float) * units.CM_TO_M,
48          }
```

```
    )
    return out


def coverage_report(a1: pd.DataFrame, a2: pd.DataFrame) -> str:
    """打印覆盖范围，并**显式警告缺口**——这是最容易在后期才发现的坑。"""
    t1_max = float(a1["t_s"].max())
    t2_max = float(a2["t_s"].max())
    lines = [
        "",
        "== 数据覆盖范围 ==",
        f"  附件1 烘房条件: 0 ~ {t1_max:.0f} s ({t1_max / 3600:.2f} h), "
        f"步长 {float(a1['t_s'].diff().dropna().iloc[0]):.0f} s, {len(a1)} 点",
        f"      T_inf: {a1['T_inf_K'].min() - 273.15:.3f} ~ {a1['T_inf_K'].max() - 273.15:.3f} °C",
        f"      C_inf: {a1['C_inf'].min():.5f} ~ {a1['C_inf'].max():.5f} kg/kg",
        f"  附件2 半径:     0 ~ {t2_max:.0f} s ({t2_max / 3600:.2f} h), "
        f"步长 {float(a2['t_s'].diff().dropna().iloc[0]):.0f} s, {len(a2)} 点",
        f"      R: {a2['R_m'].min() * 100:.4f} ~ {a2['R_m'].max() * 100:.4f} cm",
    ]

    if t2_max > t1_max * 4:
        lines += [
            "",
            "  附件1覆盖 %.1f h，附件2半径数据覆盖 %.1f h。" % (t1_max / 3600, t2_max / 3600),
            "  后续比较文献60°C和附件拟合温度两种恒温方案，RH均20%。",
            "  这是用户授权的工况设计，不是新增实测数据；见当前DECISIONS。",
        ]

    c0 = 2.55
    if a1["C_inf"].max() < c0 / 10:
        lines += [
            "",
            "  ⚠ 量级警告：烘房水分浓度最大 %.5f，而药材初始水分浓度 %.2f，相差 %.0f 倍。"
            % (a1["C_inf"].max(), c0, c0 / a1["C_inf"].max()),
            "      当前模型将空气列解释为kg水/kg干空气，药材为kg水/kg干物质。",
            "      两者不直接相减，气膜/供水闭合见docs/model/environment.md。",
        ]

    return "\n".join(lines)


def main() -> int:
```

```
    paths.ensure_output_dirs()

    a1 = parse_attachment1()
    a2 = parse_attachment2()

    p1 = paths.DATA_RAW / "attachment1.csv"
    p2 = paths.DATA_RAW / "attachment2.csv"
    a1.to_csv(p1, index=False)
    a2.to_csv(p2, index=False)

    pa = paths.DATA_PROCESSED / "ambient.csv"
    pr = paths.DATA_PROCESSED / "radius.csv"
    a1.to_csv(pa, index=False)
    a2.to_csv(pr, index=False)

    for p in (p1, p2, pa, pr):
        print(f"  写出 {paths.rel(p)}  ({p.stat().st_size} B)")
    print(coverage_report(a1, a2))
    print()
    print("  本脚本仅转换单位；线性插值和后续两种工况由Boundary负责。")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

scripts/probe_templates.py

```
"""打印 problem/附件/附件3/result1~4.xlsx 的真实结构。

运行:  make probe

**开工第一件事就是跑这个**（AGENTS.md 铁律 3：不许猜交付物格式）。
把输出贴进 data/README.md §4 与 docs/INTERFACES.md §6。
"""

from __future__ import annotations

import sys

from _bootstrap import setup

setup()

```

```python
from common import io_excel, paths  # noqa: E402


def main() -> int:
    print("=" * 72)
    print("交付模板结构探查 —— 以实际文件为准，不要按题目文字描述猜测")
    print("=" * 72)
    for n in (1, 2, 3, 4):
        try:
            print(io_excel.probe_template(f"q{n}"))
        except FileNotFoundError as e:
            print(f"  !! {e}")
        print("-" * 72)

    print()
    print("=" * 72)
    print("附件结构")
    print("=" * 72)
    for n in (1, 2):
        df = io_excel.read_attachment(n)
        t = df["时间"].to_numpy(dtype=float)
        step = float(t[1] - t[0]) if t.size > 1 else float("nan")
        print(
            f"附件{n}: {paths.rel(paths.attachment_path(n))}\n"
            f"  列名={list(df.columns)}  行数={len(df)}\n"
            f"  时间 {t.min():.0f}~{t.max():.0f} s  步长={step:.0f} s"
        )
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

scripts/run_all.py

```python
"""Windows/POSIX共用四问复现入口；不含论文第二至四问生成。"""
from _runner import run


if __name__ == "__main__":
    for question in ("q1", "q3", "q4", "q2"):
        code = run(question)
        if code:
            raise SystemExit(code)
```

scripts/run_fitted_scenario.py

```
"""附件温度拟合所得恒温值+文献20%RH，独立输出Q2—4。"""
import json
from _runner import run
from common import paths
from common.boundary import Boundary


def main():
    fit=json.loads((paths.RESULTS_DIR/"validation/ambient_fit.json").read_text(encoding="utf-8"))
    T=fit["selected"]["T_inf_K"]
    paths.RESULTS_DIR=paths.RESULTS_DIR/"scenarios/fitted_temperature"
    paths.RESULTS_TABLES=paths.RESULTS_DIR/"tables"
    paths.OUTPUTS_CACHE=paths.OUTPUTS_CACHE/"fitted_temperature"
    paths.ensure_output_dirs()
    comparison={"temperature_K":T,"relative_humidity":.20,"source":"results/validation/ambient_fit.json"}
    for q in ("q3","q4","q2"):
        code=run(q,boundary=Boundary.for_problem(q,T-273.15,.20))
        if code:
            return code
        comparison[q]=json.loads((paths.RESULTS_DIR/f"{q}_summary.json").read_text(encoding="utf-8"))
        (paths.RESULTS_DIR/"comparison.json").write_text(json.dumps(comparison,indent=2),encoding="utf-8")
    return 0


if __name__=="__main__":
    raise SystemExit(main())
```

scripts/run_pipeline.py

```
"""完整本地复现：计算、拟合、验证、表图、XeLaTeX、ZIP；不联网提交。"""
import argparse
import subprocess
import sys
from pathlib import Path

ROOT=Path(__file__).resolve().parents[1]


```

```
def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--skip-compute',action='store_true',help='已完成两种工况计算时，仅更新验证后续交付')
    parser.add_argument('--skip-validation',action='store_true',help='已有对应当前模型的验证报告时跳过昂贵加密重算')
    args=parser.parse_args()
    def python(script,*options):
        subprocess.run([sys.executable,str(ROOT/script),*options],cwd=ROOT,check=True)
    python('scripts/probe_templates.py')
    python('scripts/fit_ambient_temperature.py')
    if not args.skip_compute:
        python('run_all.py')
        python('scripts/run_fitted_scenario.py')
    if not args.skip_validation:
        python('scripts/validate_latent.py')
        python('scripts/validate_fitted_scenario.py')
    python('scripts/audit_q4_boundary_layer.py')
    python('scripts/selfcheck.py')
    for pattern in ('test_numerics.py','test_accuracy.py'):
        subprocess.run([sys.executable,'-m','unittest','discover','-s','tests','-p',pattern,'-v'],cwd=ROOT,check=True)
    python('scripts/verify_accuracy_outputs.py')
    python('scripts/verify_accuracy_outputs.py','--fitted')
    python('scripts/build_paper_assets.py')
    python('paper/build_sources.py')
    subprocess.run(['latexmk','-xelatex','-interaction=nonstopmode','-halt-on-error','-outdir=build','main.tex'],cwd=ROOT/'paper',check=True)
    python('scripts/package_delivery.py')
    return 0


if __name__=='__main__':
    raise SystemExit(main())
```

scripts/run_q1.py

```
"""问题 1 的入口。运行： make q1

逻辑全在 scripts/_runner.py，这里只做分发。
"""

from __future__ import annotations
```

```

import sys

from _runner import main

if __name__ == "__main__":
    sys.exit(main("q1"))
```

scripts/run_q123_accuracy.py

```
"""重算第1--3问；只读依赖与模板预检，不安装依赖。"""

import importlib
import platform

from _runner import run
from common.io_excel import verify_layout_against_template


def main(questions=("q1", "q3", "q2")):
    print("Python:", platform.python_version(), flush=True)
    for name in ("numpy", "scipy", "pandas", "openpyxl", "matplotlib"):
        module = importlib.import_module(name)
        print(name, module.__version__, flush=True)
    for question in questions:
        problems = verify_layout_against_template(question)
        if problems:
            raise RuntimeError(str(problems))
    for question in questions:
        code = run(question)
        if code:
            return code
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/run_q2.py

```
"""问题 2 的入口。运行： make q2

逻辑全在 scripts/_runner.py，这里只做分发。
```

```
"""

from __future__ import annotations

import sys

from _runner import main

if __name__ == "__main__":
    sys.exit(main("q2"))
```

scripts/run_q3.py

```
"""问题 3 的入口。运行: make q3

逻辑全在 scripts/_runner.py，这里只做分发。
"""

from __future__ import annotations

import sys

from _runner import main

if __name__ == "__main__":
    sys.exit(main("q3"))
```

scripts/run_q4.py

```
"""问题 4 的入口。运行: make q4

逻辑全在 scripts/_runner.py，这里只做分发。
"""

from __future__ import annotations

import sys

from _runner import main

if __name__ == "__main__":
    sys.exit(main("q4"))
```

scripts/selfcheck.py

```
"""框架自检 —— **不依赖 pytest**，只要有 numpy/pandas 就能跑。

运行：  make selfcheck

用途：
    本仓库的正式测试在 tests/（pytest）。但竞赛现场的环境常常没装 pytest，
    而「框架是通的」这件事必须随时可验证。因此把最关键的几条不变式抽到这里，
    用纯 assert 表达。

    `make check` 在检测到没有 pytest 时会自动回退到本脚本。

覆盖：
    路径定位 / 单位换算与防呆 / 题目常量与附录不混用 / 附件结构 /
    交付模板结构（实测布局）/ 交付格式校验会硬失败 / Solution 语义
"""

from __future__ import annotations

import sys
import traceback

import numpy as np
import pandas as pd

from _bootstrap import setup

setup()

from common import correlations, io_excel, paths, problem_spec, units  # noqa:
    E402
from common.solution import Solution  # noqa: E402

_FAILED: list[tuple[str, str]] = []
_PASSED = 0


def check(name: str):
    """把函数注册为一条自检项。"""

    def deco(fn):
        try:
            fn()
        except Exception as e:  # noqa: BLE001
```

```
            _FAILED.append((name, f"{type(e).__name__}: {e}"))
            print(f"  ✗ {name}")
            print(f"      {type(e).__name__}: {e}")
            if "--verbose" in sys.argv:
                traceback.print_exc()
        else:
            globals()["_PASSED"] += 1
            print(f"  ✓ {name}")
        return fn

    return deco


print("=" * 72)
print("框架自检")
print("=" * 72)

# --- 路径 ------------------------------------------------------------------


@check("仓库根定位正确，题目原件存在")
def _():
    assert (paths.ROOT / "AGENTS.md").exists(), "AGENTS.md 不在仓库根"
    assert (paths.ROOT / "problem" / "A题.pdf").exists(), "题目原件缺失"
    assert paths.template_path("q1").name == "result1.xlsx"
    assert paths.result_path("4").name == "result4.xlsx"


# --- 单位 ------------------------------------------------------------------


@check("单位换算往返一致")
def _():
    assert abs(units.m_to_cm(units.cm_to_m(2.0)) - 2.0) < 1e-12
    assert abs(units.k_to_c(units.c_to_k(28.0)) - 28.0) < 1e-12
    assert abs(units.c_to_k(28.0) - 301.15) < 1e-12
    assert abs(units.h_to_s(1.0) - 3600.0) < 1e-12


@check("摄氏度冒充开尔文会被防呆断言拦下")
def _():
    try:
        units.assert_kelvin(np.array([28.0, 50.0]))
    except ValueError as e:
        assert "摄氏度" in str(e), f"错误信息应提示摄氏度，实际：{e}"
```

```
    else:
        raise AssertionError("未拦下摄氏度，防呆失效（这是本项目最危险的错误）")
    units.assert_kelvin(np.array([301.15, 323.15]))  # 正常开尔文不应抛


@check("干基/湿基换算正确")
def _():
    assert abs(correlations.wet_basis(0.0) - 0.0) < 1e-12
    assert abs(correlations.wet_basis(1.0) - 0.5) < 1e-12
    assert abs(correlations.wet_basis(2.55) - 2.55 / 3.55) < 1e-12


# --- 题目常量 ---------------------------------------------------------------


@check("题目给定量与 A题.pdf 一致")
def _():
    assert problem_spec.LENGTH_M == 0.25  # 长 25 cm
    assert problem_spec.RADIUS0_M == 0.02  # 半径 2 cm
    assert problem_spec.INITIAL_T_C == 28.0
    assert problem_spec.INITIAL_C_DRY == 2.55
    assert problem_spec.TARGET_C_DRY == 0.15
    assert problem_spec.Q1_T_END_S == 1800.0
    assert problem_spec.Geometry().aspect_ratio == 6.25  # L/(2R0) = 25/4


@check("附录 2/3/4 未混用；仅问题 4 有收缩")
def _():
    assert problem_spec.Q1.properties.appendix == 2
    assert problem_spec.Q2.properties.appendix == 3
    assert problem_spec.Q3.properties.appendix == 3
    assert problem_spec.Q4.properties.appendix == 4
    assert problem_spec.Q1.uses_shrinkage is False
    assert problem_spec.Q4.uses_shrinkage is True


@check("问题 1 交付网格 = 1801 时间点 x 21 空间点")
def _():
    assert problem_spec.Q1.grid.shape == (1801, 21), problem_spec.Q1.grid.shape
    assert problem_spec.Q1.grid.r[0] == 0.0, "交付网格必须含轴心 r=0"


# --- 附件 -------------------------------------------------------------------
```

```
@check("附件 1 结构：241 点 / 60 s 步长 / 覆盖 4 h")
def _():
    df = io_excel.read_attachment(1)
    assert list(df.columns) == ["时间", "温度", "水分浓度"]
    assert len(df) == 241
    t = df["时间"].to_numpy(dtype=float)
    assert t[-1] == 14400.0 and np.allclose(np.diff(t), 60.0)


@check("附件 2 结构：145 点 / 1800 s 步长 / 覆盖 72 h / 半径单调不增")
def _():
    df = io_excel.read_attachment(2)
    assert list(df.columns) == ["时间", "半径"]
    assert len(df) == 145
    r = df["半径"].to_numpy(dtype=float)
    t = df["时间"].to_numpy(dtype=float)
    assert t[-1] == 259200.0
    assert r[0] == 2.0 and np.all(np.diff(r) <= 0)
    assert abs(r[-1] - r[-2]) < 1e-12, "D-010：末段应有半径平台"


@check("数据缺口 D-009 / D-011 确实存在（不是幻觉）")
def _():
    a1 = io_excel.read_attachment(1)
    assert a1["时间"].max() == problem_spec.AMBIENT_COVERAGE_S < problem_spec.
    RADIUS_COVERAGE_S
    assert a1["水分浓度"].max() < problem_spec.INITIAL_C_DRY / 10, "D-011 量级差异应
     > 10 倍"


# --- 交付模板（实测布局）-----------------------------------------------------


@check("实测布局与 problem/附件/附件3 的真实模板一致")
def _():
    bad: list[str] = []
    for p in ("q1", "q2", "q3", "q4"):
        bad += io_excel.verify_layout_against_template(p)
    assert not bad, "布局与模板不一致：\n      - " + "\n      - ".join(bad)


@check("工作表名按模板：result1/2 是 温度+水分浓度，result3/4 是 Sheet1")
def _():
    assert set(io_excel.layout("q1").sheet_names) == {"温度", "水分浓度"}
    assert set(io_excel.layout("q2").sheet_names) == {"温度", "水分浓度"}
```

```
    assert set(io_excel.layout("q3").sheet_names) == {"Sheet1"}
    assert set(io_excel.layout("q4").sheet_names) == {"Sheet1"}


@check("距离列是完整 21 列（模板里的 '…' 是省略示意）")
def _():
    h = io_excel.layout("q1").distance_headers()
    assert len(h) == 21 and h[0] == "0" and h[1] == "0.1" and h[-1] == "2"
    h4 = io_excel.layout("q4").distance_headers()
    assert h4[-1] == io_excel.SURFACE_HEADER and h4[-2] == "1.9"


@check("交付时间单位是秒（不是题目表 3~表 6 表头写的 h）")
def _():
    assert io_excel.layout("q1").dt_s == 1.0
    assert io_excel.layout("q3").dt_s == 60.0


# --- 交付生成与格式校验 ------------------------------------------------------


def _dummy(n_t: int = 1801, radius: float = problem_spec.RADIUS0_M) -> Solution:
    return Solution(
        t=np.linspace(0.0, problem_spec.Q1_T_END_S, n_t),
        r=np.linspace(0.0, radius, 21),
        T=np.full((n_t, 21), 300.0),
        C=np.linspace(2.55, 0.10, n_t)[:, None] * np.ones((1, 21)),
    )


@check("export 生成的列结构与布局一致（1 列时间 + 21 列距离）")
def _():
    from common import export

    sheets = export.result_sheets(_dummy(), problem_spec.Q1)
    for name, df in sheets.items():
        assert list(df.columns) == io_excel.layout("q1").expected_columns(), name
    assert sheets["水分浓度"].shape == (1801, 22), sheets["水分浓度"].shape


@check("问题 4 末列是「药材表面」")
def _():
    from common import export

    sheets = export.result_sheets(_dummy(n_t=101), problem_spec.Q4)
```

```
    df = sheets["Sheet1"]
    assert list(df.columns) == io_excel.layout("q4").expected_columns()
    assert io_excel.SURFACE_HEADER in df.columns


@check("格式校验是硬失败：工作表名错 / 列数少 / 含 NaN 都必须抛异常")
def _():
    import tempfile
    from pathlib import Path

    from common import export

    saved = paths.RESULTS_DIR
    try:
        with tempfile.TemporaryDirectory() as d:
            paths.RESULTS_DIR = Path(d)

            try:
                io_excel.write_result("q1", {"错的表名": pd.DataFrame({io_excel.
    TIME_HEADER: [1.0]})})
            except io_excel.ResultFormatError as e:
                assert "工作表名不匹配" in str(e)
            else:
                raise AssertionError("工作表名错误未被拦下")

            short = {io_excel.TIME_HEADER: [1.0, 2.0]}
            for h in ["0", "0.1", "0.2", "2"]:
                short[h] = [0.0, 0.0]
            try:
                io_excel.write_result(
                    "q1", {n: pd.DataFrame(short) for n in io_excel.layout("q1").
    sheet_names}
                )
            except io_excel.ResultFormatError as e:
                assert "列结构不匹配" in str(e)
            else:
                raise AssertionError("列数不足未被拦下")

            sheets = export.result_sheets(_dummy(), problem_spec.Q1)
            sheets["水分浓度"].iloc[3, 1] = float("nan")
            try:
                io_excel.write_result("q1", sheets)
            except io_excel.ResultFormatError as e:
                assert "NaN" in str(e)
            else:
```

```
                raise AssertionError("NaN 未被拦下")
    finally:
        paths.RESULTS_DIR = saved


@check("合规工作表能正常写出，且能被读回")
def _():
    import tempfile
    from pathlib import Path

    from common import export

    saved = paths.RESULTS_DIR
    try:
        with tempfile.TemporaryDirectory() as d:
            paths.RESULTS_DIR = Path(d)
            dest = io_excel.write_result("q1", export.result_sheets(_dummy(),
    problem_spec.Q1))
            assert dest.exists()
            back = pd.read_excel(dest, sheet_name=None)
            assert set(back) == {"温度", "水分浓度"}
            assert back["水分浓度"].shape == (1801, 22)
    finally:
        paths.RESULTS_DIR = saved


# --- Solution 语义 ------------------------------------------------------------


@check("终点判据取 max_r（题目说「各处」），且达不到时抛错")
def _():
    sol = _dummy(n_t=19)
    assert sol.shape == (19, 21)
    assert sol.surface_index == 20
    assert abs(sol.first_time_below(0.15) - sol.t[-1]) < 1e-9
    try:
        sol.first_time_below(0.05)
    except ValueError as e:
        assert "从未低于" in str(e)
    else:
        raise AssertionError("达不到终点时应抛错，不许静默返回（铁律 10）")


@check("形状不一致的 Solution 会被拒绝")
def _():
```

```
    try:
        Solution(
            t=np.zeros(3), r=np.zeros(4), T=np.zeros((3, 4)), C=np.zeros((3, 5))
        )
    except ValueError as e:
        assert "形状不一致" in str(e)
    else:
        raise AssertionError("T/C 形状不一致未被拒绝")


# --- 骨架阶段的预期状态 ------------------------------------------------------


@check("三套物性公式按附录实现，求解器入口可调用")
def _():
    for cls, expected in zip((correlations.Appendix2, correlations.Appendix3,
                              correlations.Appendix4), (820,778,850)):
        assert float(cls().rho(1.0,300.0)) == expected
    for n in (1, 2, 3, 4):
        mod = __import__(f"q{n}.solver", fromlist=["solve"])
        assert callable(mod.solve)


@check("汽化潜热符合FA056附录3式(3-1)，温度输入K")
def _():
    np.testing.assert_allclose(correlations.latent_heat_vaporization(
        units.c_to_k(np.array([0., 20., 60.]))), [2501000., 2453780., 2359340.])


# --- 汇总 ------------------------------------------------------------------

print()
print("=" * 72)
if _FAILED:
    print(f"✗ {len(_FAILED)} 项失败，{_PASSED} 项通过")
    for name, err in _FAILED:
        print(f"    - {name}: {err}")
    print("=" * 72)
    sys.exit(1)

print(f"✓ 全部 {_PASSED} 项自检通过")
print("=" * 72)
```

scripts/validate_accuracy.py

```
"""Q1--3精度审计；保持物理模型与附件线性插值不变。"""

import json
import platform

import numpy as np
import scipy
from scipy.optimize import brentq
from scipy.special import j0, j1, jn_zeros

from _bootstrap import setup

setup()

from common.boundary import Boundary
from common.correlations import h_conv
from common.paths import ROOT
from common.problem_spec import Q1, Q3
from common.radial import solve_radial


def bessel_heat(times, r, modes=512):
    """Q1实际分段线性环境的Bessel模态卷积；每段时间积分解析计算。

    对T-T_inf展开，模态满足q'=-lambda*q-T_inf'。
    Robin本征值满足z*J1(z)=Bi*J0(z)。无有限体积或ODE误差。
    """
    b = Boundary.from_attachments()
    p = Q1.properties
    R = Q1.geometry.radius0
    k = float(p.k(Q1.initial.C, Q1.initial.T))
    alpha = k / float(p.rho(Q1.initial.C, Q1.initial.T)
                      * p.cp(Q1.initial.C, Q1.initial.T))
    Bi = h_conv("q1") * R / k
    ends = np.r_[0.0, jn_zeros(0, modes)]
    z = np.array([brentq(lambda x: x*j1(x)-Bi*j0(x),
                         ends[i]+1e-10, ends[i+1]) for i in range(modes)])
    lam = alpha*z*z/(R*R)
    coefficients = 2*j1(z)/(z*(j0(z)**2+j1(z)**2))
    basis = coefficients[:, None]*j0(z[:, None]*np.asarray(r)[None, :]/R)
    states = np.zeros((len(b.ambient_t), modes))
    states[0] = Q1.initial.T-b.T_amb[0]
    slopes = np.diff(b.T_amb)/np.diff(b.ambient_t)
```

```
    for i, dt in enumerate(np.diff(b.ambient_t)):
        states[i+1] = states[i]*np.exp(-lam*dt)+slopes[i]*np.expm1(-lam*dt)/lam
    out = []
    for t in times:
        i = min(np.searchsorted(b.ambient_t, t, side="right")-1, len(slopes)-1)
        dt = t-b.ambient_t[i]
        q = states[i]*np.exp(-lam*dt)+slopes[i]*np.expm1(-lam*dt)/lam
        out.append(float(b.ambient(t)[0])+q@basis)
    return np.asarray(out)


def differences(a, b):
    return {"delta_T_K": float(np.max(np.abs(a.T-b.T))),
            "delta_C": float(np.max(np.abs(a.C-b.C))),
            "delta_event_s": (a.meta["event_time_s"]-b.meta["event_time_s"]
                              if a.meta["event_time_s"] is not None else None)}


def main():
    folder = ROOT / "outputs" / "accuracy-v2"
    folder.mkdir(parents=True, exist_ok=True)
    dest = ROOT / "results" / "validation" / "accuracy.json"
    report = {"python": platform.python_version(), "numpy": np.__version__,
              "scipy": scipy.__version__, "grid": [], "cross_solver": [],
              "sampling": {"q1": "every second 0--1800 s; all 21 output radii",
                           "q3": "every 60 s 0--3 h; every hour 4--54 h; all 21 radii; continuous event"}}
    settings = dict(rtol=2e-11, atol=2e-12, max_step=60, align_boundary_knots=True)

    def save():
        dest.write_text(json.dumps(report, indent=2, ensure_ascii=False), encoding="utf-8")

    for spec in (Q1, Q3):
        times = (np.arange(1801, dtype=float) if spec.name == "q1" else
                 np.r_[np.arange(0, 10801, 60), np.arange(4, 55)*3600.0])
        runs = {}
        for n in (641, 1281, 2561, 5121):
            cache = folder / f"{spec.name}_{n}.npz"
            if cache.exists():
                from common.solution import Solution
                with np.load(cache) as f:
                    s = Solution(f["t"], f["r"], f["T"], f["C"], json.loads(str(f["meta"])))
```

```
            else:
                print(f"START {spec.name} n={n}", flush=True)
                s = solve_radial(spec, n_internal=n, sample_times=times, **
    settings)
                np.savez_compressed(cache, t=s.t, r=s.r, T=s.T, C=s.C,
                                    meta=json.dumps(s.meta))
            row = {"q": spec.name, "n": n, "seconds": s.meta["wall_time_s"],
                   "mass_residual": s.meta["mass_balance_relative"],
                   "event_s": s.meta["event_time_s"]}
            if runs:
                previous = runs[(n-1)//2+1]
                row.update(differences(s, previous))
                row["richardson_fine_T_K"] = row["delta_T_K"]/3
                row["richardson_fine_C"] = row["delta_C"]/3
                if row["delta_event_s"] is not None:
                    row["richardson_fine_event_s"] = abs(row["delta_event_s"])/3
            runs[n] = s
            report["grid"].append(row)
            print(json.dumps(row), flush=True)
            save()
        if spec.name == "q1":
            exact = bessel_heat(times, runs[2561].r)
            exact_fine = bessel_heat(times, runs[2561].r, modes=1024)
            report["bessel"] = {
                "modes_delta_T_K": float(np.max(np.abs(exact-exact_fine))),
                "max_error_by_grid_K": {str(n): float(np.max(np.abs(s.T-exact_fine
    )))
                                        for n, s in runs.items()}}
            save()
        # 独立五阶隐式RK在同一网格隔离时间推进误差，不冒充空间独立验证。
        print(f"START Radau cross-check {spec.name}", flush=True)
        bdf = solve_radial(spec, n_internal=161, sample_times=times, **settings)
        radau = solve_radial(spec, n_internal=161, sample_times=times, method="
    Radau", **settings)
        report["cross_solver"].append({"q": spec.name, "n": 161,
                                       **differences(radau, bdf)})
        print(json.dumps(report["cross_solver"][-1]), flush=True)
        save()
    report["settings"] = settings
    report["note"] = "Richardson estimates assume the observed second-order
    asymptotic regime; not physical uncertainty bounds."
    save()


if __name__ == "__main__":
```

```
    main()
```

scripts/validate_fitted_scenario.py

```
"""拟合温度情景：641与1281网格在全部60s输出点和连续终点比较。"""
import json
import numpy as np
from _bootstrap import setup
setup()
from common import export, paths
from common.boundary import Boundary
from common.problem_spec import Q3,Q4
from common.radial import solve_radial
from common.solution import Solution


def main():
    fit=json.loads((paths.RESULTS_DIR/"validation/ambient_fit.json").read_text(encoding="utf-8"))
    temp=fit["selected"]["T_inf_K"]
    report={"temperature_K":temp,"rows":[]}
    for spec in (Q3,Q4):
        with np.load(paths.OUTPUTS_CACHE/f"fitted_temperature/{spec.name}.npz") as f:
            ref=Solution(f['t'],f['r'],f['T'],f['C'],json.loads(str(f['meta'])))
        # 两个网格严格达标时刻可能相差一个输出格，比较共同时间范围。
        times=ref.t[:-1]
        s=solve_radial(spec,n_internal=641,rtol=2e-10,atol=2e-11,max_step=60.,align_boundary_knots=True,
                       boundary=Boundary.for_problem(spec.name,temp-273.15,.2),sample_times=times)
        if spec.name=="q4":
            columns=export.result_sheets(s,spec)['Sheet1'].iloc[:,1:].to_numpy()
            reference=export.result_sheets(ref,spec)['Sheet1'].iloc[:-1,1:].to_numpy()
        else:
            columns,reference=s.C,ref.C[:-1]
        row={"q":spec.name,"coarse_n":641,"fine_n":1281,
             "delta_output_C":float(np.nanmax(abs(columns-reference))),
             "delta_event_s":abs(s.meta['event_time_s']-ref.meta['event_time_s']),
             "mass_balance_relative":s.meta['mass_balance_relative'],
             "sample_count":len(times)}
        report['rows'].append(row)
        print(row,flush=True)
```

```
        (paths.RESULTS_DIR/"validation/fitted_numerics.json").write_text(json.dumps(report,indent=2),encoding="utf-8")
    return 0


if __name__=="__main__":
    raise SystemExit(main())
```

scripts/validate_latent.py

```
"""潜热互补边界模型独立数值验证；缓存含代码指纹，防复用旧模型结果。"""
import hashlib
import json
import numpy as np
from _bootstrap import setup
setup()
from common import export, paths
from common.boundary import Boundary
from common.problem_spec import Q1, Q3, Q4
from common.radial import solve_radial


def main():
    signature = hashlib.sha256(b"".join(p.read_bytes() for p in sorted((paths.ROOT/"src").rglob("*.py")))).hexdigest()
    report = {"model_version":"latent-v2", "source_fingerprint":signature,
              "grid":[], "time":[], "sensitivity":[], "ablation":[]}
    dest = paths.RESULTS_DIR/"validation/latent.json"
    folder = paths.OUTPUTS_CACHE/"latent_validation"
    folder.mkdir(parents=True, exist_ok=True)
    settings = dict(rtol=2e-10, atol=2e-11, max_step=60., align_boundary_knots=True)
    runs = {}

    def save():
        dest.write_text(json.dumps(report,indent=2,ensure_ascii=False),encoding="utf-8")

    def solve_cached(spec, label, **options):
        key = hashlib.sha256((signature+spec.name+label+repr(options)).encode()).hexdigest()[:16]
        file = folder/f"{spec.name}_{label}_{key}.npz"
        if file.exists():
            from common.solution import Solution
```

```
            with np.load(file) as f:
                return Solution(f["t"],f["r"],f["T"],f["C"],json.loads(str(f["meta"])))
        print("START", spec.name,label,flush=True)
        s = solve_radial(spec,**options)
        np.savez_compressed(file,t=s.t,r=s.r,T=s.T,C=s.C,meta=json.dumps(s.meta))
        return s

    def fields(s,spec):
        if spec.name != "q4":
            return s.T,s.C
        return tuple(export.result_sheets(s,spec)["Sheet1"].iloc[:,1:].to_numpy() for _ in (0,))

    def diff(s,ref,spec):
        if spec.name == "q4":
            # 同时验证全材料域及固定物理交付位置，避免遗漏薄表层。
            Ts=np.array([np.interp(ref.r,s.r,row) for row in s.T])
            Cs=np.array([np.interp(ref.r,s.r,row) for row in s.C])
        else:
            Ts,Cs=s.T,s.C
        return {"delta_T_K":float(np.max(np.abs(Ts-ref.T))),
                "delta_C":float(np.max(np.abs(Cs-ref.C))),
                "delta_event_s":abs(s.meta["event_time_s"]-ref.meta["event_time_s"]) if s.meta["event_time_s"] else None}

    for spec in (Q1,Q3,Q4):
        times = np.arange(0.,1801.,10.) if spec.name=="q1" else np.unique(np.r_[np.arange(0.,10801.,60.),np.arange(4.,40.)*3600.])
        previous=None
        for n in (321,641,1281,2561):
            s=solve_cached(spec,f"grid{n}",n_internal=n,sample_times=times,**settings)
            runs[spec.name,n]=s
            row={"q":spec.name,"n":n,"event_s":s.meta["event_time_s"],
                 "wall_time_s":s.meta["wall_time_s"],"mass_residual":s.meta["mass_balance_relative"],
                 "q1_energy_residual":s.meta["q1_energy_balance_relative"]}
            if previous is not None:
                row.update(diff(s,previous,spec))
            report["grid"].append(row)
            print(row,flush=True)
            save()
            previous=s
        # 同空间网格的BDF/Radau交叉，检验时间积分，不冒充独立空间模型。
```

```
    cross_times = times if spec.name=="q1" else np.array([0.,10800.,86400.,129600.])
    ref=solve_cached(spec,"bdf81",n_internal=81,sample_times=cross_times,**settings)
    s=solve_cached(spec,"radau81",method="Radau",n_internal=81,sample_times=cross_times,**settings)
    report["time"].append({"q":spec.name,"n":81,**diff(s,ref,spec)})
    save()
# 参数均为方案比较/结构假设扰动，不标记为额外实测数据。
for spec in (Q3,Q4):
    base=solve_cached(spec,"sensitivity_base",n_internal=161,sample_times=[0.],**settings)
    cases=[("55C_20RH",dict(boundary=Boundary.for_problem(spec.name,55.,.20))),
           ("60C_40RH",dict(boundary=Boundary.for_problem(spec.name,60.,.40))),
           ("Lewis_one",dict(lewis=1.))]
    for label,extra in cases:
        s=solve_cached(spec,label,n_internal=161,sample_times=[0.],**settings,**extra)
        row={"q":spec.name,"case":label,"event_h":s.meta["event_time_s"]/3600.,
             "base_h":base.meta["event_time_s"]/3600.,
             "delta_h":(s.meta["event_time_s"]-base.meta["event_time_s"])/3600.}
        report["sensitivity"].append(row)
        print(row,flush=True)
        save()
    s=solve_cached(spec,"no_latent",n_internal=161,sample_times=[0.],latent_heat=False,**settings)
    report["ablation"].append({"q":spec.name,"case":"no_latent","event_h":s.meta["event_time_s"]/3600.,"base_h":base.meta["event_time_s"]/3600.})
    save()
s=solve_cached(Q4,"fixed_radius",n_internal=161,sample_times=[0.],fixed_radius=True,t_end=7*86400.,**settings)
report["ablation"].append({"q":"q4","case":"fixed_radius_same_appendix4","event_h":s.meta["event_time_s"]/3600.,"base_h":base.meta["event_time_s"]/3600.})
report["settings"]=settings
report["production_n"]=1281
report["sampling"]="Q1 every 10 s at all 21 output radii; Q3/Q4 every 60 s through 3 h and hourly from 4 to 39 h, plus continuous termination event; Q4 material grids compared by interpolation."
save()
print(dest,flush=True)
```

```

if __name__=="__main__":
    main()
```

scripts/verify_accuracy_outputs.py

```
"""流式回读全部交付单元格，与未舍入求解缓存核验。"""

import hashlib
import json
import sys

import numpy as np
import openpyxl

from _bootstrap import setup

setup()

from common.io_excel import layout
from common.paths import ROOT
from common.units import cm_to_m, k_to_c


def main():
    fitted = "--fitted" in sys.argv
    questions = [q for q in sys.argv[1:] if q != "--fitted"] or (["q2", "q3", "q4"] if fitted else ["q1", "q2", "q3", "q4"])
    if any(q not in {"q1", "q2", "q3", "q4"} for q in questions):
        raise ValueError("参数只能是q1/q2/q3/q4")
    results_folder = ROOT / "results"
    cache_folder = ROOT / "outputs/cache"
    if fitted:
        results_folder = results_folder / "scenarios/fitted_temperature"
        cache_folder = cache_folder / "fitted_temperature"
    dest = ROOT / "results/validation" / ("fitted-output-check.json" if fitted else "accuracy-output-check.json")
    report = json.loads(dest.read_text(encoding="utf-8")) if dest.exists() else {"files": []}
    report["files"] = [x for x in report["files"] if x["q"] not in questions]
    cached = {}
    for question in questions:
        path = results_folder / f"result{question[-1]}.xlsx"
```

```
        with np.load(cache_folder / f"{question}.npz") as f:
            t, r, T, C = f["t"], f["r"], f["T"], f["C"]
            meta = json.loads(str(f["meta"]))
        if question in {"q2", "q3"}:
            cached[question] = (t, C)
        book = openpyxl.load_workbook(path, read_only=True, data_only=True)
        expected_layout = layout(question)
        assert book.sheetnames == list(expected_layout.sheet_names)
        sheet_records = []
        for name, field in expected_layout.sheets:
            rows = book[name].iter_rows(values_only=True)
            assert list(next(rows)) == expected_layout.expected_columns()
            if question == "q4":
                targets = cm_to_m(np.array([float(x) for x in expected_layout.
distance_headers()[:-1]]))
                expected = np.empty((len(t), 21))
                for i, R in enumerate(meta["R"]):
                    expected[i, :-1] = np.interp(targets, r/r[-1]*R, C[i], right=
np.nan)
                    expected[i, -1] = C[i, -1]
                expected = np.round(expected, 4)
            else:
                expected = np.round(k_to_c(T) if field == "T" else C, 4)
            count = 0
            for i, row in enumerate(rows):
                if i >= len(t):
                    raise AssertionError("输出行数过多")
                assert len(row) == 22 and row[0] == t[i]
                values = np.asarray(row[1:], dtype=float)
                if not np.array_equal(np.isnan(values), np.isnan(expected[i])) or
not np.allclose(
                        values, expected[i], rtol=0, atol=1e-11, equal_nan=True):
                    raise AssertionError(f"{question}/{name}/row {i+2} 数值不一致")
                count += 1
            assert count == len(t)
            assert np.all(np.diff(t) == expected_layout.dt_s)
            sheet_records.append({"sheet": name, "data_rows": count, "columns":
22})
            print(question, name, count, "rows verified", flush=True)
        book.close()
        report["files"].append({"q": question, "sheets": sheet_records,
                                "sha256": hashlib.sha256(path.read_bytes()).
hexdigest()})
        manifest_path = results_folder / f"{question}_manifest.json"
        manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
```

```
        manifest["status"] = "computed_and_readback_verified"
        manifest["readback"] = str(dest.relative_to(ROOT))
        if fitted:
            manifest["validation"] = "results/validation/fitted_numerics.json"
            manifest["temperature_source"] = "results/validation/ambient_fit.json"
        manifest_path.write_text(json.dumps(manifest, indent=2, ensure_ascii=False
    ), encoding="utf-8")
    if "q2" in cached and "q3" in cached:
        t2, c2 = cached["q2"]
        t3, c3 = cached["q3"]
        mask = t3 <= t2[-1]
        indices = np.searchsorted(t2, t3[mask])
        assert np.array_equal(t2[indices], t3[mask])
        delta = float(np.max(np.abs(c2[indices]-c3[mask])))
        assert delta < 1e-8
        report["q2_q3_common_time_max_C_difference"] = delta
    dest.write_text(json.dumps(report, indent=2, ensure_ascii=False), encoding="
    utf-8")
    print("Requested output cells verified:", questions, flush=True)


if __name__ == "__main__":
    main()
```

tests/test_accuracy.py

```
"""独立实际边界解析解、隐式算法交叉及细网格启动回归。"""

import sys
import unittest
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT / "scripts"))

from common.problem_spec import Q1
from common.radial import solve_radial
from validate_accuracy import bessel_heat


class AccuracyTests(unittest.TestCase):
```

```
    def test_actual_boundary_heat_converges_to_independent_modes(self):
        times = np.array([60, 100, 300, 900, 1800.0])
        opts = dict(sample_times=times, rtol=2e-11, atol=2e-12,
                    align_boundary_knots=True, latent_heat=False)
        coarse = solve_radial(Q1, n_internal=81, **opts)
        fine = solve_radial(Q1, n_internal=161, **opts)
        exact = bessel_heat(times, fine.r)
        error_coarse = np.max(np.abs(coarse.T-exact))
        error_fine = np.max(np.abs(fine.T-exact))
        self.assertLess(error_fine, error_coarse/3)
        self.assertLess(error_fine, 1.5e-4)

    def test_boundary_knots_radau_matches_bdf(self):
        opts = dict(n_internal=41, t_end=300, sample_times=[0, 60, 61, 300],
                    rtol=2e-11, atol=2e-12, align_boundary_knots=True)
        bdf = solve_radial(Q1, **opts)
        radau = solve_radial(Q1, method="Radau", **opts)
        np.testing.assert_allclose(radau.T, bdf.T, rtol=0, atol=2e-7)
        np.testing.assert_allclose(radau.C, bdf.C, rtol=0, atol=2e-8)

    def test_fine_surface_startup_remains_positive(self):
        sol = solve_radial(Q1, n_internal=5121, t_end=1, sample_times=[0, 1],
                           rtol=2e-11, atol=2e-12, align_boundary_knots=True)
        self.assertGreater(float(np.min(sol.C)), 0)
        self.assertLess(sol.meta["mass_balance_relative"], 1e-11)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

tests/test_framework.py

```
"""框架级冒烟测试。

这些测试验证的是**框架本身是通的**：路径、单位、数据读取、格式校验。
**不涉及任何物理求解** —— 求解器的测试由 @impl 在实现后补（见 docs/OWNERS.md §2）。

跑法：  make check    （或 make test-fast）
"""

from __future__ import annotations

import numpy as np
import pandas as pd
```

```python
import pytest

from common import io_excel, paths, problem_spec, units
from common.solution import Solution


# --- 路径 ----------------------------------------------------------------


def test_root_is_repo_root():
    assert (paths.ROOT / "AGENTS.md").exists(), "ROOT 定位错误"
    assert (paths.ROOT / "problem" / "A题.pdf").exists(), "题目原件缺失"


def test_template_and_result_paths():
    assert paths.template_path("q1").name == "result1.xlsx"
    assert paths.template_path("result3").name == "result3.xlsx"
    assert paths.result_path("4").name == "result4.xlsx"
    with pytest.raises(ValueError):
        paths.template_path("q9")


def test_rel_is_relative():
    assert paths.rel(paths.ROOT / "AGENTS.md") == "AGENTS.md"


# --- 单位 ----------------------------------------------------------------


def test_unit_roundtrip():
    assert units.m_to_cm(units.cm_to_m(2.0)) == pytest.approx(2.0)
    assert units.k_to_c(units.c_to_k(28.0)) == pytest.approx(28.0)
    assert units.h_to_s(1.0) == pytest.approx(3600.0)
    assert units.c_to_k(28.0) == pytest.approx(301.15)


def test_unit_conversion_on_arrays():
    arr = units.cm_to_m(np.array([0.0, 0.5, 2.0]))
    assert arr.shape == (3,)
    assert arr[2] == pytest.approx(0.02)


def test_assert_kelvin_rejects_celsius():
    """把摄氏度当开尔文传是本题最危险的错误，必须有防呆。"""
    with pytest.raises(ValueError, match="摄氏度"):
```

```
        units.assert_kelvin(np.array([28.0, 50.0]))
    units.assert_kelvin(np.array([301.15, 323.15]))  # 不抛


def test_assert_kelvin_rejects_nan():
    with pytest.raises(ValueError, match="NaN"):
        units.assert_kelvin(np.array([300.0, np.nan]))


def test_wet_basis():
    from common.correlations import wet_basis

    assert wet_basis(0.0) == pytest.approx(0.0)
    assert wet_basis(1.0) == pytest.approx(0.5)
    assert wet_basis(2.55) == pytest.approx(2.55 / 3.55)


# --- 题目常量 ----------------------------------------------------------------


def test_spec_constants_match_problem():
    """题目给定量必须与 problem/A题.pdf 一致。改动这里请先核对题目。"""
    assert problem_spec.LENGTH_M == pytest.approx(0.25)
    assert problem_spec.RADIUS0_M == pytest.approx(0.02)
    assert problem_spec.INITIAL_T_C == pytest.approx(28.0)
    assert problem_spec.INITIAL_C_DRY == pytest.approx(2.55)
    assert problem_spec.TARGET_C_DRY == pytest.approx(0.15)
    assert problem_spec.Q1_T_END_S == pytest.approx(1800.0)


def test_geometry_aspect_ratio():
    g = problem_spec.Geometry()
    assert g.aspect_ratio == pytest.approx(25.0 / 4.0)


def test_geometry_rejects_short_cylinder():
    """长径比过小意味着一维径向简化（D-007）不成立，应拒绝。"""
    with pytest.raises(ValueError, match="长径比"):
        problem_spec.Geometry(length=0.05, radius0=0.02)


def test_all_specs_defined_and_consistent():
    assert set(problem_spec.ALL) == {"q1", "q2", "q3", "q4"}
    for spec in problem_spec.ALL.values():
        n_t, n_r = spec.grid.shape
```

```
        assert n_t > 1 and n_r > 1
        assert spec.grid.r[0] == 0.0, "交付网格必须包含轴心 r=0"
        assert spec.grid.r[-1] == pytest.approx(spec.geometry.radius0)


def test_spec_property_sets_do_not_mix():
    """附录 2/3/4 不可混用（AGENTS.md §4.3）。"""
    assert problem_spec.Q1.properties.appendix == 2
    assert problem_spec.Q2.properties.appendix == 3
    assert problem_spec.Q3.properties.appendix == 3
    assert problem_spec.Q4.properties.appendix == 4
    assert problem_spec.Q1.uses_shrinkage is False
    assert problem_spec.Q4.uses_shrinkage is True


def test_q1_grid_matches_delivery_requirement():
    """问题 1：交付 0~1800 s 每隔 1 s、0~2 cm 每隔 0.1 cm。"""
    g = problem_spec.Q1.grid
    assert g.shape == (1801, 21)


# --- 附件读取 ----------------------------------------------------------------


def test_read_attachment1_structure():
    df = io_excel.read_attachment(1)
    assert list(df.columns) == ["时间", "温度", "水分浓度"]
    assert len(df) == 241
    t = df["时间"].to_numpy(dtype=float)
    assert t[0] == 0.0
    assert t[-1] == pytest.approx(14400.0)
    assert np.allclose(np.diff(t), 60.0)


def test_read_attachment2_structure():
    df = io_excel.read_attachment(2)
    assert list(df.columns) == ["时间", "半径"]
    assert len(df) == 145
    t = df["时间"].to_numpy(dtype=float)
    assert t[-1] == pytest.approx(259200.0)
    r = df["半径"].to_numpy(dtype=float)
    assert r[0] == pytest.approx(2.0)
    assert np.all(np.diff(r) <= 0), "半径应单调不增"
```

```python
def test_ambient_and_herb_concentration_magnitudes_differ():
    """D-011: 烘房水分浓度与药材初始值差一个数量级以上，基准问题必须论证。"""
    df = io_excel.read_attachment(1)
    assert df["水分浓度"].max() < problem_spec.INITIAL_C_DRY / 10


def test_ambient_coverage_is_shorter_than_process():
    """D-009: 附件 1 只覆盖 4 h，而烘干过程 72 h，存在信息缺口。"""
    df = io_excel.read_attachment(1)
    assert df["时间"].max() == pytest.approx(problem_spec.AMBIENT_COVERAGE_S)
    assert problem_spec.AMBIENT_COVERAGE_S < problem_spec.RADIUS_COVERAGE_S


def test_radius_plateaus_at_end():
    """D-010: 附件 2 末段半径出现平台，需在建模时解释。"""
    df = io_excel.read_attachment(2)
    r = df["半径"].to_numpy(dtype=float)
    assert r[-1] == pytest.approx(r[-2])


# --- 交付模板与格式校验 ------------------------------------------------------


@pytest.mark.parametrize("problem", ["q1", "q2", "q3", "q4"])
def test_layout_matches_real_template(problem):
    """实测布局必须与 problem/附件/附件3 的真实模板一致。

    这是**防止赛方更新模板而我们没跟上**的守门测试。
    失败时跑 `make probe` 看差异，然后更新 io_excel.LAYOUTS 的实测结论。
    """
    mism = io_excel.verify_layout_against_template(problem)
    assert not mism, "布局与模板不一致：\n  - " + "\n  - ".join(mism)


def test_template_sheet_names_are_as_measured():
    """result1/2 是 温度+水分浓度；result3/4 的工作表名是 Sheet1（实测，不是猜的）。"""
    assert set(io_excel.layout("q1").sheet_names) == {"温度", "水分浓度"}
    assert set(io_excel.layout("q2").sheet_names) == {"温度", "水分浓度"}
    assert set(io_excel.layout("q3").sheet_names) == {"Sheet1"}
    assert set(io_excel.layout("q4").sheet_names) == {"Sheet1"}


def test_result3_and_4_have_no_temperature_sheet():
    """result3/4 只有水分浓度 —— 多写一个温度表会被校验拦下。"""
    assert all(f == "C" for _, f in io_excel.layout("q3").sheets)
```

```python
    assert all(f == "C" for _, f in io_excel.layout("q4").sheets)


def test_distance_columns_are_full_01cm_grid():
    """模板里的 0, 0.1, 0.2, …, 2 是示意（'…' 是省略），实际应为 21 列。"""
    heads = io_excel.layout("q1").distance_headers()
    assert len(heads) == 21
    assert heads[0] == "0"
    assert heads[1] == "0.1"
    assert heads[-1] == "2"


def test_result4_last_column_is_surface_text():
    """问题 4 的末列是文字「药材表面」，不是数值 2。"""
    heads = io_excel.layout("q4").distance_headers()
    assert heads[-1] == io_excel.SURFACE_HEADER
    assert heads[-2] == "1.9"


def test_delivery_time_unit_is_seconds():
    """实测：模板 A 列数据是 1 或 60（秒），**不是**题目表 3~表 6 表头写的 h。"""
    assert io_excel.layout("q1").dt_s == 1.0
    assert io_excel.layout("q2").dt_s == 1.0
    assert io_excel.layout("q3").dt_s == 60.0
    assert io_excel.layout("q4").dt_s == 60.0


def test_result_sheets_columns_match_layout():
    """export 生成的工作表列结构必须与布局一致，否则写出去会被校验拦下。"""
    from common import export

    sol = _dummy_solution(n_t=1801)
    sheets = export.result_sheets(sol, problem_spec.Q1)
    for name, df in sheets.items():
        assert list(df.columns) == io_excel.layout("q1").expected_columns(), name
    assert sheets["水分浓度"].shape == (1801, 22)  # 1 列时间 + 21 列距离


def test_result_sheets_q4_surface_column():
    from common import export

    sol = _dummy_solution(n_t=101, radius=problem_spec.RADIUS0_M)
    sheets = export.result_sheets(sol, problem_spec.Q4)
    df = sheets["Sheet1"]
    assert list(df.columns) == io_excel.layout("q4").expected_columns()
```

```
    assert io_excel.SURFACE_HEADER in df.columns


def test_write_result_rejects_wrong_sheet_names(tmp_path, monkeypatch):
    """格式校验必须是硬失败，不能静默写出错误交付物（AGENTS.md 铁律 3）。"""
    import pandas as pd

    monkeypatch.setattr(paths, "RESULTS_DIR", tmp_path)
    with pytest.raises(io_excel.ResultFormatError, match="工作表名不匹配"):
        io_excel.write_result("q1", {"错的表名": pd.DataFrame({io_excel.TIME_HEADER
    : [1.0]})})


def test_write_result_rejects_wrong_column_count(tmp_path, monkeypatch):
    """模板只有 4 个距离列是示意；实际必须写满 21 列，少写要报错。"""
    import pandas as pd

    monkeypatch.setattr(paths, "RESULTS_DIR", tmp_path)
    short = {io_excel.TIME_HEADER: [1.0, 2.0]}
    for h in ["0", "0.1", "0.2", "2"]:
        short[h] = [0.0, 0.0]
    sheets = {n: pd.DataFrame(short) for n in io_excel.layout("q1").sheet_names}
    with pytest.raises(io_excel.ResultFormatError, match="列结构不匹配"):
        io_excel.write_result("q1", sheets)


def test_write_result_rejects_nan(tmp_path, monkeypatch):
    from common import export

    monkeypatch.setattr(paths, "RESULTS_DIR", tmp_path)
    sol = _dummy_solution(n_t=1801)
    sheets = export.result_sheets(sol, problem_spec.Q1)
    sheets["水分浓度"].iloc[3, 1] = float("nan")
    with pytest.raises(io_excel.ResultFormatError, match="NaN"):
        io_excel.write_result("q1", sheets)


def test_write_result_accepts_valid_sheets(tmp_path, monkeypatch):
    """正例：合规的工作表应能写出，且文件真实存在。"""
    from common import export

    monkeypatch.setattr(paths, "RESULTS_DIR", tmp_path)
    sol = _dummy_solution(n_t=1801)
    dest = io_excel.write_result("q1", export.result_sheets(sol, problem_spec.Q1))
    assert dest.exists()
```

```
    back = pd.read_excel(dest, sheet_name=None)
    assert set(back) == {"温度", "水分浓度"}
    assert back["水分浓度"].shape == (1801, 22)


def test_write_result_rejects_uneven_time_step(tmp_path, monkeypatch):
    from common import export

    monkeypatch.setattr(paths, "RESULTS_DIR", tmp_path)
    sol = _dummy_solution(n_t=1801)
    t = sol.t.copy()
    t[5] += 0.5
    sol.t = t
    with pytest.raises(io_excel.ResultFormatError, match="时间步长"):
        io_excel.write_result("q1", export.result_sheets(sol, problem_spec.Q1))


# --- Solution ------------------------------------------------------------


def _dummy_solution(
    t_end_s: float = 1800.0,
    n_t: int = 19,
    n_r: int = 21,
    radius: float = problem_spec.RADIUS0_M,
) -> Solution:
    t = np.linspace(0.0, t_end_s, n_t)
    r = np.linspace(0.0, radius, n_r)
    T = np.full((n_t, n_r), 300.0)
    C = np.linspace(2.55, 0.10, n_t)[:, None] * np.ones((1, n_r))
    return Solution(t=t, r=r, T=T, C=C)


def test_solution_shape_validation():
    with pytest.raises(ValueError, match="形状不一致"):
        Solution(
            t=np.zeros(3),
            r=np.zeros(4),
            T=np.zeros((3, 4)),
            C=np.zeros((3, 5)),
        )


def test_solution_first_time_below_uses_max_over_r():
    """终点判据取 max_r C（题目说"各处"），不是平均值。"""
```

```
    sol = _dummy_solution()
    t_hit = sol.first_time_below(0.15)
    assert t_hit == pytest.approx(sol.t[-1])


def test_solution_first_time_below_raises_when_never_reached():
    """达不到终点时必须报错，不许静默返回（AGENTS.md 铁律 10）。"""
    sol = _dummy_solution()
    with pytest.raises(ValueError, match="从未低于"):
        sol.first_time_below(0.05)


def test_solution_table_grid_is_21_by_21_for_q1():
    sol = _dummy_solution(n_t=1801)
    assert sol.shape == (1801, 21)
    assert sol.surface_index == 20


# --- 未实现的东西必须显式失败 ----------------------------------------------


def test_property_sets_match_appendices():
    """原题三套密度关系，独立点值核验。"""
    from common import correlations

    for cls, expected in zip((correlations.Appendix2, correlations.Appendix3,
                              correlations.Appendix4),(820,778,850)):
        ps = cls()
        assert ps.rho(1.0,300.0) == pytest.approx(expected)


def test_solvers_have_callable_entrypoints():
    """数值验证在test_numerics.py，框架只检查入口。"""
    for n in (1, 2, 3, 4):
        mod = __import__(f"q{n}.solver", fromlist=["solve"])
        assert callable(mod.solve)


def test_latent_heat_requires_sourced_value():
    """FAO56式(3-1)的独立数值点值。"""
    from common import correlations

    assert float(correlations.latent_heat_vaporization(333.15)) == pytest.approx
    (2359340.)
```

tests/test_numerics.py

```
"""独立解析解、守恒、边界、变域及输出掩码验证；可直接用unittest。"""
import sys
import unittest
from pathlib import Path

import numpy as np
from scipy.optimize import brentq
from scipy.special import j0, j1

sys.path.insert(0, str(Path(__file__).resolve().parents[1]/"src"))
from common import correlations, export, io_excel
from common.boundary import Boundary
from common.problem_spec import Q1, Q3, Q4
from common.radial import RadialModel, outward_flux, solve_radial
from common.psychrometrics import humidity_ratio, gas_evaporation_capacity


class NumericalTests(unittest.TestCase):
    def test_property_units_and_zero_limit(self):
        for p in (correlations.Appendix2(),correlations.Appendix3(),correlations.
    Appendix4()):
            self.assertEqual(float(p.D(0,300)),0)
            with self.assertRaises(ValueError):
                p.D(1,50)
        self.assertAlmostEqual(float(correlations.Appendix2().D(2.55,301.15)),
                               7e-9*np.exp(-0.89/2.55),places=18)
        self.assertAlmostEqual(float(correlations.Appendix3().D(1,300)),
                               2.4e-3*np.exp(-0.45-3850/300),places=18)
        self.assertAlmostEqual(float(correlations.Appendix4().D(1,300)),
                               4.2e-4*np.exp(-0.30-3850/300),places=18)

    def test_boundary_sign_and_equilibrium(self):
        self.assertLess(outward_flux(300,320,25),0)
        b=Boundary.from_attachments()
        b.T_amb[:]=Q1.initial.T
        b.C_amb[:]=humidity_ratio(Q1.initial.T, 1.0)
        m=RadialModel(Q1,41,b)
        np.testing.assert_allclose(m.rhs(600,m.initial()),0,atol=1e-13)

    def test_instantaneous_mass_and_heat_balance(self):
        for spec in (Q1,Q3,Q4):
            m=RadialModel(spec,41)
            y=m.initial()
```

```
            y[:41]+=2*m.xi**2
            y[41:82]-=0.2*m.xi**2
            dy=m.rhs(7200,y)
            self.assertLess(abs(m.weights@dy[41:82]+dy[82]),1e-16)
            capacity=spec.properties.rho(y[41:82],y[:41])*spec.properties.cp(y[41:82],y[:41])
            self.assertLess(abs(m.radius(7200)**2*np.dot(m.weights*capacity,dy[:41])-dy[83]),1e-12)

    def test_jacobian_directional_derivative(self):
        m=RadialModel(Q3,21)
        y=m.initial()
        y[:21]+=m.xi
        y[21:42]-=m.xi/4
        d=np.random.default_rng(2026).normal(size=len(y))
        eps=1e-5
        reference=(m.rhs(500,y+eps*d)-m.rhs(500,y-eps*d))/(2*eps)
        np.testing.assert_allclose(m.jacobian(500,y)@d,reference,rtol=5e-5,atol=2e-7)

    def test_constant_radius_reduces_to_fixed_model(self):
        b=Boundary.from_attachments()
        b.R[:]=Q4.geometry.radius0
        moving=RadialModel(Q4,41,b)
        fixed=RadialModel(Q4,41,b,fixed_radius=True)
        y=moving.initial()
        y[:41]+=moving.xi
        np.testing.assert_allclose(moving.rhs(3600,y),fixed.rhs(3600,y),atol=0)

    def test_closed_system_conservation_and_bessel_heat_solution(self):
        b=Boundary.from_attachments()
        b.T_amb[:]=Q1.initial.T+10
        b.C_amb[:]=Q1.initial.C
        times=np.array([100,600,1800.])
        s=solve_radial(Q1,boundary=b,n_internal=321,sample_times=times,
                       moisture_boundary="legacy_effective")
        # 圆柱第三类边界解析级数，独立于有限体积实现。
        p=Q1.properties
        k=float(p.k(1,300)); alpha=k/float(p.rho(1,300)*p.cp(1,300))
        Bi=correlations.h_conv("q1")*Q1.geometry.radius0/k
        f=lambda x:x*j1(x)-Bi*j0(x)
        scan=np.linspace(1e-8,130,20000)
        roots=[brentq(f,a,b) for a,b in zip(scan[:-1],scan[1:]) if f(a)*f(b)<0]
        expected=np.zeros_like(s.T)
        for z in roots:
```

```
            A=2*j1(z)/(z*(j0(z)**2+j1(z)**2))
            expected+=A*np.exp(-alpha*z*z*times[:,None]/Q1.geometry.radius0**2)*j0(z*s.r/Q1.geometry.radius0)
        exact=Q1.initial.T+10-10*expected
        self.assertLess(float(np.max(np.abs(s.T-exact))),2e-4)
        np.testing.assert_allclose(s.C,Q1.initial.C,atol=1e-11)
        self.assertLess(s.meta["mass_balance_relative"],1e-10)
        self.assertLess(s.meta["q1_energy_balance_relative"],1e-8)

    def test_q4_outside_domain_mask_is_strict(self):
        s=solve_radial(Q4,n_internal=41,seek_end=False,t_end=1800,sample_times=[0,60,120])
        df=export.result_sheets(s,Q4)
        io_excel._validate("q4",df,io_excel.layout("q4"))
        df["Sheet1"].iloc[1,1]=np.nan
        with self.assertRaises(io_excel.ResultFormatError):
            io_excel._validate("q4",df,io_excel.layout("q4"))

    def test_radius_extrapolation_rejected(self):
        with self.assertRaises(ValueError):
            Boundary.from_attachments().radius(259201)

    def test_dewpoint_prevents_evaporation_and_condensation_releases_heat(self):
        b = Boundary.from_attachments()
        b.T_amb[:] = 310.0
        b.C_amb[:] = humidity_ratio(300.0, 1.0)
        m = RadialModel(Q1, 21, b)
        moisture, _, latent = m.surface_fluxes(0, 299.0, 2.55)
        self.assertLess(moisture, 0)
        self.assertLess(latent, 0)
        self.assertAlmostEqual(gas_evaporation_capacity(300., b.C_amb[0], 25.), 0.)

    def test_latent_heat_is_coupled_in_all_questions(self):
        for spec in (Q1, Q3, Q4):
            m = RadialModel(spec, 21)
            y = m.initial()
            y[:21] += 10.
            with_latent = m.rhs(0., y)
            m.latent_heat = False
            without_latent = m.rhs(0., y)
            self.assertLess(with_latent[20], without_latent[20])
            np.testing.assert_array_equal(with_latent[21:42], without_latent[21:42])

```

```
    def test_dry_solid_mass_preserved_during_shrinkage(self):
        m = RadialModel(Q4, 21)
        for t in (0., 3600., 86400., 259200.):
            self.assertAlmostEqual(m.dry_density(t)*m.radius(t)**2,
                                   m.rho_d0*Q4.geometry.radius0**2)


if __name__=="__main__":
    unittest.main(verbosity=2)
```

paper/build_sources.py

```
"""Build the manuscript appendix from the actual current source files."""
from pathlib import Path

root = Path(__file__).resolve().parents[1]
paths = [root / 'run_all.py'] + sorted((root / 'src').rglob('*.py'))
exclude = {'make_accuracy_report.py',
           'validate_numerics.py', 'validate_q4_accuracy.py',
           'verify_accuracy_reproduction.py'}
paths += [p for p in sorted((root / 'scripts').glob('*.py')) if p.name not in
    exclude]
paths += sorted((root / 'tests').glob('*.py'))
paths += [Path(__file__).resolve()]
lines = ['% Generated by paper/build_sources.py; source is included directly.']
for p in paths:
    if not p.is_file() or not p.read_text(encoding='utf-8-sig').strip():
        continue
    rel = p.relative_to(root).as_posix()
    safe = rel.replace('_', r'\_')
    lines += [r'\subsubsection*{\texttt{' + safe + '}}',
              r'\lstinputlisting[language=Python]{../' + rel + '}']
(root / 'paper' / 'source_listings.tex').write_text('\n'.join(lines) + '\n',
    encoding='utf-8')
print(f'Included {sum(x.startswith(chr(92)+"lstinputlisting") for x in lines)}
    source files')
```